

Getting Started

Thank you for choosing Freenove products!

After you download the ZIP file we provide. Unzip it and you will get a folder contains several files and folders. There are three PDF files:

- **Tutorial.pdf**
It contains basic operations such as installing system for Raspberry Pi.
The code in this PDF is in C.
- **Tutorial_GPIOZero.pdf**
It contains basic operations such as installing system for Raspberry Pi.
The code in this PDF is in Python.
- **Processing.pdf** in Freenove_Ultrasonic_Starter_Kit_for_Raspberry_Pi\Processing
The code in this PDF is in Java.

We recommend you to start with **Tutorial.pdf** or **Tutorial_GPIOZero.pdf** first.

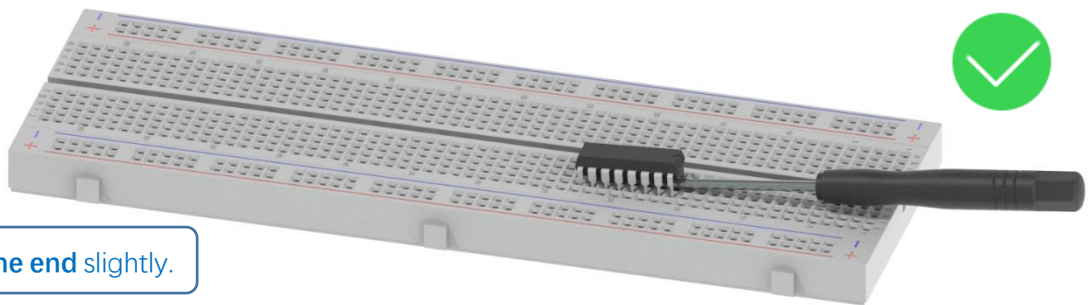
If you want to start with Processing.pdf or skip some chapters of Tutorial.pdf, you need to finish necessary steps in **Chapter 7 AD/DA** of **Tutorial.pdf** first.

Remove the Chips

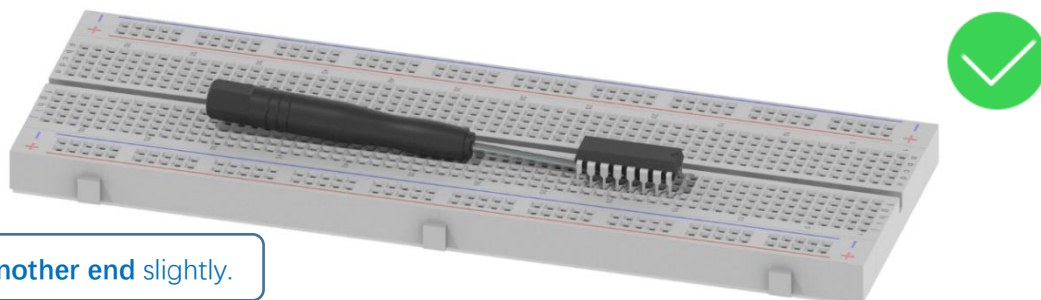
Some chips and modules are inserted into the breadboard to protect their pins.

You need to remove them from breadboard before use. (There is no need to remove GPIO Extension Board.)

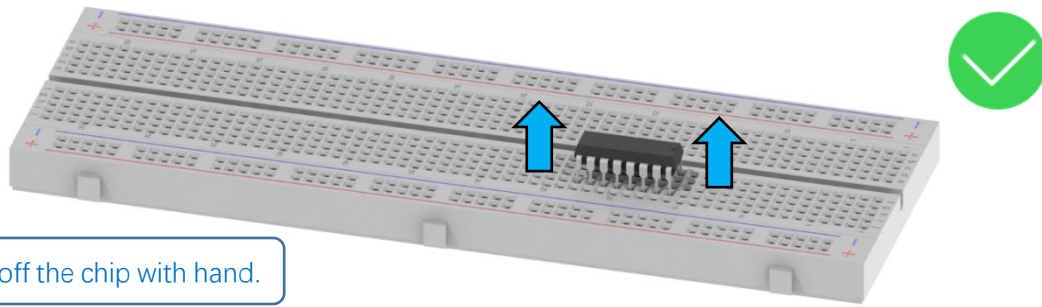
Please find a tool (like a little screw driver) to handle them like below:



Step 1, lift **one end** slightly.



Step 2, lift **another end** slightly.



Avoid lifting one end with big angle directly.



Get Support and Offer Input

Freenove provides free and responsive product and technical support, including but not limited to:

- Product quality issues
- Product use and build issues
- Questions regarding the technology employed in our products for learning and education
- Your input and opinions are always welcome
- We also encourage your ideas and suggestions for new products and product improvements

For any of the above, you may send us an email to:

support@freenove.com

Safety and Precautions

Please follow the following safety precautions when using or storing this product:

- Keep this product out of the reach of children under 6 years old.
- This product should be used only when there is adult supervision present as young children lack necessary judgment regarding safety and the consequences of product misuse.
- This product contains small parts and parts, which are sharp. This product contains electrically conductive parts. Use caution with electrically conductive parts near or around power supplies, batteries and powered (live) circuits.
- When the product is turned ON, activated or tested, some parts will move or rotate. To avoid injuries to hands and fingers, keep them away from any moving parts!
- It is possible that an improperly connected or shorted circuit may cause overheating. Should this happen, immediately disconnect the power supply or remove the batteries and do not touch anything until it

cools down! When everything is safe and cool, review the product tutorial to identify the cause.

- Only operate the product in accordance with the instructions and guidelines of this tutorial, otherwise parts may be damaged or you could be injured.
- Store the product in a cool dry place and avoid exposing the product to direct sunlight.
- After use, always turn the power OFF and remove or unplug the batteries before storing.

About Freenove

Freenove provides open source electronic products and services worldwide.

Freenove is committed to assist customers in their education of robotics, programming and electronic circuits so that they may transform their creative ideas into prototypes and new and innovative products. To this end, our services include but are not limited to:

- Educational and Entertaining Project Kits for Robots, Smart Cars and Drones
- Educational Kits to Learn Robotic Software Systems for Arduino, Raspberry Pi and micro: bit
- Electronic Component Assortments, Electronic Modules and Specialized Tools
- **Product Development and Customization Services**

You can find more about Freenove and get our latest news and updates through our website:

<http://www.freenove.com>

Copyright

All the files, materials and instructional guides provided are released under [Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License](#). A copy of this license can be found in the folder containing the Tutorial and software files associated with this product.



This means you can use these resource in your own derived works, in part or completely, but **NOT for the intent or purpose of commercial use.**

Freenove brand and logo are copyright of Freenove Creative Technology Co., Ltd. and cannot be used without written permission.



Raspberry Pi® is a trademark of Raspberry Pi Foundation (<https://www.raspberrypi.org/>).

Contents

Getting Started	I
Remove the Chips.....	I
Safety and Precautions.....	II
About Freenove	III
Copyright	III
Contents	IV
Preface	1
Raspberry Pi	2
Chapter 0 Preparation	9
Linux Command	9
Install GPIO Zero Python library.....	12
Obtain the Project Code	13
Python2 & Python3	14
Chapter 1 LED	17
Project 1.1 Blink	17
Freenove Car, Robot and other products for Raspberry Pi	32
Chapter 2 Buttons & LEDs	33
Project 2.1 Push Button Switch & LED	33
Project 2.2 MINI Table Lamp	37
Chapter 3 LED Bar Graph	40
Project 3.1 Flowing Water Light	40
Chapter 4 Analog & PWM	44
Project 4.1 Breathing LED	44
Chapter 5 RGB LED	49
Project 5.1 Multicolored LED	50
Chapter 6 Buzzer	53
Project 6.1 Doorbell	53
Project 6.2 Alertor	59
(Important) Chapter 7 ADC	61
Project 7.1 Read the Voltage of Potentiometer	61
Chapter 8 Potentiometer & LED	73
Project 8.1 Soft Light	73
Chapter 9 Potentiometer & RGBLED	78
Project 9.1 Colorful Light	78
Chapter 10 Photoresistor & LED	83
Project 10.1 NightLamp	83
Chapter 11 Thermistor	89
Project 11.1 Thermometer	89
Chapter 12 Joystick	95
Project 12.1 Joystick	95

Chapter 13 Motor & Driver	101
Project 13.1 Control a DC Motor with a Potentiometer	101
Chapter 14 Relay & Motor	112
Project 14.1.1 Relay & Motor	112
Chapter 15 Servo.....	118
Project 15.1 Servo Sweep	118
Chapter 16 Stepper Motor	125
Project 16.1 Stepper Motor	125
Chapter 17 74HC595 & Bar Graph LED	133
Project 17.1 Flowing Water Light.....	133
Chapter 18 74HC595 & 7-Segment Display	139
Project 18.1 7-Segment Display	139
Chapter 19 74HC595 & LED Matrix.....	144
Project 19.1 LED Matrix.....	144
Chapter 20 LCD1602	152
Project 20.1 I2C LCD1602.....	152
Chapter 21 Ultrasonic Ranging	157
Project 21.1 Ultrasonic Ranging	157
Chapter 22 Web IoT	163
Project 22.1 Remote LED	163
What's Next?	169

Preface

Raspberry Pi is a low cost, **credit card sized computer** that plugs into a computer monitor or TV, and uses a standard keyboard and mouse. It is an incredibly capable little device that enables people of all ages to explore computing, and to learn how to program in a variety of computer languages like Scratch and Python. It is capable of doing everything you would expect from a desktop computer, such as browsing the internet, playing high-definition video content, creating spreadsheets, performing word-processing, and playing video games. For more information, you can refer to Raspberry Pi official [website](#). For clarification, this tutorial will also reference Raspberry Pi as RPi, RPI and RasPi.

In this tutorial, most chapters consist of **Components List**, **Component Knowledge**, **Circuit**, and **Code** (**Python** code). We provide Python code for each project in this tutorial. After completing this tutorial, you can learn Java by reading Processing.pdf.

This kit does not contain [Raspberry and its accessories](#). You can also use the components and modules in this kit to create projects of your own design.

Additionally, if you encounter any issues or have questions about this tutorial or the contents of kit, you can always contact us for free technical support at:

support@freenove.com

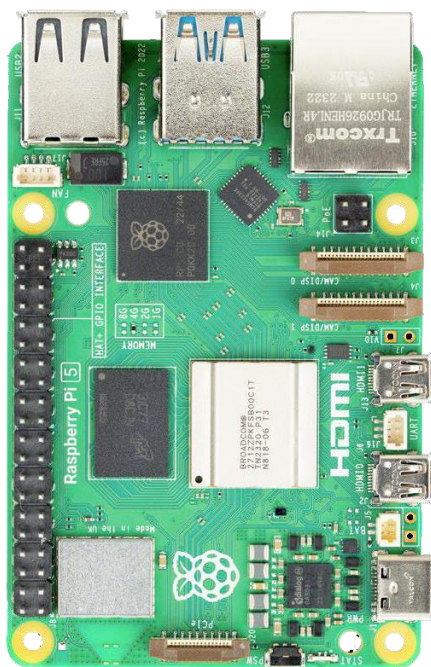
Raspberry Pi

So far, at this writing, Raspberry Pi has advanced to its fifth generation product offering. Version changes are accompanied by increases in upgrades in hardware and capabilities.

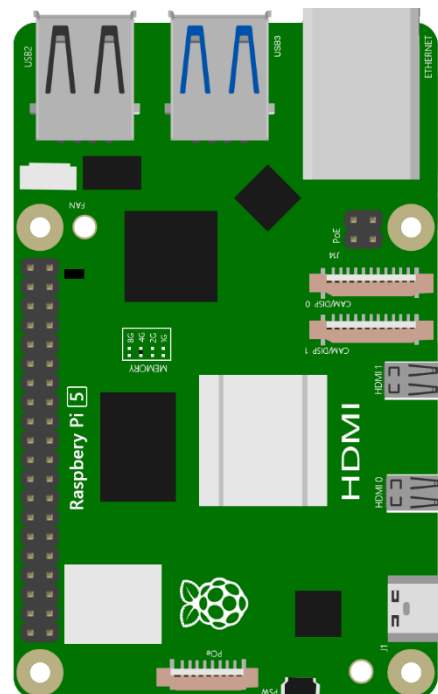
The A type and B type versions of the first generation products have been discontinued due to various reasons. What is most important is that other popular and currently available versions are consistent in the order and number of pins and their assigned designation of function, making compatibility of peripheral devices greatly enhanced between versions.

Below are the raspberry pi pictures and model pictures supported by this product. They have 40 pins.

Practicality picture of Raspberry Pi 5:



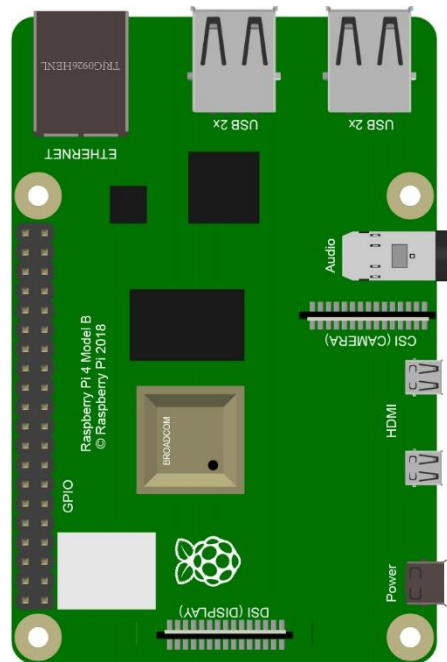
Model diagram of Raspberry Pi 5:



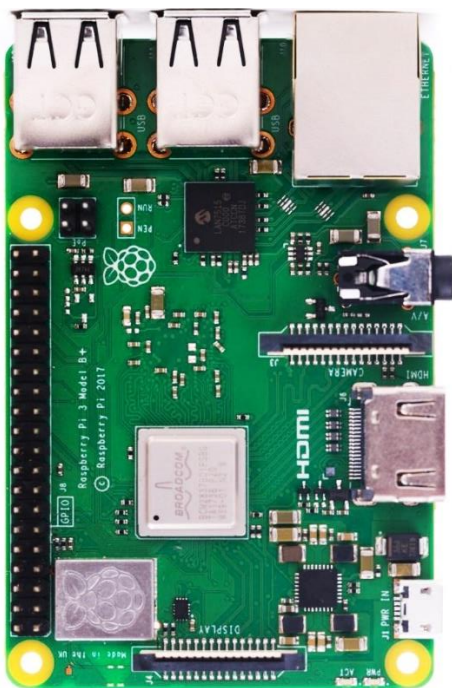
Actual image of Raspberry Pi 4 Model B:



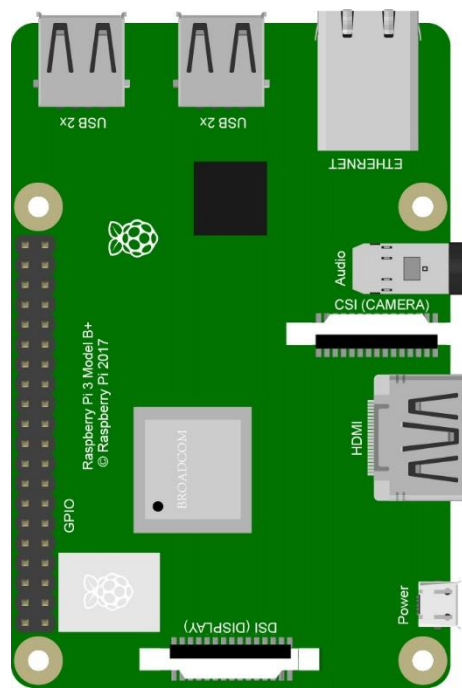
CAD image of Raspberry Pi 4 Model B:



Actual image of Raspberry Pi 3 Model B+:



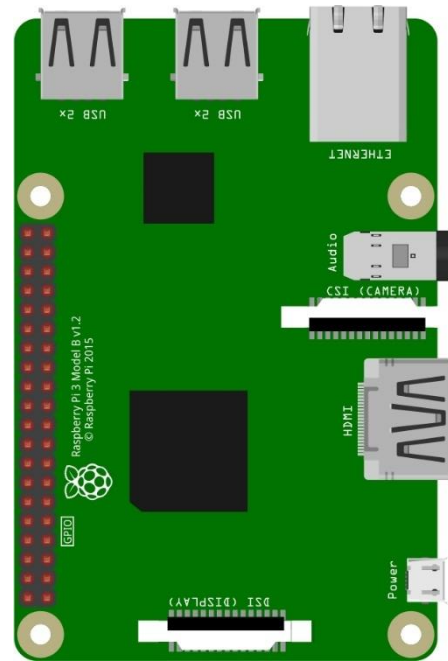
CAD image of Raspberry Pi 3 Model B+:



Actual image of Raspberry Pi 3 Model B:



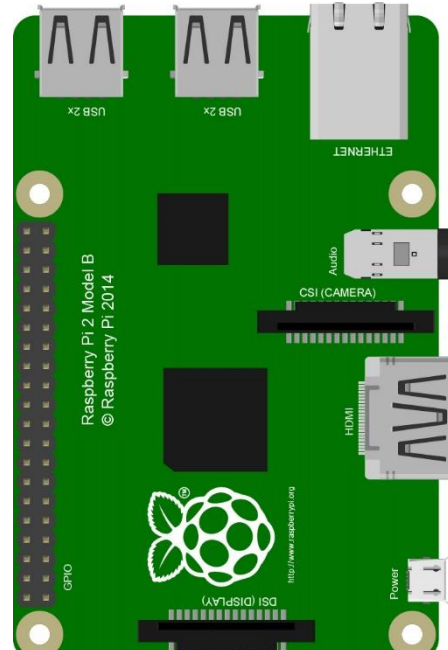
CAD image of Raspberry Pi 3 Model B:



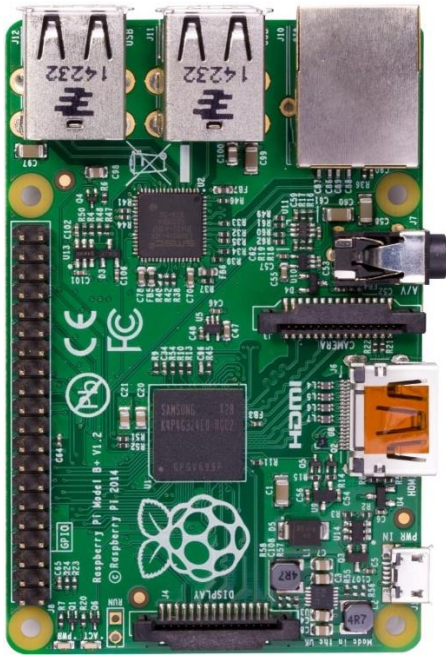
Actual image of Raspberry Pi 2 Model B:



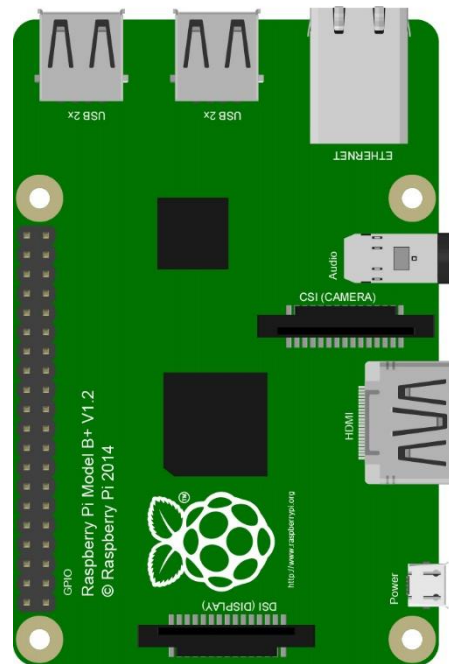
CAD image of Raspberry Pi 2 Model B:



Actual image of Raspberry Pi 1 Model B+:



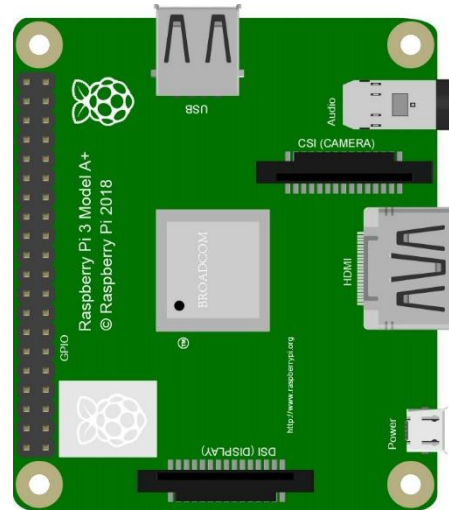
CAD image of Raspberry Pi 1 Model B+:



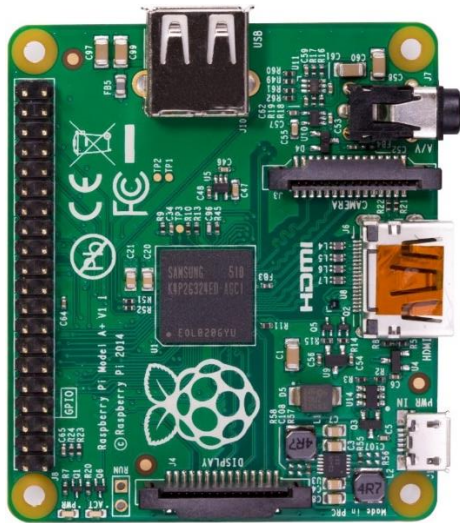
Actual image of Raspberry Pi 3 Model A+:



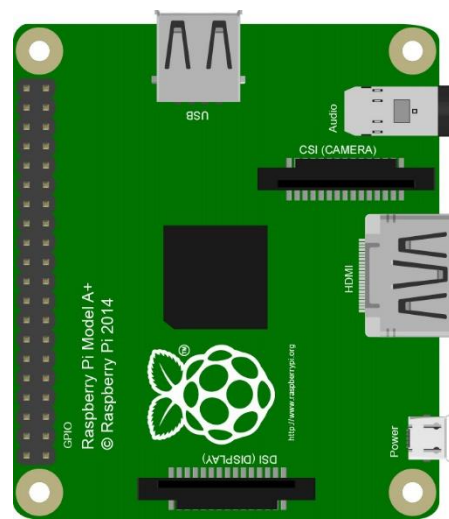
CAD image of Raspberry Pi 3 Model A+:



Actual image of Raspberry Pi 1 Model A+:



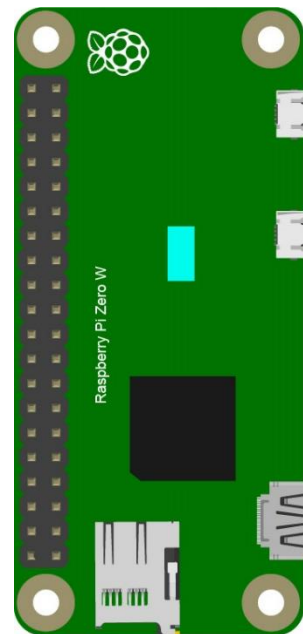
CAD image of Raspberry Pi 1 Model A+:



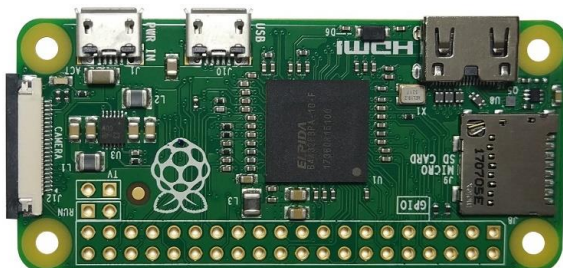
Actual image of Raspberry Pi Zero W:



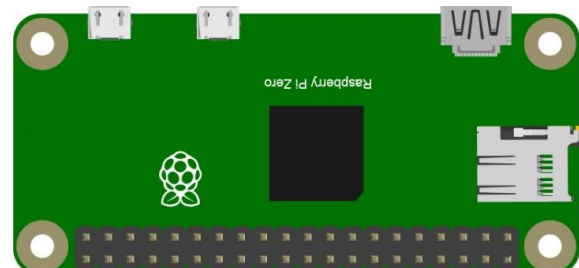
CAD image of Raspberry Pi Zero W:



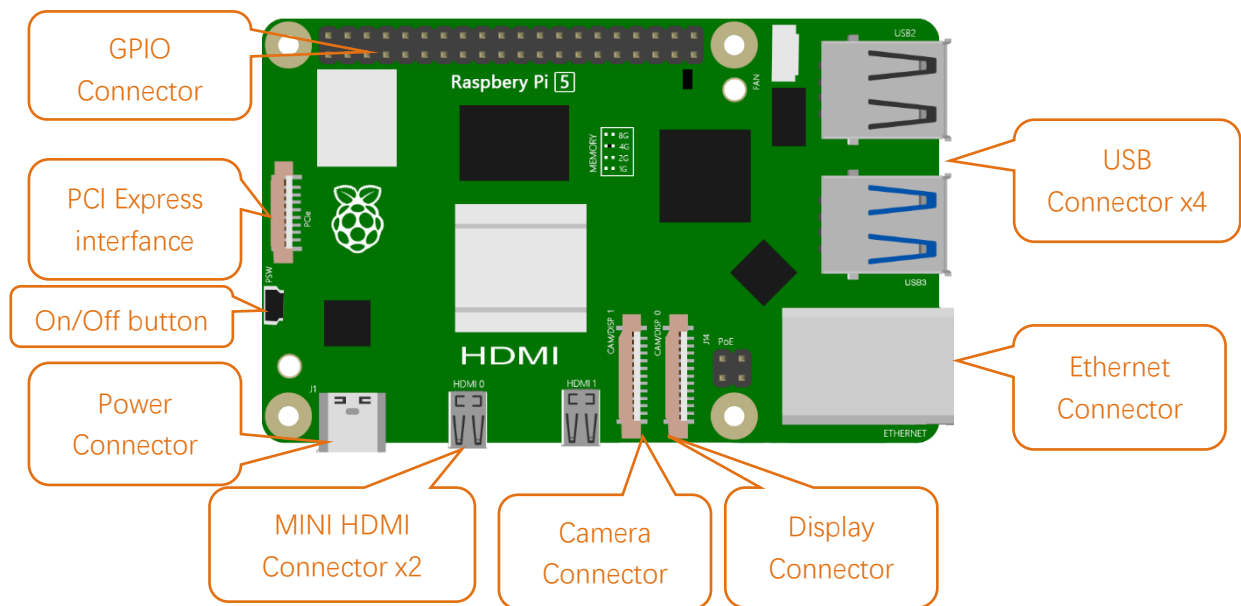
Actual image of Raspberry Pi Zero:



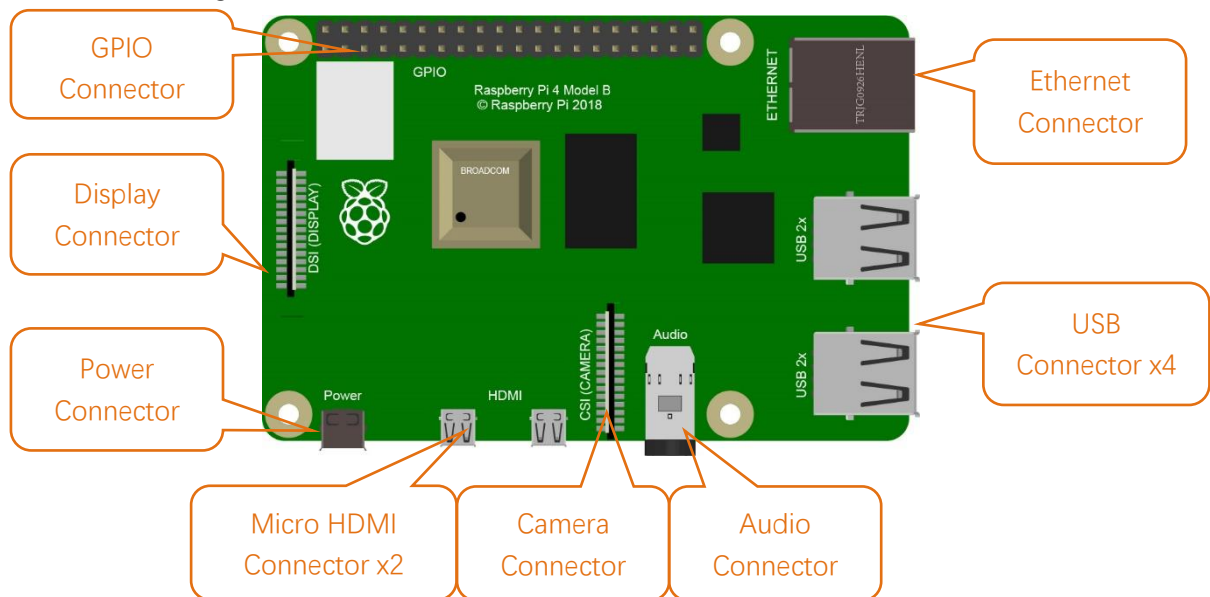
CAD image of Raspberry Pi Zero:



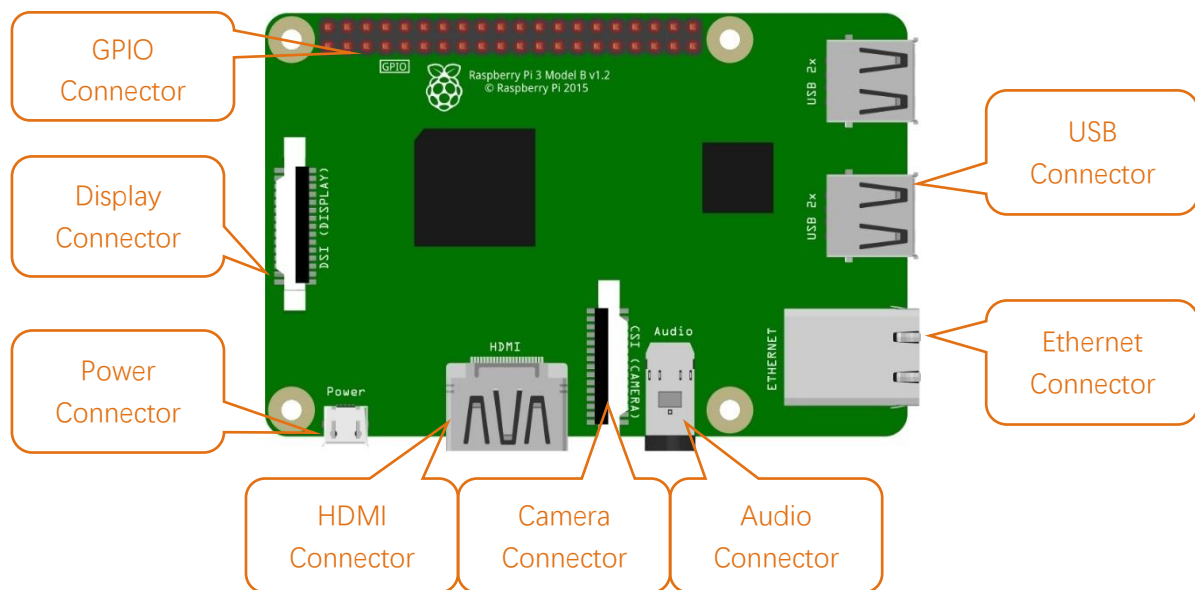
Below are the raspberry pi pictures and model pictures supported by this product. They have 40 pins.
Hardware interface diagram of RPi 5 is shown below:



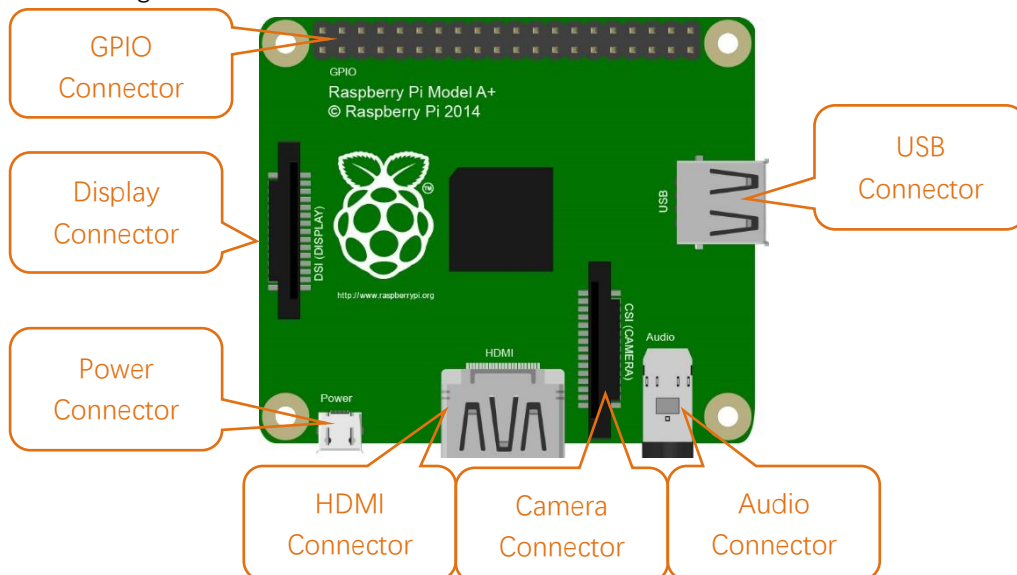
Hardware interface diagram of RPi 4B is shown below:



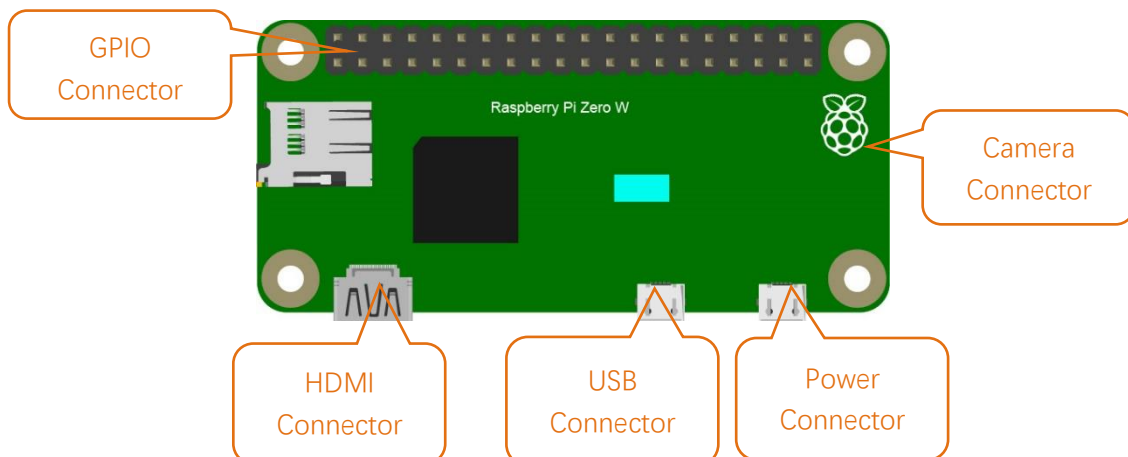
Hardware interface diagram of RPi 3B+/3B/2B/1B+:



Hardware interface diagram of RPi 3A+/A+:



Hardware interface diagram of RPi Zero/Zero W:



Chapter 0 Preparation

Why “Chapter 0”? Because in program code the first number is 0. We choose to follow this rule. In this chapter, we will do some necessary foundational preparation work: Start your Raspberry Pi and install some necessary libraries.

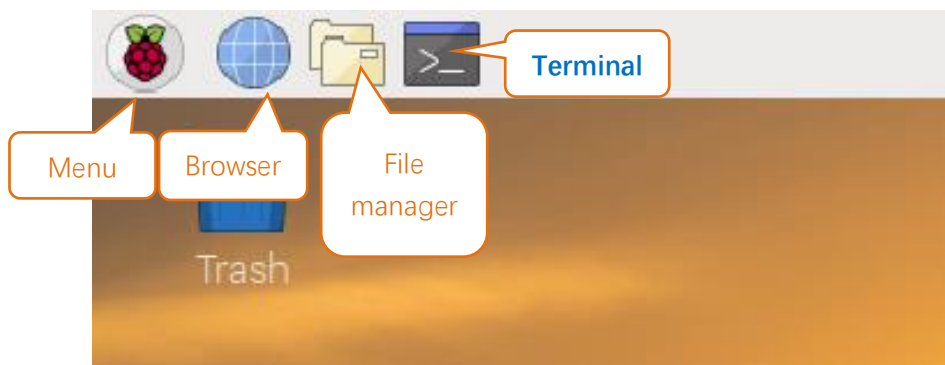
If you haven't installed the system for Raspberry Pi yet, please refer to:

[Freenove_Ultrasonic_Starter_Kit_for_Raspberry_Pi/Installing Raspberry Pi OS.pdf](#)

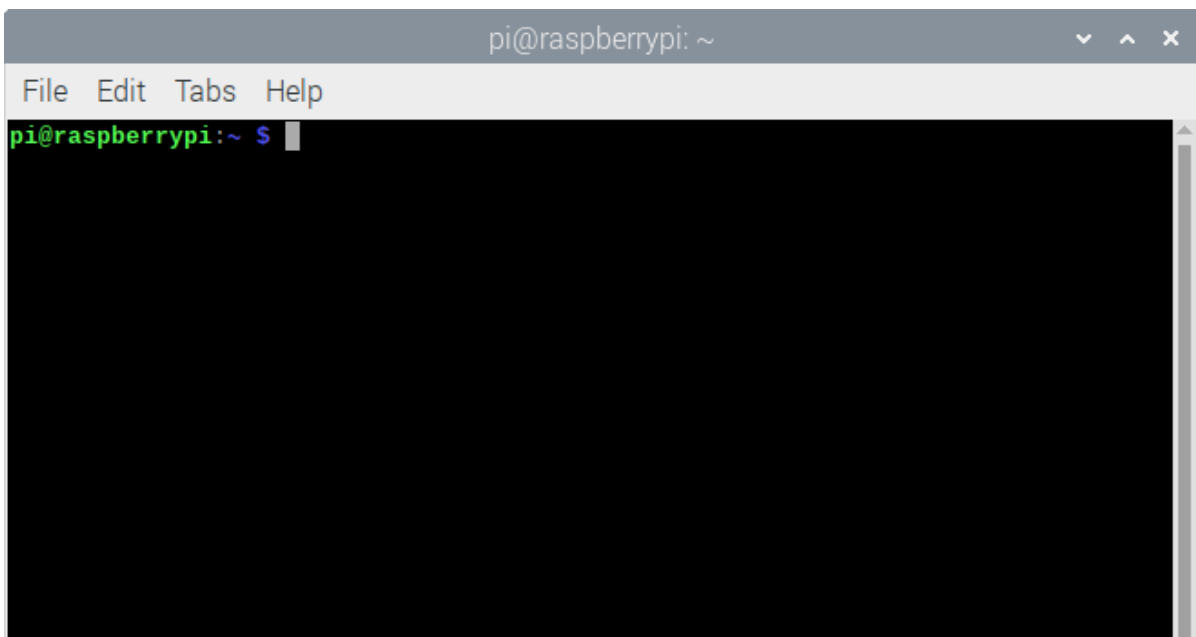
Linux Command

Raspberry Pi OS is based on the Linux Operation System. Now we will introduce you to some frequently used Linux commands and rules.

First, open the Terminal. All commands are executed in Terminal.



When you click the Terminal icon, following interface appears.



Note: The Linux is case sensitive.

First, type "ls" into the Terminal and press the "Enter" key. The result is shown below:

```

pi@raspberrypi: ~
File Edit Tabs Help
pi@raspberrypi:~ $ ls
Desktop                               Music
Documents                             Pictures
Downloads                             Public
Freenove_Three-wheeled_Smart_Car_Kit_for_Raspberry_Pi Templates
Freenove_Ultimate_Starter_Kit_for_Raspberry_Pi thinclient_drives
MagPi                                 Videos
mu_code

```

The "ls" command lists information about the files (the current directory by default).

Content between "\$" and "pi@raspberrypi:" is the current working path. "~" represents the user directory, which refers to "/home/pi" here.

```

pi@raspberrypi:~ $ pwd
/home/pi

```

"cd" is used to change directory. "/" represents the root directory.

```

pi@raspberrypi:~ $ cd /usr
pi@raspberrypi:/usr $ ls
bin  games  include  lib  local  man  sbin  share  src
pi@raspberrypi:/usr $ cd ~
pi@raspberrypi:~ $

```

Later in this Tutorial, we will often change the working path. Typing commands under the wrong directory may cause errors and break the execution of further commands.

Many frequently used commands and instructions can be found in the following reference table.

Command	instruction
ls	Lists information about the FILES (the current directory by default) and entries alphabetically.
cd	Changes directory
sudo + cmd	Executes cmd under root authority
./	Under current directory
gcc	GNU Compiler Collection
git clone URL	Use git tool to clone the contents of specified repository, and URL in the repository address.

There are many commands, which will come later. For more details about commands. You can refer to:

<http://www.linux-commands-examples.com>

Shortcut Key

Now, we will introduce several commonly used shortcuts that are very useful in Terminal.

1. **Up and Down Arrow Keys:** Pressing “↑” (the Up key) will go backwards through the command history and pressing “↓” (the Down Key) will go forwards through the command history.
2. **Tab Key:** The Tab key can automatically complete the command/path you want to type. When there is only one eligible option, the command/path will be completely typed as soon as you press the Tab key even you only type one character of the command/path.

As shown below, under the '~' directory, you enter the Documents directory with the “cd” command. After typing “cd D”, pressing the Tab key (there is no response), pressing the Tab key again then all the files/folders that begin with “D” will be listed. Continue to type the letters “oc” and then pressing the Tab key, the “Documents” is typed automatically.

```
pi@raspberrypi:~ $ cd D
Desktop/  Documents/ Downloads/
pi@raspberrypi:~ $ cd Doc
```

```
pi@raspberrypi:~ $ cd D
Desktop/  Documents/ Downloads/
pi@raspberrypi:~ $ cd Documents/
```

Install GPIO Zero Python library

GPIO Zero is a simple interface to GPIO devices with Raspberry Pi. GPIO Zero is installed by default in the Raspberry Pi OS desktop image, and the Raspberry Pi Desktop image for PC/Mac, both available from raspberrypi.org. Follow these guides to installing on Raspberry Pi OS Lite and other operating systems.

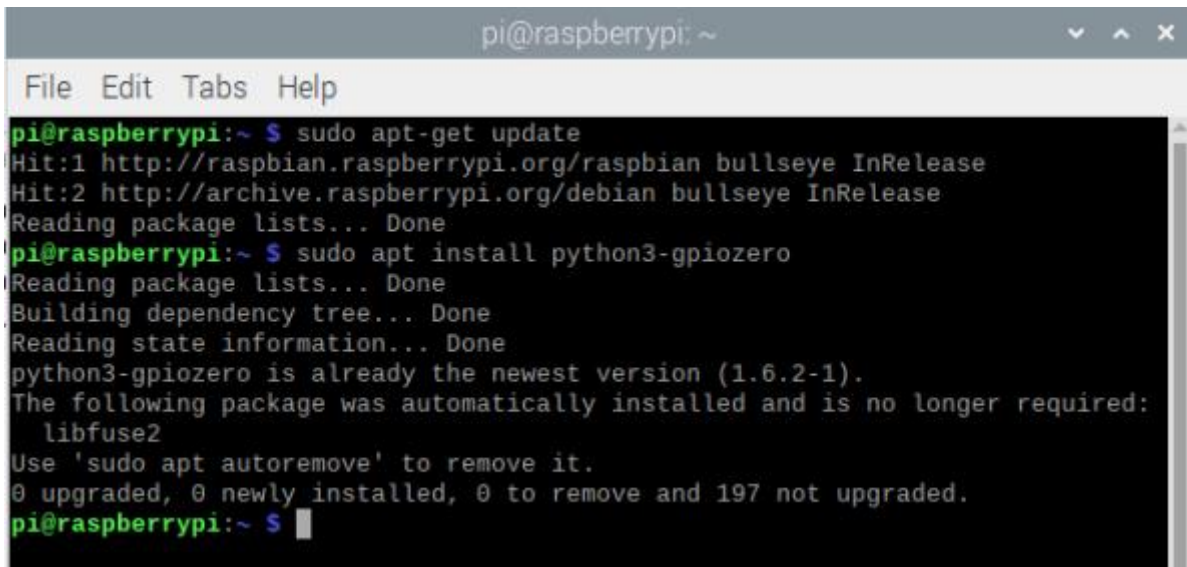
GPIO Zero Python library Installation Steps

To install the GPIO Zero Python library, please open the Terminal and then follow the steps and commands below.

Note: For a command containing many lines, execute them one line at a time.

Enter the following commands **one by one** in the **terminal** to install GPIO Zero:

```
sudo apt-get update
sudo apt install python3-gpiozero
```



```
pi@raspberrypi: ~
File Edit Tabs Help
pi@raspberrypi:~ $ sudo apt-get update
Hit:1 http://raspbian.raspberrypi.org/raspbian bullseye InRelease
Hit:2 http://archive.raspberrypi.org/debian bullseye InRelease
Reading package lists... Done
pi@raspberrypi:~ $ sudo apt install python3-gpiozero
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-gpiozero is already the newest version (1.6.2-1).
The following package was automatically installed and is no longer required:
  libfuse2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 197 not upgraded.
pi@raspberrypi:~ $
```

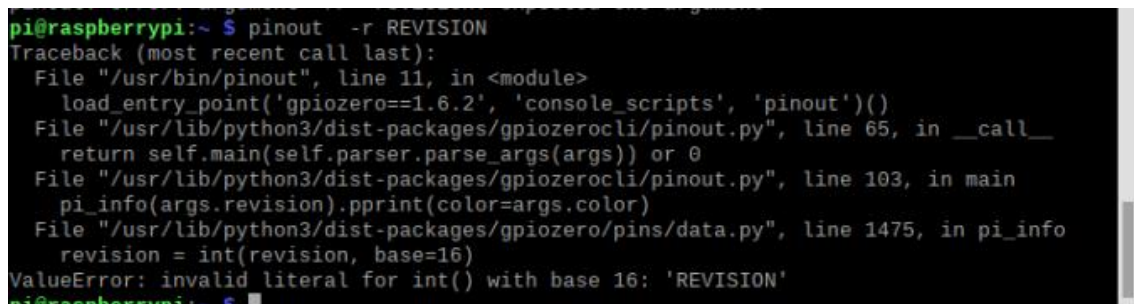
If you're using another operating system on your Raspberry Pi, you may need to use pip to install GPIO Zero instead. Install pip using get-pip and then type:

```
sudo pip3 install gpiozero
```

Run the gpiozero command to check the installation:

```
pinout -r REVISION
```

That should give you some confidence that the installation was a success.



```
pi@raspberrypi:~ $ pinout -r REVISION
Traceback (most recent call last):
  File "/usr/bin/pinout", line 11, in <module>
    load_entry_point('gpiozero==1.6.2', 'console_scripts', 'pinout')()
  File "/usr/lib/python3/dist-packages/gpiozerocli/pinout.py", line 65, in __call__
    return self.main(self.parser.parse_args(args)) or 0
  File "/usr/lib/python3/dist-packages/gpiozerocli/pinout.py", line 103, in main
    pi_info(args.revision).pprint(color=args.color)
  File "/usr/lib/python3/dist-packages/gpiozero/pins/data.py", line 1475, in pi_info
    revision = int(revision, base=16)
ValueError: invalid literal for int() with base 16: 'REVISION'
pi@raspberrypi:~ $
```

Obtain the Project Code

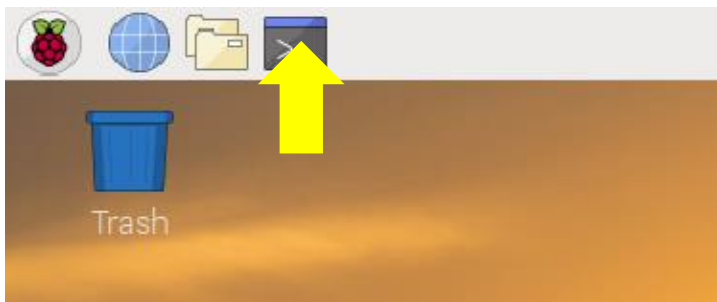
After the above installation is completed, you can visit our official website (<http://www.freenove.com>) or our GitHub resources at (<https://github.com/freenove>) to download the latest available project code. In this tutorial, we provide Python language code for each project.

This is the method for obtaining the code:

In the pi directory of the RPi terminal, enter the following command.

```
cd
git clone --depth 1 https://github.com/freenove/Freenove_Ultrasonic_Starter_Kit_for_Raspberry_Pi
```

(There is no need for a password. If you get some errors, please check your commands.)

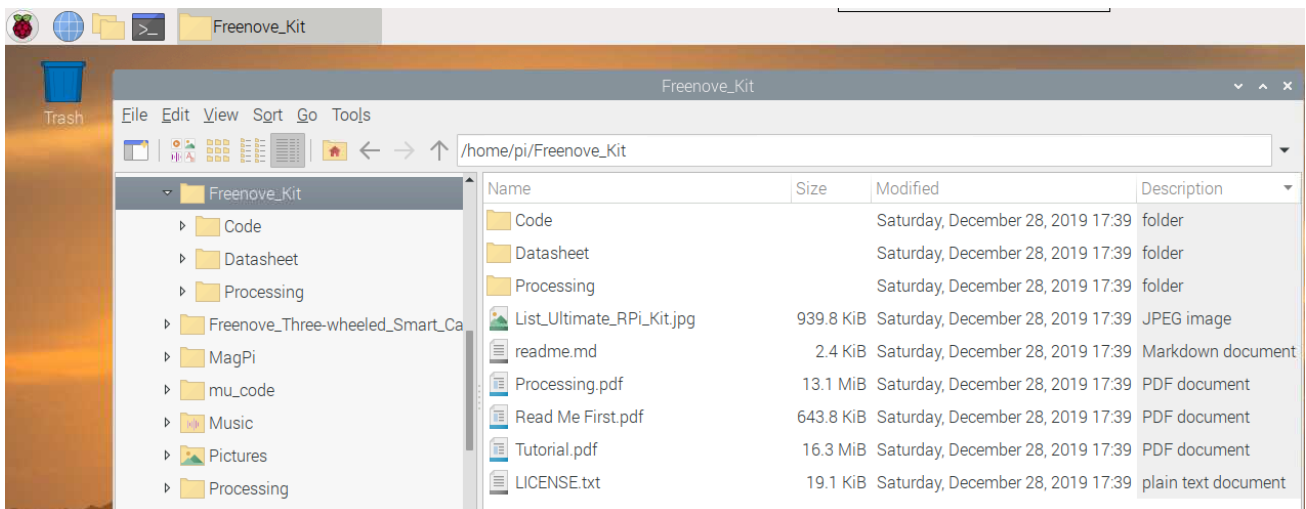


After the download is completed, a new folder "Freenove_Ultrasonic_Starter_Kit_for_Raspberry_Pi" is generated, which contains all of the tutorials and required code.

This folder name seems a little too long. We can simply rename it by using the following command.

```
mv Freenove_Ultrasonic_Starter_Kit_for_Raspberry_Pi Freenove_Kit
```

"Freenove_Kit" is now the new and much shorter folder name.



If you have no experience with Python, we suggest that you refer to this website for basic information and knowledge.

<https://python.swaroopch.com/basics.html>

Python2 & Python3

Python code, used in our kits, can now run on Python2 and Python3. **Python3 is recommend**. If you want to use Python2, please make sure your Python version is 2.7 or above. Python2 and Python3 are not fully compatible. However, Python2.6 and Python2.7 are transitional versions to python3, therefore you can also use Python2.6 and 2.7 to execute some Python3 code.

You can type "python2" or "python3" respectively into Terminal to check if python has been installed. Press Ctrl-Z to exit.

```
pi@raspberrypi:~ $ python2
Python 2.7.18 (default, Jul 14 2021, 08:11:37)
[GCC 10.2.1 20210110] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
[1]+  Stopped                  python2
pi@raspberrypi:~ $ python3
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
[2]+  Stopped                  python3
pi@raspberrypi:~ $
```

Type "python", and Terminal shows that it links to python3.

```
pi@raspberrypi:~ $ python
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
[3]+  Stopped                  python
^Xpi@raspberrypi:~ $
```

Set Python3 as default python

First, execute python to check the default python on your raspberry Pi. Press Ctrl-Z to exit.

```
pi@raspberrypi:~ $ python
Python 3.9.2 (default, Mar 12 2021, 04:06:34)
[GCC 10.2.1 20210110] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
[3]+  Stopped                  python
^Xpi@raspberrypi:~ $
```

If it is python3, you can skip this section.

If it is python2, you need execute the following commands to set default python to python3.

1. Enter directory /usr/bin

```
cd /usr/bin
```

2. Delete the originalpython link.

```
sudo rm python
```

3. Create new python links to python.

```
sudo ln -s python3 python
```

4. Check python. Press Ctrl-Z to exit.

```
python
```

```
pi@raspberrypi:/usr/bin $ sudo rm python
pi@raspberrypi:/usr/bin $ sudo ln -s python3 python
pi@raspberrypi:/usr/bin $ python
Python 3.5.3 (default, Jan 19 2017, 14:11:04)
[GCC 6.3.0 20170124] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

If you want to set python2 as default python in **other projects**, just repeat the commands above and change python3 to python2.

Shortcut Key

Now, we will introduce several shortcuts that are very **useful** and **commonly used** in terminal.

1. **up and down arrow keys**. History commands can be quickly brought back by using up and down arrow keys, which are very useful when you need to reuse certain commands.

When you need to type commands, pressing “↑” will go backwards through the history of typed commands, and pressing “↓” will go forwards through the history of typed command.

2. **Tab key**. The Tab key can automatically complete the command/path you want to type. When there are multiple commands/paths conforming to the already typed letter, pressing Tab key once won't have any result. And pressing Tab key again will list all the eligible options. This command/path will be completely typed as soon as you press the Tab key when there is only one eligible option.

As shown below, under the '~' directory, enter the Documents directory with the “cd” command. After typing “cd D”, press Tab key, then there is no response. Press Tab key again, then all the files/folders that begin with “D” is listed. Continue to type the character “oc”, then press the Tab key, and then “Documents” is completely typed automatically.

```
pi@raspberrypi:~ $ cd D
Desktop/  Documents/ Downloads/
pi@raspberrypi:~ $ cd Doc
```

```
pi@raspberrypi:~ $ cd D
Desktop/  Documents/ Downloads/
pi@raspberrypi:~ $ cd Documents/
```

Chapter 1 LED

This chapter is the Start Point in the journey to build and explore RPi electronic projects. We will start with simple “Blink” project.

Project 1.1 Blink

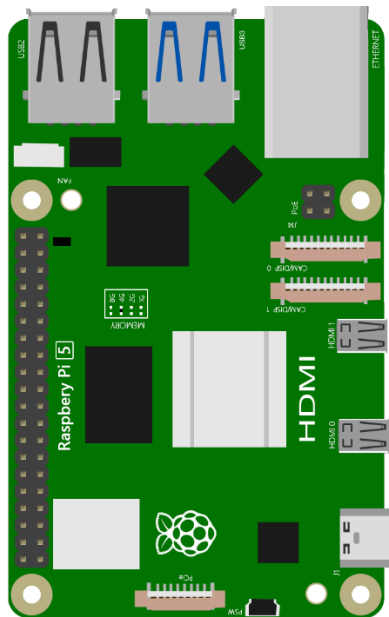
In this project, we will use RPi to control blinking a common LED.

Component List

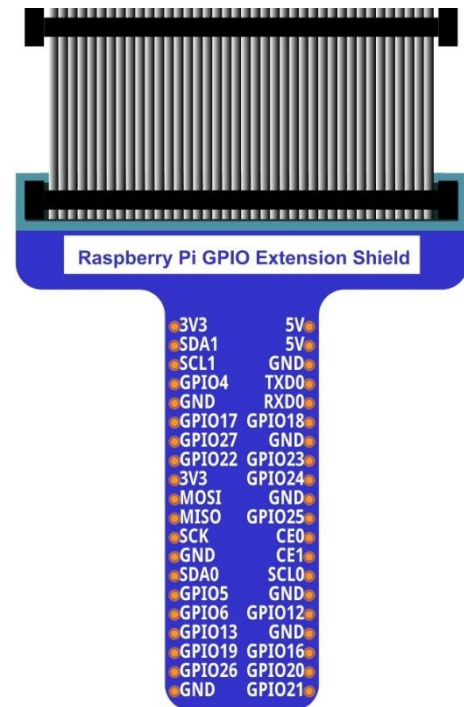
Raspberry Pi

(Recommended: Raspberry Pi 4B / 3B+ / 3B)

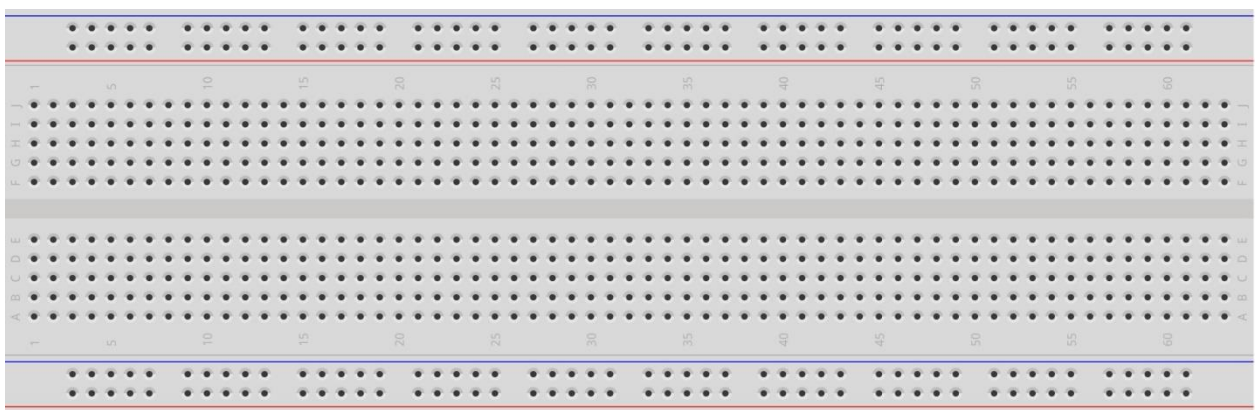
Compatible: 3A+ / 2B / 1B+ / 1A+ / Zero W / Zero)



GPIO Extension Board & Ribbon Cable



Breadboard x1



LED x1 	Resistor 220Ω x1 	Jumper Specific quantity depends on the circuit. 
---	---	--

In the components list, 3B GPIO, Extension Shield Raspberry and Breadboard are necessary for each project. Later, they will be reference by text only (no images as in above).

GPIO

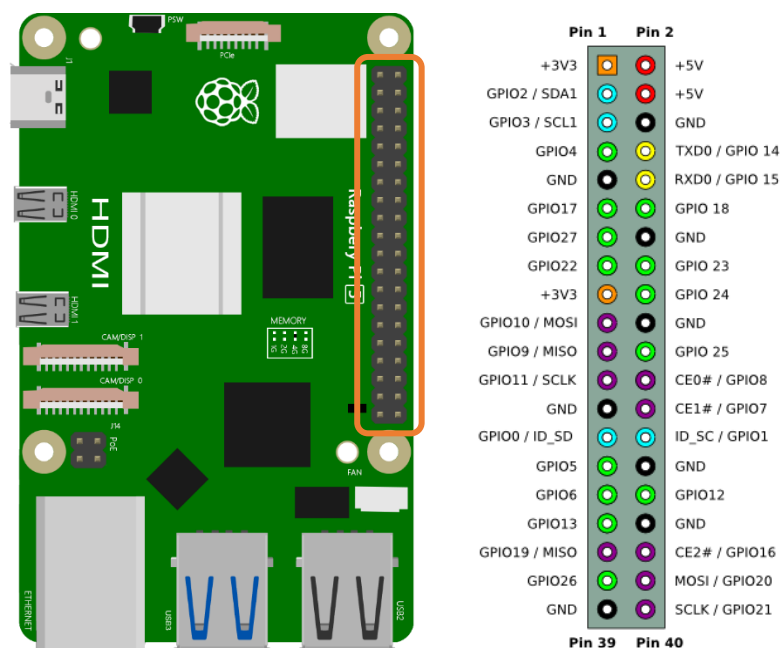
GPIO: General Purpose Input/Output. Here we will introduce the specific function of the pins on the Raspberry Pi and how you can utilize them in all sorts of ways in your projects. Most RPi Module pins can be used as either an input or output, depending on your program and its functions.

When programming GPIO pins there are 3 different ways to reference them: GPIO Numbering, Physical Numbering and WiringPi GPIO Numbering.

BCM GPIO Numbering

The Raspberry Pi CPU uses Broadcom (BCM) processing chips BCM2835, BCM2836 or BCM2837. GPIO pin numbers are assigned by the processing chip manufacturer and are how the computer recognizes each pin. The pin numbers themselves do not make sense or have meaning as they are only a form of identification. Since their numeric values and physical locations have no specific order, there is no way to remember them so you will need to have a printed reference or a reference board that fits over the pins.

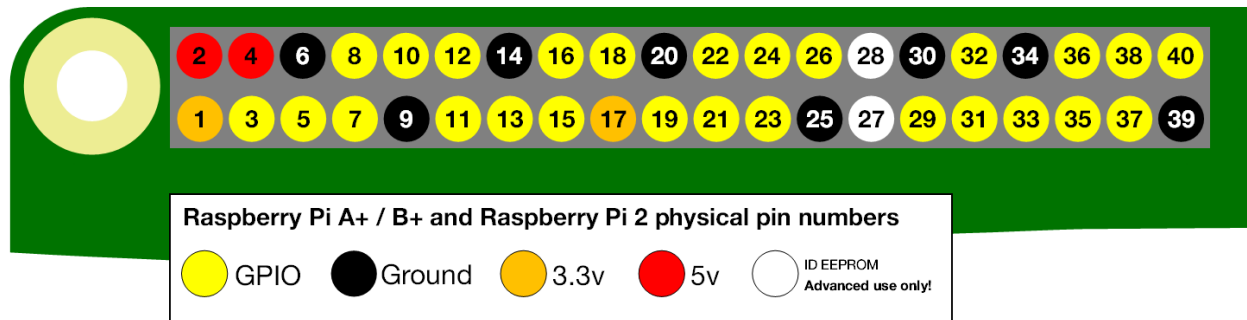
Each pin's functional assignment is defined in the image below:



For more details about pin definition of GPIO, please refer to <http://pinout.xyz/>

PHYSICAL Numbering

Another way to refer to the pins is by simply counting across and down from pin 1 at the top left (nearest to the SD card). This is 'Physical Numbering', as shown below:



GPIO Numbering

You can use the following command to view their correlation.

Pinout

```

pi@raspberrypi: ~
File Edit Tabs Help

pi@raspberrypi:~ $ pinout

  oooooooooooooooooooooo J8
  1ooooooooooooooooooooo
  Wi
  Fi Pi Model 3A+ V1.0
  |D| |SoC|
  |S|
  |I|
  +-----+
  |C| |USB|
  |S|
  |I| |A|
  |V|
  pwr HDMI

Revision      : 9020e0
SoC           : BCM2837
RAM           : 512MB
Storage       : MicroSD
USB ports     : 1 (of which 0 USB3)
Ethernet ports : 0 (0Mbps max. speed)
Wi-fi        : True
Bluetooth    : True
Camera ports (CSI) : 1
Display ports (DSI): 1

J8:
  3V3 (1) (2) 5V
  GPIO2 (3) (4) 5V
  GPIO3 (5) (6) GND
  GPIO4 (7) (8) GPIO14
  GND (9) (10) GPIO15
  GPIO17 (11) (12) GPIO18
  GPIO27 (13) (14) GND
  GPIO22 (15) (16) GPIO23
  3V3 (17) (18) GPIO24
  GPIO10 (19) (20) GND
  GPIO9 (21) (22) GPIO25
  GPIO11 (23) (24) GPIO8
  GND (25) (26) GPIO7
  GPIO0 (27) (28) GPIO1
  GPIO5 (29) (30) GND
  GPIO6 (31) (32) GPIO12
  GPIO13 (33) (34) GND
  GPIO19 (35) (36) GPIO16
  GPIO26 (37) (38) GPIO20
  GND (39) (40) GPIO21

For further information, please refer to https://pinout.xyz/
pi@raspberrypi:~ $

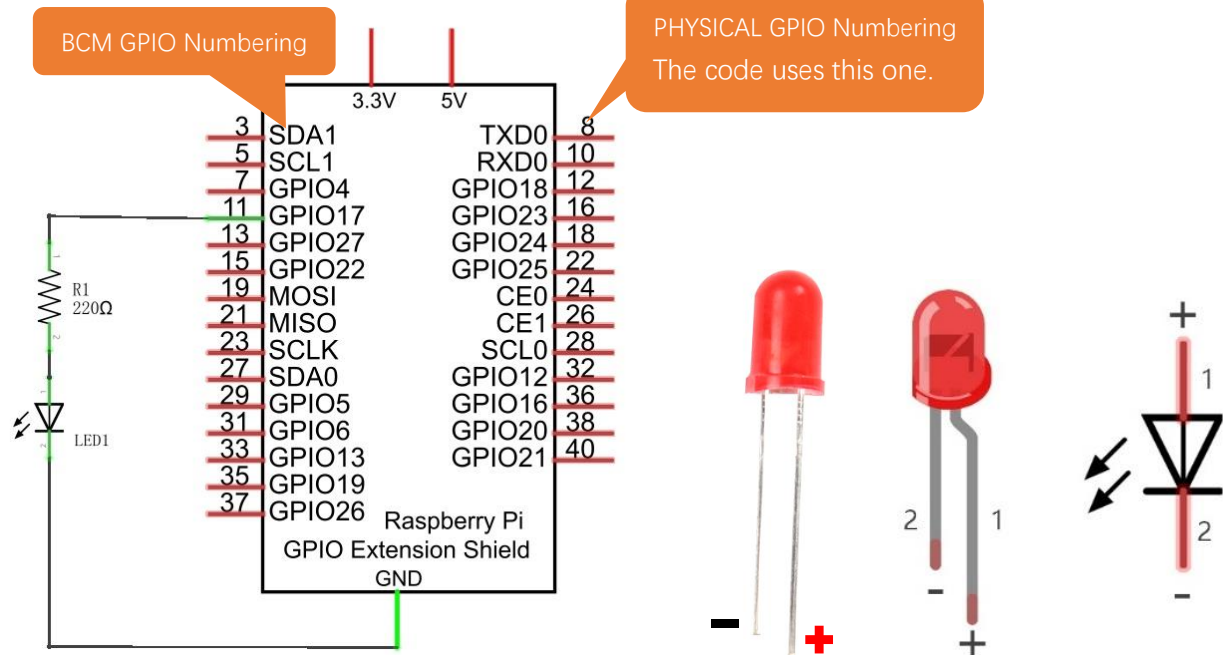
```

Circuit

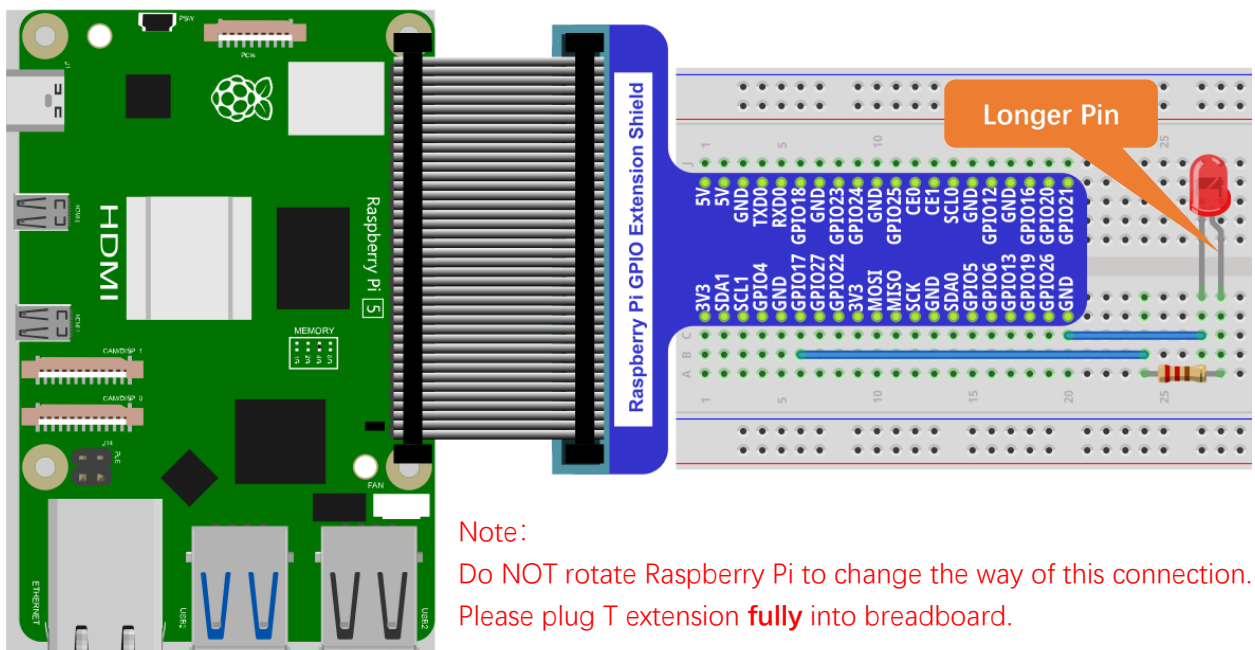
First, disconnect your RPi from the GPIO Extension Shield. Then build the circuit according to the circuit and hardware diagrams. After the circuit is built and verified correct, connect the RPi to GPIO Extension Shield. CAUTION: Avoid any possible short circuits (especially connecting 5V or GND, 3.3V and GND)!

WARNING: A short circuit can cause high current in your circuit, create excessive component heat and cause permanent damage to your RPi!

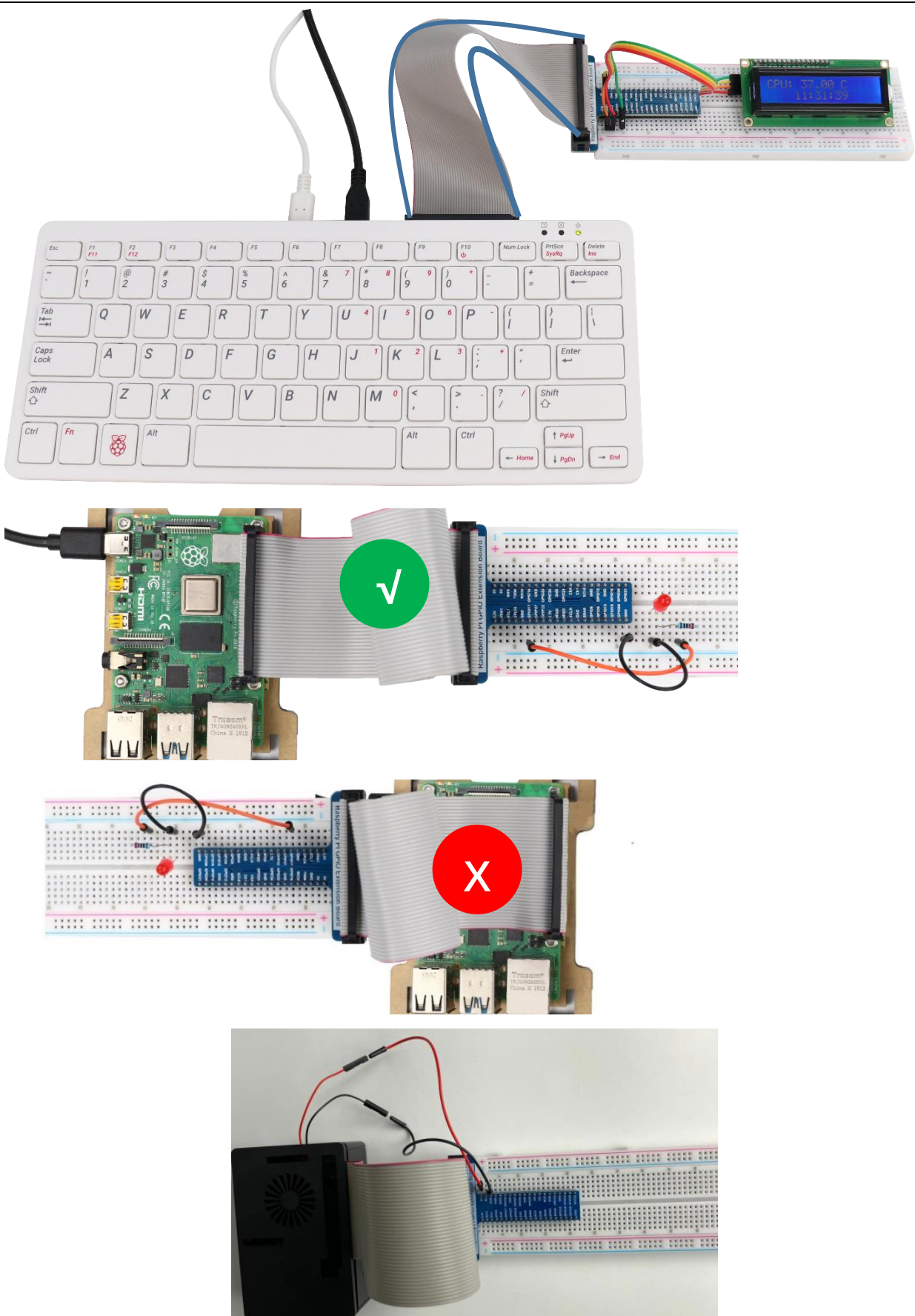
Schematic diagram



Hardware connection. If you need any support, please contact us via: support@freenove.com



The connection of Raspberry Pi T extension board is as below. Don't reverse the ribbon.



If you have a fan, you can connect it to 5V GND of breadboard via jumper wires.

How to distinguish resistors?

There are only three kind of resistors in this kit.

The one with 1 red ring is $10K\Omega$ 

The one with 2 red rings is 220Ω 

The one with 0 red ring is $1K\Omega$ 

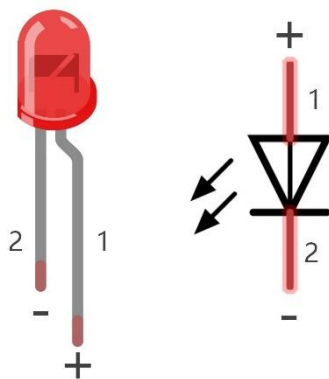
Future hardware connection diagrams will only show that part of breadboard and GPIO Extension Shield.

Component knowledge

LED

An LED is a type of diode. All diodes only work if current is flowing in the correct direction and have two Poles. An LED will only work (light up) if the longer pin (+) of LED is connected to the positive output from a power source and the shorter pin is connected to the negative (-) output, which is also referred to as Ground (GND). This type of component is known as "Polar" (think One-Way Street).

All common 2 lead diodes are the same in this respect. Diodes work only if the voltage of its positive electrode is higher than its negative electrode and there is a narrow range of operating voltage for most all common diodes of 1.9 and 3.4V. If you use much more than 3.3V the LED will be damaged and burnt out.



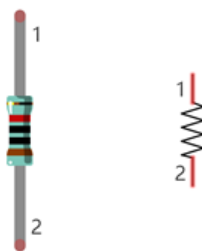
LED	Voltage	Maximum current	Recommended current
Red	1.9-2.2V	20mA	10mA
Green	2.9-3.4V	10mA	5mA
Blue	2.9-3.4V	10mA	5mA
Volt ampere characteristics conform to diode			

Note: LEDs cannot be directly connected to a power supply, which usually ends in a damaged component. A resistor with a specified resistance value must be connected in series to the LED you plan to use.

Resistor

Resistors use Ohms (Ω) as the unit of measurement of their resistance (R). $1\text{M}\Omega=1000\text{k}\Omega$, $1\text{k}\Omega=1000\Omega$.

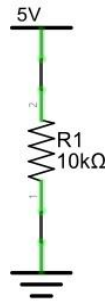
A resistor is a passive electrical component that limits or regulates the flow of current in an electronic circuit. On the left, we see a physical representation of a resistor, and the right is the symbol used to represent the presence of a resistor in a circuit diagram or schematic.



The bands of color on a resistor is a shorthand code used to identify its resistance value. For more details of resistor color codes, please refer to the card in the kit package.

With a fixed voltage, there will be less current output with greater resistance added to the circuit. The relationship between Current, Voltage and Resistance can be expressed by this formula: $I=V/R$ known as Ohm's Law where I = Current, V = Voltage and R = Resistance. Knowing the values of any two of these allows you to solve the value of the third.

In the following diagram, the current through R1 is: $I=U/R=5\text{V}/10\text{k}\Omega=0.0005\text{A}=0.5\text{mA}$.

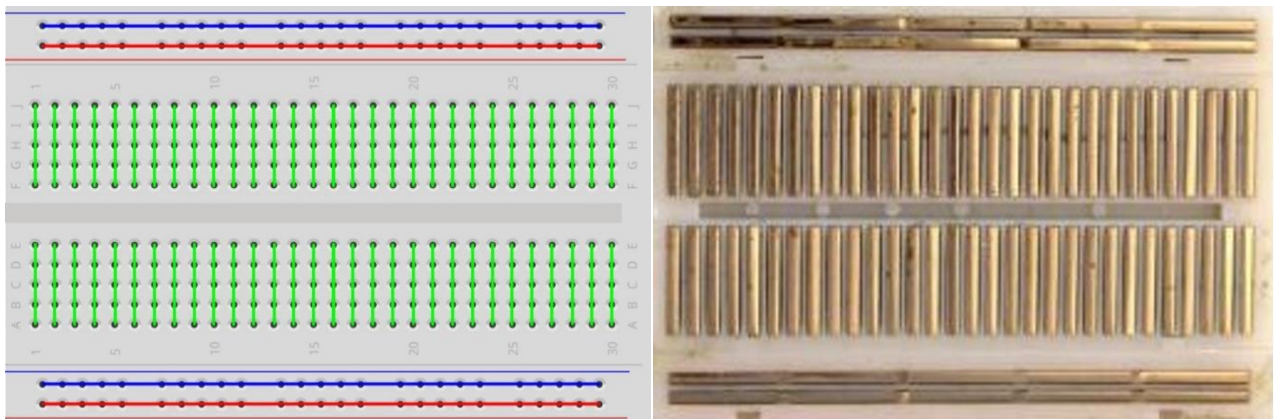


WARNING: Never connect the two poles of a power supply with anything of low resistance value (i.e. a metal object or bare wire) this is a Short and results in high current that may damage the power supply and electronic components.

Note: Unlike LEDs and Diodes, Resistors have no poles and are non-polar (it does not matter which direction you insert them into a circuit, it will work the same)

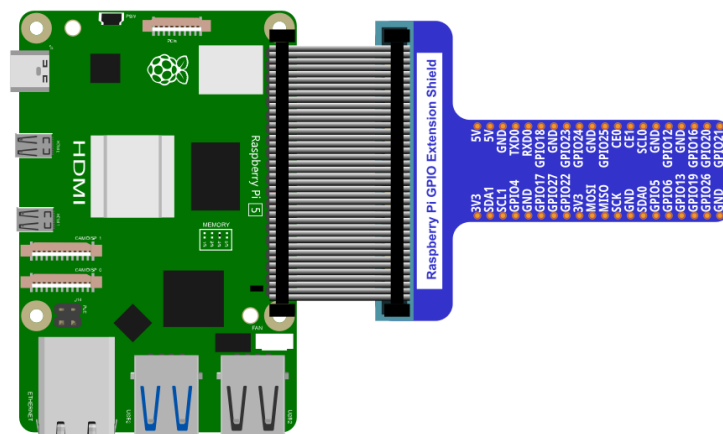
Breadboard

Here we have a small breadboard as an example of how the rows of holes (sockets) are electrically attached. The left picture shows the ways the pins have shared electrical connection and the right picture shows the actual internal metal, which connect these rows electrically.



GPIO Extension Board

GPIO board is a convenient way to connect the RPi I/O ports to the breadboard directly. The GPIO pin sequence on Extension Board is identical to the GPIO pin sequence of RPi.



Code

According to the circuit, when the GPIO17 of RPi output level is high, the LED turns ON. Conversely, when the GPIO17 RPi output level is low, the LED turns OFF. Therefore, we can let GPIO17 cycle output high and output low level to make the LED blink. We will use Python code to achieve the target.

Python Code 1.1.1 Blink

Now, we will use Python language to make a LED blink.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 01.1.1_Blink directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/01.1.1_Blink
```

2. Use python command to execute python code blink.py.

```
python Blink.py
```

The LED starts blinking.

```
pi@raspberrypi: ~/Freenove_Kit/Code/Python_GPIOZero_Code/01.1.1_Blink
File Edit Tabs Help
pi@raspberrypi:~ $ cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/01.1.1_Blink
pi@raspberrypi:~/Freenove_Kit/Code/Python_GPIOZero_Code/01.1.1_Blink $ python Blink.py
Program is starting ...

led turned on >>>
led turned off <<<
led turned on >>>
led turned off <<<
^CEnding program
pi@raspberrypi:~/Freenove_Kit/Code/Python_GPIOZero_Code/01.1.1_Blink $
```

You can press "Ctrl+C" to end the program. The following is the program code:

```
1  from gpiozero import LED
2  from time import sleep
3
4  led = LED(17)          # define LED pin according to BCM Numbering
5  #led = LED("J8:11")    # BOARD Numbering
6  '''
7  # pins numbering, the following lines are all equivalent
8  led = LED(17)          # BCM
9  led = LED("GPIO17")    # BCM
10 led = LED("BCM17")     # BCM
11 led = LED("BOARD11")   # BOARD
12 led = LED("WPI0")      # WiringPi
13 led = LED("J8:11")     # BOARD
14 '''
15 def loop():
16     while True:
```

```

17         led.on()    # turn on LED
18         print ('led turned on >>>') # print message on terminal
19         sleep(1)    # wait 1 second
20         led.off()   # turn off LED
21         print ('led turned off <<<') # print message on terminal
22         sleep(1)    # wait 1 second
23
24     if __name__ == '__main__':    # Program entrance
25         print ('Program is starting ... \n')
26         try:
27             loop()
28         except KeyboardInterrupt: # Press ctrl-c to end the program.
29             print("Ending program")

```

Import the LED class from the gpiozero library.

```
from gpiozero import LED
```

Create an LED assembly for controlling the LED.

```
led = LED(17)    # define LED pin according to BCM Numbering
```

Turn on LED device.

```
led.on()    # turn on LED
```

Turn off LED devices.

```
led.off()   # turn off LED
```

The main function turns on the LED for one second and then turns it off for one second, which repeats endlessly.

```

def loop():
    while True:
        led.on()    # turn on LED
        print ('led turned on >>>') # print message on terminal
        sleep(1)    # wait 1 second
        led.off()   # turn off LED
        print ('led turned off <<<') # print message on terminal
        sleep(1)    # wait 1 second

```

Reference

About GPIO Zero:

GPIO Zero

A simple interface to GPIO devices with Raspberry Pi, Using the GPIO Zero library makes it easy to get started with controlling GPIO devices with Python. The library is comprehensively documented at

<https://gpiozero.readthedocs.io/en/stable/>

<https://github.com/gpiozero/gpiozero>

For more information about the methods used by the LED class in the GPIO Zero library, please refer to:

https://gpiozero.readthedocs.io/en/stable/api_output.html#led

For more information about the methods used by the DigitalOutputDevice class in the GPIO Zero library, please refer to:

https://gpiozero.readthedocs.io/en/stable/api_output.html#digitaloutputdevice

“import time” time is a module of python.

<https://docs.python.org/2/library/time.html?highlight=time%20time#module-time>

In Python, libraries and functions used in a script must be imported by name at the top of the file, with the exception of the functions built into Python by default.

For example, to use the LED interface from GPIO Zero, it should be explicitly imported:

```
from gpiozero import LED
```

Now LED is available directly in your script:

```
led = LED(17)           # define LED pin according to BCM Numbering
#led = LED("J8:11")     # BOARD Numbering
```

Alternatively, the whole GPIO Zero library can be imported:

```
import gpiozero
```

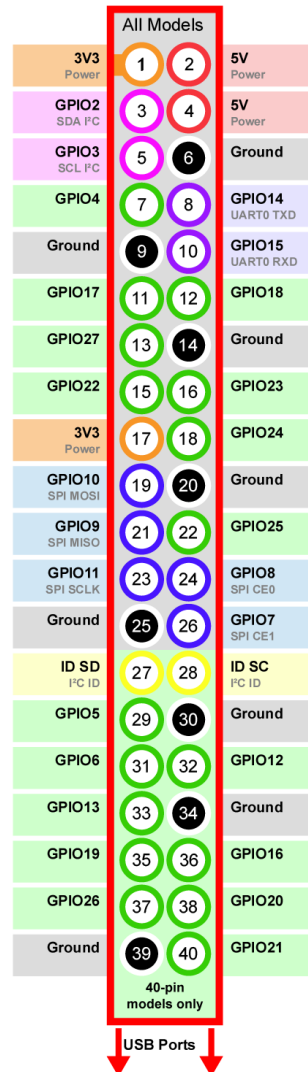
In this case, all references to items within GPIO Zero must be prefixed:

```
led = gpiozero.LED(17)      # define LED pin according to BCM Numbering
#led = gpiozero.LED("J8:11") # BOARD Numbering
```

Pin Numbering

This library uses Broadcom (BCM) pin numbering for the GPIO pins, as opposed to physical (BOARD) numbering. Unlike in the RPi.GPIO library, this is not configurable. However, translation from other schemes can be used by providing prefixes to pin numbers (see below).

Any pin marked "GPIO" in the diagram below can be used as a pin number. For example, if an LED was attached to "GPIO17" you would specify the pin number as 17 rather than 11:



If you wish to use physical (BOARD) numbering you can specify the pin number as "BOARD11". If you are familiar with the wiringPi pin numbers (another physical layout) you could use "WPI0" instead. Finally, you can specify pins as "header:number", e.g. "J8:11" meaning physical pin 11 on header J8 (the GPIO header on modern Pis). Hence, the following lines are all equivalent:

```
led = LED(17)
led = LED("GPIO17")
led = LED("BCM17")
led = LED("BOARD11")
led = LED("WPI0")
led = LED("J8:11")
```

Note that these alternate schemes are merely translations. If you request the state of a device on the command line, the associated pin number will always be reported in the Broadcom (BCM) scheme:

```
led = LED("BOARD11")
led
<gpiozero.LED object on pin GPIO17, active_high=True, is_active=False>
```

In this tutorial, we will use the default integer pin number in the Broadcom (BCM) layout.

GPIO Numbering Relationship

WingPi	BCM(Extension)	Physical		BCM(Extension)	WingPi
3.3V	3.3V	1	2	5V	5V
8	GPIO2/SDA1	3	4	5V	5V
9	GPIO3/SCL1	5	6	GND	GND
7	GPIO4	7	8	GPIO14/TXD0	15
GND	GND	9	10	GPIO15/RXD0	16
0	GPIO17	11	12	GPIO18	1
2	GPIO27	13	14	GND	GND
3	GPIO22	15	16	GPIO23	4
3.3V	3.3V	17	18	GPIO24	5
12	GPIO10/MOSI	19	20	GND	GND
13	GPIO9/MOIS	21	22	GPIO25	6
14	GPIO11/SCLK	23	24	GPIO8/CE0	10
GND	GND	25	26	GPIO7/CE1	11
30	GPIO0/SDA0	27	28	GPIO1/SCL0	31
21	GPIO5	29	30	GND	GND
22	GPIO6	31	32	GPIO12	26
23	GPIO13	33	34	GND	GND
24	GPIO19	35	36	GPIO16	27
25	GPIO26	37	38	GPIO20	28
GND	GND	39	40	GPIO21	29

In `loop()`, there is a while loop, which is an endless loop (a while loop). That is, the program will always be executed in this loop, unless it is ended because of external factors. In this loop, set LED output high level, then the LED turns ON. After a period of time delay, set LED output low level, then the LED turns OFF, which is followed by a delay. Repeat the loop, then LED will start blinking.

```
def loop():
    while True:
        led.on()    # turn on LED
        print('led turned on >>>') # print message on terminal
        sleep(1)    # wait 1 second
        led.off()   # turn off LED
        print('led turned off <<<') # print message on terminal
        sleep(1)    # wait 1 second
```

In `gpiozero`, at the end of your script, `cleanup` is run automatically, restoring your GPIO pins to the state they were found. To explicitly close a connection to a pin, you can manually call the `close()` method on a device object:

```
led = LED(17)
```

```
    led.on()
    led
    <gpiozero.LED object on pin GPIO17, active_high=True, is_active=True>
    led.close()
    led
    <gpiozero.LED object closed>
```

This means that you can reuse the pin for another device, and that despite turning the LED on (and hence, the pin high), after calling `close()` it is restored to its previous state (LED off, pin low).

In this tutorial, most projects have added an active run cleanup program to restore the GPIO pin to the found default state.

Freenove Car, Robot and other products for Raspberry Pi

We also have car and robot kits for Raspberry Pi. You can visit our website for details.

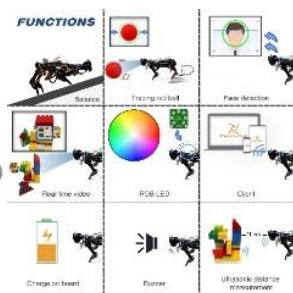
<https://www.amazon.com/freenove>

FNK0043 Freenove 4WD Smart Car Kit for Raspberry Pi



<https://www.youtube.com/watch?v=4Zv0GZUQjZc>

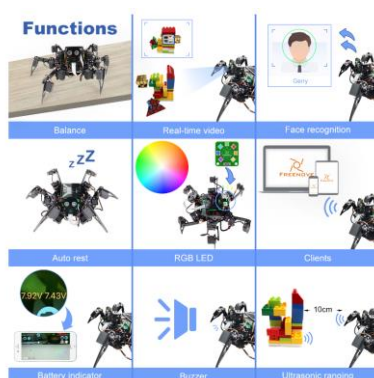
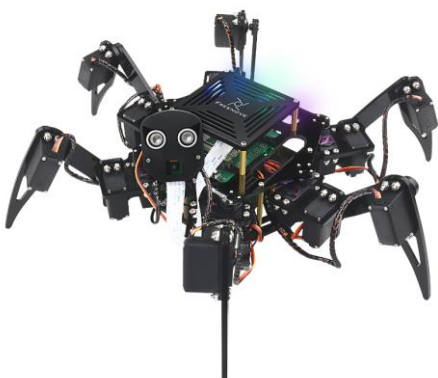
FNK0050 Freenove Robot Dog Kit for Raspberry Pi



https://www.youtube.com/watch?v=7BmIZ8_R9d4

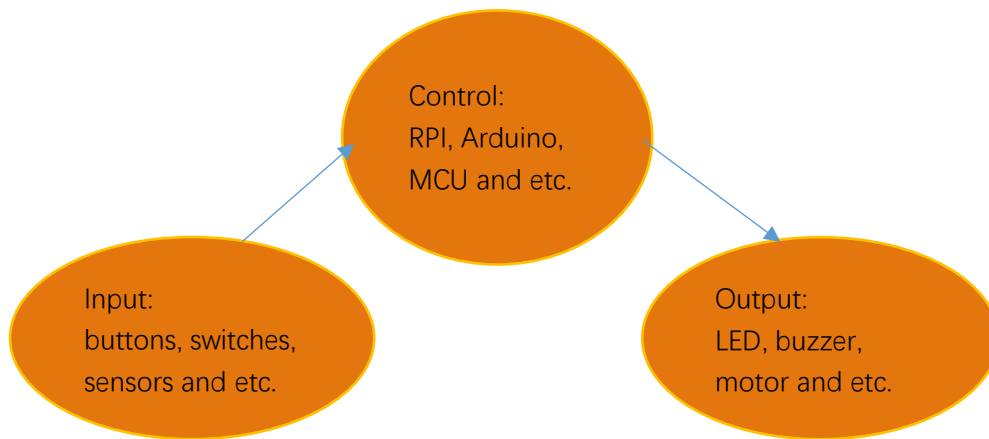
FNK0052 Freenove Big Hexapod Robot Kit for Raspberry Pi

<https://youtu.be/LvghnJ2DNZ0>



Chapter 2 Buttons & LEDs

Usually, there are three essential parts in a complete automatic control device: INPUT, OUTPUT, and CONTROL. In last section, the LED module was the output part and RPI was the control part. In practical applications, we not only make LEDs flash, but also make a device sense the surrounding environment, receive instructions and then take the appropriate action such as turn on LEDs, make a buzzer beep and so on.



Next, we will build a simple control system to control an LED through a push button switch.

Project 2.1 Push Button Switch & LED

In the project, we will control the LED state through a Push Button Switch. When the button is pressed, our LED will turn ON, and when it is released, the LED will turn OFF. This describes a Momentary Switch.

Component List

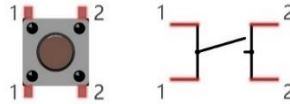
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Wire x1 Breadboard x1	LED x1 	Resistor 220Ω x1 	Resistor 10kΩ x2 	Push Button Switch x1 
Jumper Wire 				

Please Note: In the code “button” represents switch action.

Component knowledge

Push Button Switch

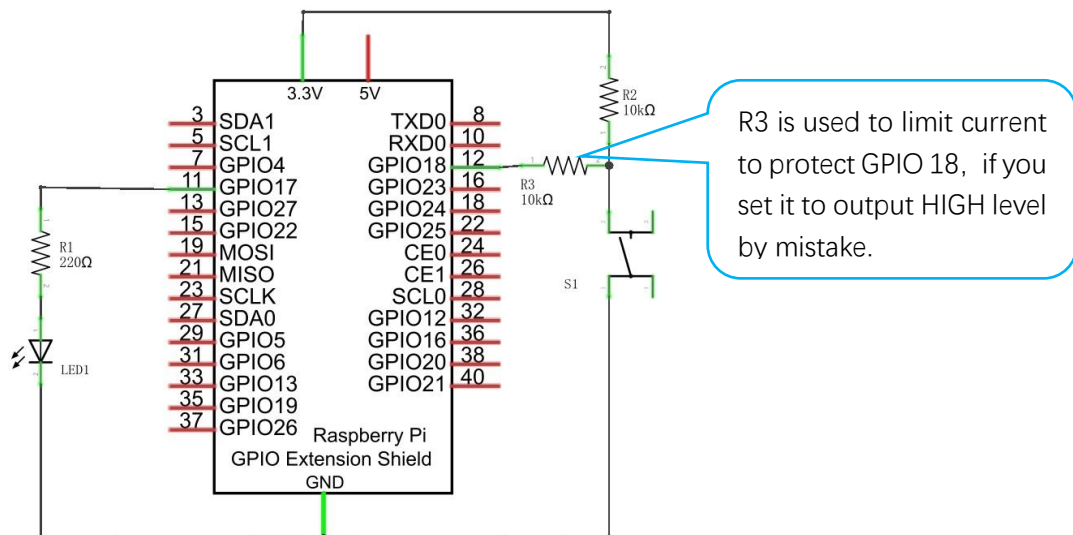
This type of Push Button Switch has 4 pins (2 Pole Switch). Two pins on the left are connected, and both left and right sides are the same per the illustration:



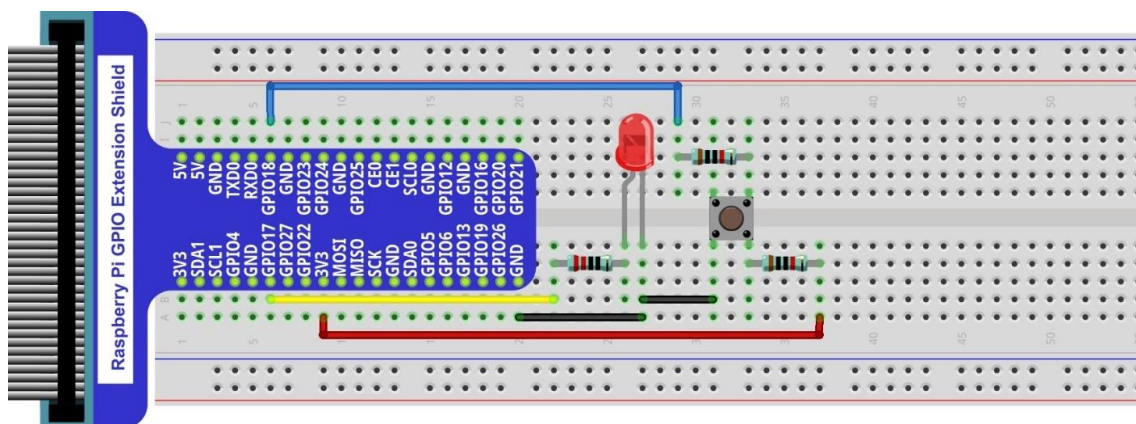
When the button on the switch is pressed, the circuit is completed (your project is Powered ON).

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



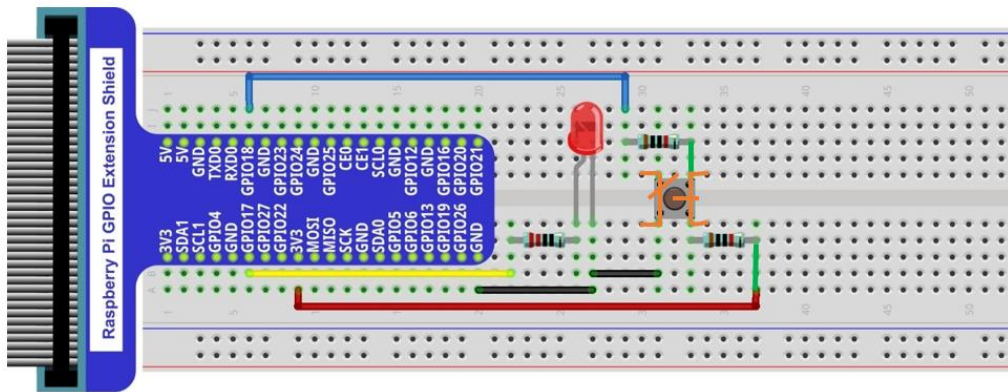
There are two kinds of push button switch in this kit.

The smaller push button switches are contained in a plastic bag.

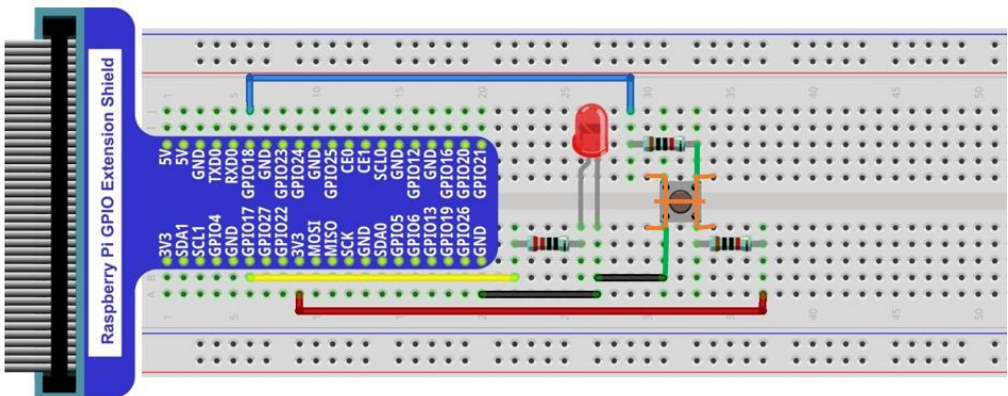
Youtube video: https://youtu.be/_5ge1d6f1nM

This is how it works.

When button switch is released:



When button switch is pressed:



Code

This project is designed for learning how to use Push Button Switch to control an LED. We first need to read the state of switch, and then determine whether to turn the LED ON in accordance to the state of the switch.

Python Code 2.1.1 ButtonLED

First, observe the project result, then learn about the code in detail. Remember in code "button" = switch function

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 02.1.1_ButtonLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/02.1.1_ButtonLED
```

2. Use Python command to execute btnLED.py.

```
python ButtonLED.py
```

Then the Terminal window continues to show the characters "led off...", press the switch button and the LED turns ON and then Terminal window shows "led on...". Release the button, then LED turns OFF and then the terminal window text "led off..." appears. You can press "Ctrl+C" at any time to terminate the program.

The following is the program code:

```
1 from gpiozero import LED, Button
2
```

```

3  led = LED(17)      # define LED pin according to BCM Numbering
4  button = Button(18) # define Button pin according to BCM Numbering
5
6  def loop():
7      while True:
8          if button.is_pressed: # if button is pressed
9              led.on()          # turn on led
10             print("Button is pressed, led turned on >>>") # print information on terminal
11             else : # if button is released
12                 led.off() # turn off led
13                 print("Button is released, led turned off <<<")
14
15 if __name__ == '__main__':    # Program entrance
16     print ('Program is starting...')
17     try:
18         loop()
19     except KeyboardInterrupt: # Press ctrl-c to end the program.
20         print("Ending program")

```

Import the Button class that controls Button from the gpiozero library.

```
from gpiozero import LED, Button
```

Define GPIO17 as the LED control pin and GPIO18 as the button control pin. The button is set to the input mode with a pull-up resistor by default.

```

led = LED(17)      # define LED pin according to BCM Numbering
button = Button(18) # define Button pin according to BCM Numbering

```

The loop continuously determines whether the key is pressed. When the button is pressed, the variable button.is_pressed has a value of 1 and the LED lights up. Otherwise, the LED will be off.

```

def loop():
    while True:
        if button.is_pressed: # if button is pressed
            led.on()          # turn on led
            print("Button is pressed, led turned on >>>") # print information on terminal
        else : # if button is released
            led.off() # turn off led
            print("Button is released, led turned on <<<")

```

For more information about GPIOZero, please refer to the link below:

<https://gpiozero.readthedocs.io/en/stable/>

For more information about the methods used by the Button class in the GPIO Zero library, please refer to:

https://gpiozero.readthedocs.io/en/stable/api_input.html#button

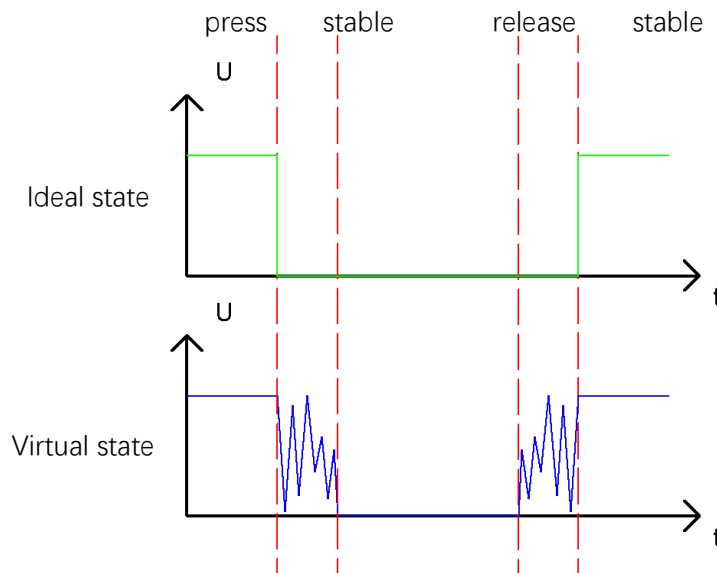
Project 2.2 MINI Table Lamp

We will also use a Push Button Switch, LED and RPi to make a MINI Table Lamp but this will function differently: Press the button, the LED will turn ON, and pressing the button again, the LED turns OFF. The ON switch action is no longer momentary (like a door bell) but remains ON without needing to continually press on the Button Switch.

First, let us learn something about the push button switch.

Debounce a Push Button Switch

When a Momentary Push Button Switch is pressed, it will not change from one state to another state immediately. Due to tiny mechanical vibrations, there will be a short period of continuous buffeting before it stabilizes in a new state too fast for Humans to detect but not for computer microcontrollers. The same is true when the push button switch is released. This unwanted phenomenon is known as “bounce”.



Therefore, if we can directly detect the state of the Push Button Switch, there are multiple pressing and releasing actions in one pressing cycle. This buffeting will mislead the high-speed operation of the microcontroller to cause many false decisions. Therefore, we need to eliminate the impact of buffeting. Our solution: to judge the state of the button multiple times. Only when the button state is stable (consistent) over a period of time, can it indicate that the button is actually in the ON state (being pressed).

This project needs the same components and circuits as we used in the previous section.

Code

In this project, we still detect the state of Push Button Switch to control an LED. Here we need to define a variable to define the state of LED. When the button switch is pressed once, the state of LED will be changed once. This will allow the circuit to act as a virtual table lamp.

Python Code 2.2.1 Tablelamp

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 02.2.1_Tablelamp directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOWZero_Code/02.2.1_Tablelamp
```

2. Use python command to execute python code "Tablelamp.py".

```
python Tablelamp.py
```

When the program is executed, pressing the Button Switch once turns the LED ON. Pressing the Button Switch again turns the LED OFF.

```
1  from gpiozero import LED, Button
2  import time
3
4  led = LED(17) # define LED pin according to BCM Numbering
5  button = Button(18) # define Button pin according to BCM Numbering
6
7  def onButtonPressed(): # When button is pressed, this function will be executed
8      led.toggle()
9      if led.is_lit :
10         print("Led turned on >>>")
11     else :
12         print("Led turned off <<<")
13
14 def loop():
15     #Button detect
16     button.when_pressed = onButtonPressed
17     while True:
18         time.sleep(1)
19
20 def destroy():
21     led.close()
22     button.close()
23
24 if __name__ == '__main__': # Program entrance
25     print ('Program is starting...')
26     try:
27         loop()
28     except KeyboardInterrupt: # Press ctrl-c to end the program.
29         destroy()
30         print("Ending program")
```

In GPIO Zero, you assign the `when_pressed` and `when_released` properties to set up callbacks on those actions.

Once it detects that the button is pressed, it executes the specified function `onButtonPressed()`. In the `onButtonPressed` function, the `led.toggle()` function reverses the state of the LED device. If it's on, turn it off; if it's off, turn it on. Each time the key is pressed, the state of the LED will change once.

```
def onButtonPressed(): # When button is pressed, this function will be executed
    led.toggle()
    if led.is_lit :
        print("Led turned on >>>")
    else :
        print("Led turned off <<<")

def loop():
    #Button detect
    button.when_pressed = onButtonPressed
    while True:
        time.sleep(1)
```

To explicitly close a connection to a pin, you can manually call the `close()` method on a device object:

```
def destroy():
    led.close()
    button.close()

except KeyboardInterrupt: # Press ctrl-c to end the program.
    destroy()
    print("Ending program")
```

For more information about the methods used by the Button class in the GPIO Zero library, please refer to:

https://gpiozero.readthedocs.io/en/stable/api_input.html#button

Chapter 3 LED Bar Graph

We have learned how to control one LED to blink. Next, we will learn how to control a number of LEDs.

Project 3.1 Flowing Water Light

In this project, we use a number of LEDs to make a flowing water light.

Component List

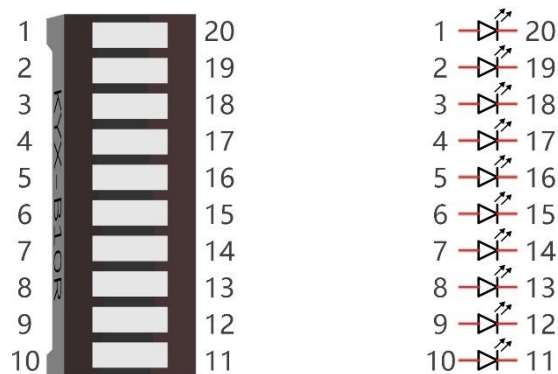
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	Bar Graph LED x1 	Resistor 220Ω x10 
Jumper Wire x 1 		

Component knowledge

Let us learn about the basic features of these components to use and understand them better.

Bar Graph LED

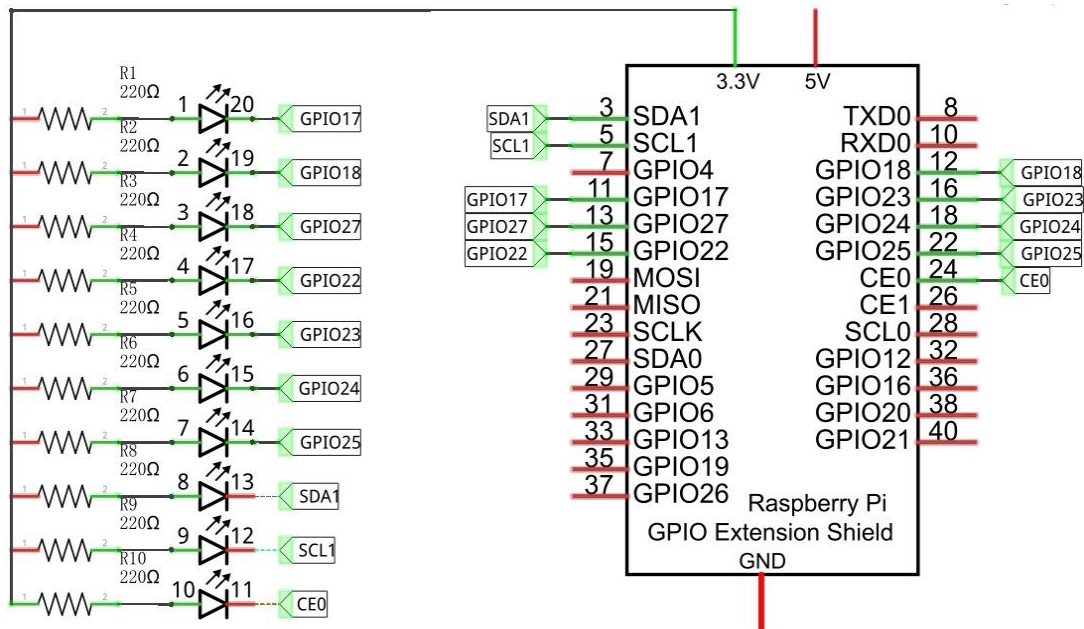
A Bar Graph LED has 10 LEDs integrated into one compact component. The two rows of pins at its bottom are paired to identify each LED like the single LED used earlier.



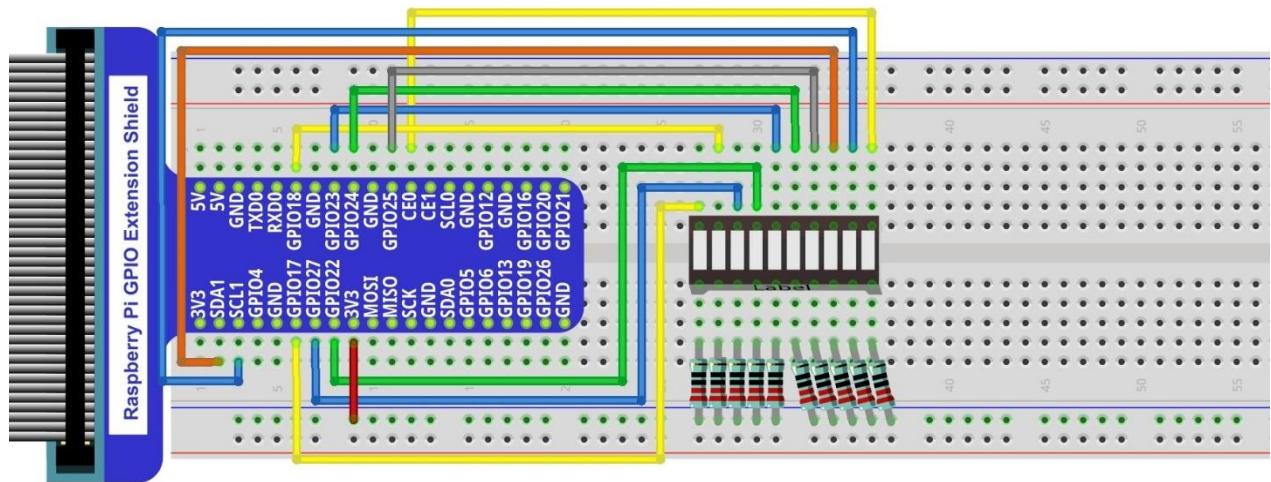
Circuit

A reference system of labels is used in the circuit diagram below. Pins with the same network label are connected together.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



If LEDbar doesn't work, rotate LEDbar 180° to try. The label is random.

Youtube video: <https://youtu.be/3rh-b05VoiU>

In this circuit, the cathodes of the LEDs are connected to the GPIO, which is different from the previous circuit. The LEDs turn ON when the GPIO output is low level in the program.

Code

This project is designed to make a flowing water lamp, which are these actions: First turn LED #1 ON, then

turn it OFF. Then turn LED #2 ON, and then turn it OFF... and repeat the same to all 10 LEDs until the last LED is turns OFF. This process is repeated to achieve the “movements” of flowing water.

Python Code 3.1.1 LightWater

First observe the project result, and then view the code.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 03.1.1_LightWater directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIZero_Code/03.1.1_LightWater
```

2. Use Python command to execute Python code “LightWater.py”.

```
python LightWater.py
```

After the program is executed, you will see that LED Bar Graph starts with the flowing water way to be turned on from left to right, and then from right to left.

The following is the program code:

```
1  from gpiozero import LED
2  from time import sleep
3  ledPins = [17, 18, 27, 22, 23, 24, 25, 2, 3, 8]
4  leds = [LED(pin=pin) for pin in ledPins]
5
6  def loop():
7      while True:
8          for index in range(0, len(ledPins), 1):      # make led(on) move from left to right
9              leds[index].off()
10             sleep(0.1)
11             leds[index].on()
12         for index in range(len(ledPins)-1, -1, -1):  #move led(on) from right to left
13             leds[index].off()
14             sleep(0.1)
15             leds[index].on()
16
17 if __name__ == '__main__':      # Program entrance
18     print ('Program is starting...')
19     try:
20         loop()
21     except KeyboardInterrupt:  # Press ctrl-c to end the program.
22         print("Ending program")
23     finally:
24         for index in range(0, len(ledPins), 1):
25             leds[index].close()
```

Import the LED class that controls LED Bar Graph from the gpiozero library.

```
from gpiozero import LED
```

Create the LED class for controlling the LEDBarGraph.

```
ledPins = [17, 18, 27, 22, 23, 24, 25, 2, 3, 8]
leds = [LED(pin=pin) for pin in ledPins]
```

The LED is turned on or off by specifying the index of the LED, if no parameter is specified, the same Settings are applied to all leds. `leds.off()` means that all leds are turned off.

```
for index in range(0, len(ledPins), 1):    # make led(on) move from left to right
    leds[index].off()
    sleep(0.1)
    leds[index].on()
for index in range(len(ledPins)-1, -1, -1): #move led(on) from right to left
    leds[index].off()
    sleep(0.1)
    leds[index].on()
```

In the program, first define 10 pins connected to the LED and set them to output mode. In the `loop()` function, two for loops are used to make the lights flow from right to left and from left to right.

```
def loop():
    while True:
        for index in range(0, len(ledPins), 1):    # make led(on) move from left to right
            leds.off(index)
            sleep(0.1)
            leds.on(index)
        for index in range(len(ledPins)-1, -1, -1): #move led(on) from right to left
            leds.off(index)
            sleep(0.1)
            leds.on(index)
```

For more information about the methods used by the LEDBoard class in the GPIO Zero library, please refer to: https://gpiozero.readthedocs.io/en/stable/api_boards.html#ledboard

For more information about the methods used by the LEDBarGraph class in the GPIO Zero library, please refer to: https://gpiozero.readthedocs.io/en/stable/api_boards.html#led bargraph

In this experiment you can use the LEDBoard and LEDBarGraph classes to control the LEDBarGraph

Chapter 4 Analog & PWM

In previous chapters, we learned that a Push Button Switch has two states: Pressed (ON) and Released (OFF), and an LED has a Light ON and OFF state. Is there a middle or intermediated state? We will next learn how to create an intermediate output state to achieve a partially bright (dim) LED.

First, let us learn how to control the brightness of an LED.

Project 4.1 Breathing LED

We describe this project as a Breathing Light. This means that an LED that is OFF will then turn ON gradually and then gradually turn OFF like "breathing". Okay, so how do we control the brightness of an LED to create a Breathing Light? We will use PWM to achieve this goal.

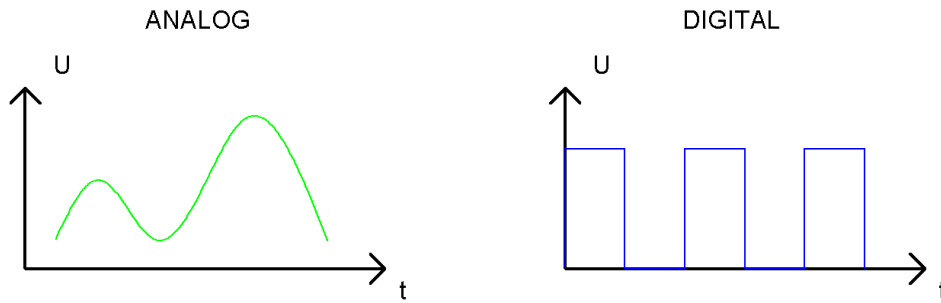
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	LED x1 	Resistor 220Ω x1 
Jumper Wire 		

Component Knowledge

Analog & Digital

An Analog Signal is a continuous signal in both time and value. On the contrary, a Digital Signal or discrete-time signal is a time series consisting of a sequence of quantities. Most signals in life are analog signals. A familiar example of an Analog Signal would be how the temperature throughout the day is continuously changing and could not suddenly change instantaneously from 0°C to 10°C. However, Digital Signals can instantaneously change in value. This change is expressed in numbers as 1 and 0 (the basis of binary code). Their differences can more easily be seen when compared when graphed as below.



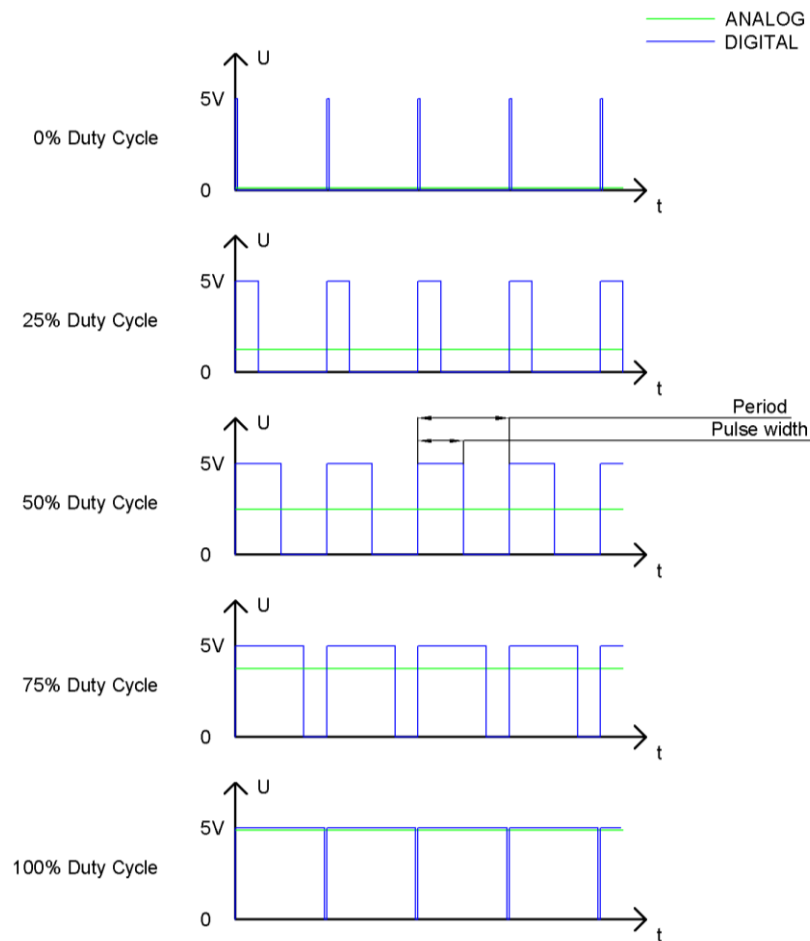
Note that the Analog signals are curved waves and the Digital signals are “Square Waves”.

In practical applications, we often use binary as the digital signal, that is a series of 0's and 1's. Since a binary signal only has two values (0 or 1) it has great stability and reliability. Lastly, both analog and digital signals can be converted into the other.

PWM

PWM, Pulse-Width Modulation, is a very effective method for using digital signals to control analog circuits. Digital processors cannot directly output analog signals. PWM technology makes it very convenient to achieve this conversion (translation of digital to analog signals).

PWM technology uses digital pins to send certain frequencies of square waves, that is, the output of high levels and low levels, which alternately last for a while. The total time for each set of high levels and low levels is generally fixed, which is called the period (Note: the reciprocal of the period is frequency). The time of high level outputs are generally called “pulse width”, and the duty cycle is the percentage of the ratio of pulse duration, or pulse width (PW) to the total period (T) of the waveform. The longer the output of high levels last, the longer the duty cycle and the higher the corresponding voltage in the analog signal will be. The following figures show how the analog signal voltages vary between 0V-5V (high level is 5V) corresponding to the pulse width 0%-100%:



The longer the PWM duty cycle is, the higher the output power will be. Now that we understand this relationship, we can use PWM to control the brightness of an LED or the speed of DC motor and so on.

It is evident, from the above, that PWM is not actually analog but the effective value of voltage is equivalent to the corresponding analog value. Therefore, by using PWM, we can control the output power of to an LED and control other devices and modules to achieve multiple effects and actions.

In RPi, GPIO18 pin has the ability to output to hardware via PWM with a 10-bit accuracy. This means that 100% of the pulse width can be divided into $2^{10}=1024$ equal parts.

The wiringPi library of C provides both a hardware PWM and a software PWM method, while the wiringPi library of Python does not provide a hardware PWM method. There is only a software PWM option for Python.

The hardware PWM only needs to be configured, does not require CPU resources and is more precise in time control. The software PWM requires the CPU to work continuously by using code to output high level and low level. This part of the code is carried out by multi-threading, and the accuracy is relatively not high enough.

In order to keep the results running consistently, we will use software PWM.

Circuit

Schematic diagram

Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

Youtube video: <https://youtu.be/rYxykuVgYtA>

Code

This project uses the PWM output from the GPIO18 pin to make the pulse width gradually increase from 0% to 100% and then gradually decrease from 100% to 0% to make the LED glow brighter then dimmer.

Python Code 4.1.1 BreathingLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 04.1.1_BreathingLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/04.1.1_BreathingLED
```

2. Use the Python command to execute Python code "BreathingLED.py".

```
python BreathingLED.py
```

After the program is executed, you will see that the LED gradually turns ON and then gradually turns OFF similar to "breathing".

The following is the program code:

```
1 from gpiozero import PWMLED
2 import time
3
4 led = PWMLED(18 , initial_value=0 , frequency=1000)
5 def loop():
6     while True:
7         for b in range(0, 101, 1):      # make the led brighter
8             led.value = b / 100.0      # set dc value as the duty cycle
```



```

9         time.sleep(0.01)
10        time.sleep(1)
11        for b in range(100, -1, -1): # make the led darker
12            led.value = b / 100.0     # set dc value as the duty cycle
13            time.sleep(0.01)
14        time.sleep(1)
15    def destroy():
16        led.close()
17    if __name__ == '__main__':      # Program entrance
18        print ('Program is starting ... ')
19        try:
20            loop()
21        except KeyboardInterrupt: # Press ctrl-c to end the program.
22            destroy()
23            print("Ending program")

```

Import the PWMLED class that controls leds from the gpiozero library.

```
from gpiozero import PWMLED
```

Create the PWMLED class for controlling the LED.

```
led = PWMLED(18 , initial_value=0 , frequency=1000)
```

PWMLED is connected to GPIO18, and its PWM frequency is set to 1000HZ, and the initial duty cycle to 0%.

```
led = PWMLED(18 , initial_value=0 , frequency=1000) # Set the PWM frequency to 1000Hz and the
initial duty cycle to 0
```

There are two “for” loops used to control the breathing LED in the next endless “while” loop. The first loop outputs a power signal to the led PWM from 0% to 100% and the second loop outputs a power signal to the led PWM from 100% to 0%.

led.value represents: The duty cycle of the PWM device. 0.0 is off, 1.0 is fully on. led.value in between may be specified for varying levels of power in the device.

```

def loop():
    while True:
        for b in range(0, 101, 1): # make the led brighter
            led.value = b / 100.0   # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)
        for b in range(100, -1, -1): # make the led darker
            led.value = b / 100.0   # set dc value as the duty cycle
            time.sleep(0.01)
        time.sleep(1)

```

For more information about the methods used by the PWMLED class in the GPIO Zero library, please refer to:

https://gpiozero.readthedocs.io/en/stable/api_output.html#pwmled

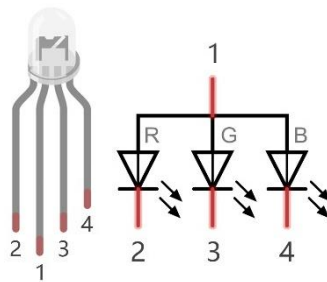
For more information about the methods used by the PWMOutputDevice class in the GPIO Zero library, please

refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#pwmoutputdevice

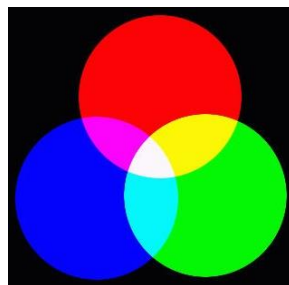
Chapter 5 RGB LED

In this chapter, we will learn how to control a RGB LED.

An RGB LED has 3 LEDs integrated into one LED component. It can respectively emit Red, Green and Blue light. In order to do this, it requires 4 pins (this is also how you identify it). The long pin (1) is the common which is the Anode (+) or positive lead, the other 3 are the Cathodes (-) or negative leads. A rendering of a RGB LED and its electronic symbol are shown below. We can make RGB LED emit various colors of light and brightness by controlling the 3 Cathodes (2, 3 & 4) of the RGB LED



Red, Green, and Blue light are called 3 Primary Colors when discussing light (Note: for pigments such as paints, the 3 Primary Colors are Red, Blue and Yellow). When you combine these three Primary Colors of light with varied brightness, they can produce almost any color of visible light. Computer screens, single pixels of cell phone screens, neon lamps, etc. can all produce millions of colors due to phenomenon.



RGB

If we use a three 8 bit PWM to control the RGB LED, in theory, we can create $2^8 * 2^8 * 2^8 = 16777216$ (16 million) colors through different combinations of RGB light brightness.

Next, we will use RGB LED to make a multicolored LED.

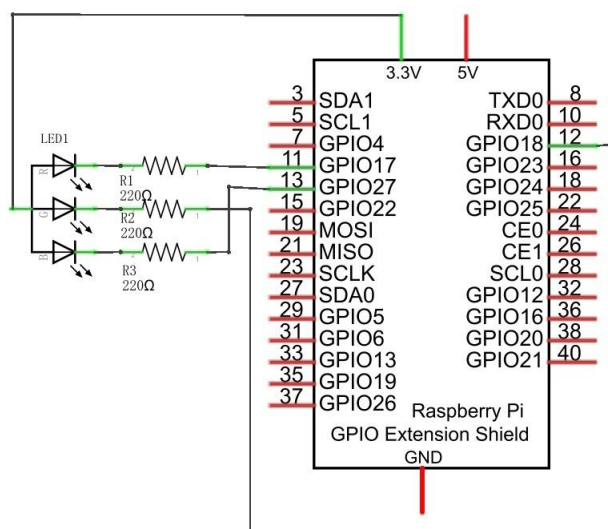
Project 5.1 Multicolored LED

Component List

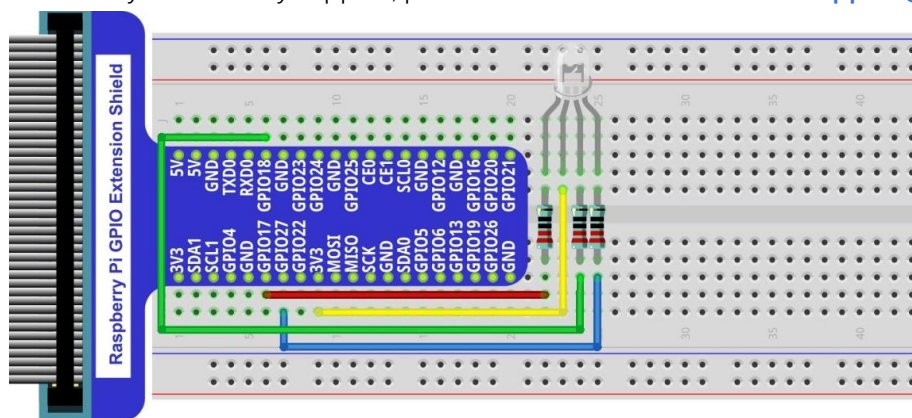
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Wire x1 Breadboard x1	RGB LED x1 	Resistor 220Ω x3 
Jumper Wire 		

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

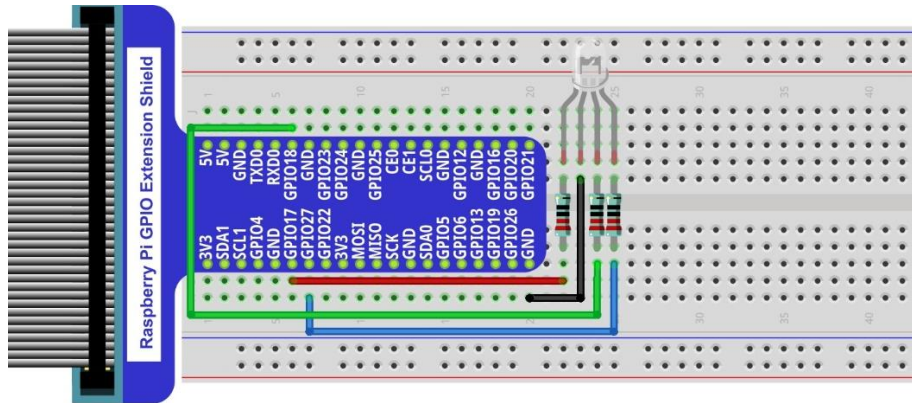


Video: <https://youtu.be/tbnX2AsX2y4>

In this kit, the RGB led is **Common anode**. The **voltage difference** between LED will make it work. There is no visible GND. The GPIO ports can also receive current while in output mode.

If circuit above doesn't work, the RGB LED may be common cathode. Please try following wiring.

There is no need to modify code for random color.



Code

We need to use RGBLED class to control RGBLED. The parameters for setting the RGBLED as common cathode or common anode are provided in the RGBLED class. You can set it according to the type of your RGB LED, and the default setting in our example code is based on common anode.

Python Code 5.1.1 ColorfulLED

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 05.1.1_ColorfulLED directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless_Code/05.1.1_ColorfulLED
```

2. Use python command to execute python code "ColorfulLED.py".

```
python ColorfulLED.py
```

After the program is executed, you will see that the RGB LED randomly lights up different colors.

The following is the program code:

```
1 from gpiozero import RGBLED
2 import time
3 import random
4
5 #led = RGBLED(red="J8:11", green="J8:12", blue="J8:13", active_high=False) # define the pins
  for R:11,G:12,B:13
6 led = RGBLED(red=17, green=18, blue=27, active_high=False) # define the pins for
  R:GPIO17,G:GPIO18,B:GPIO27
7 # If your RGBLED is a common cathode LED, set active_high to True
8
9 def setColor(r_val, g_val, b_val):      # change duty cycle for three pins to r_val, g_val, b_val
10     led.red=r_val/100                  # change pwmRed duty cycle to r_val
11     led.green = g_val/100              # change pwmRed duty cycle to r_val
12     led.blue = b_val/100               # change pwmRed duty cycle to r_val
13
14 def loop():
```

```

15     while True :
16         r=random.randint(0,100)  #get a random in (0,100)
17         g=random.randint(0,100)
18         b=random.randint(0,100)
19         setColor(r,g,b)          #set random as a duty cycle value
20         print ('r=%d, g=%d, b=%d ' %(r ,g, b))
21         time.sleep(1)
22
23 def destroy():
24     led.close()
25
26 if __name__ == '__main__':      # Program entrance
27     print ('Program is starting ... ')
28     try:
29         loop()
30     except KeyboardInterrupt:  # Press ctrl-c to end the program.
31         destroy()
32     print("Ending program")

```

Import the RGBLED class that controls RGBLED from the gpiozero library.

```
from gpiozero import RGBLED
```

Create the RGBLED class for controlling the RGBLED.

```
led = RGBLED(red=17, green=18, blue=27, active_high=False) # define the pins for
R:GPIO17,G:GPIO18,B:GPIO27
```

In the previous chapter, we learned how to make a pin output PWM using the Python language. In this project, we output to three pins via PWM. In the "while" loop of the "loop" function, we first generate three random numbers and then assign these three random numbers to the PWM values of the three pins, which will make the RGB LED randomly produce multiple colors.

```

def loop():
    while True :
        r=random.randint(0,100)  #get a random in (0,100)
        g=random.randint(0,100)
        b=random.randint(0,100)
        setColor(r,g,b)          #set random as a duty cycle value
        print ('r=%d, g=%d, b=%d ' %(r ,g, b))
        time.sleep(1)

```

For more information about the methods used by the RGBLED class in the GPIO Zero library, please refer to:

https://gpiozero.readthedocs.io/en/stable/api_output.html#rgbled

Chapter 6 Buzzer

In this chapter, we will learn about buzzers and the sounds they make. And in our next project, we will use an active buzzer to make a doorbell and a passive buzzer to make an alarm.

Project 6.1 Doorbell

We will make a doorbell with this functionality: when the Push Button Switch is pressed the buzzer sounds and when the button is released, the buzzer stops. This is a momentary switch function.

Component List

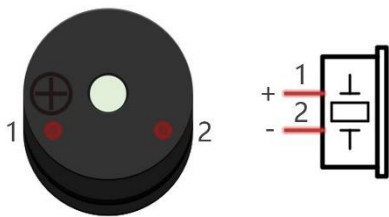
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1		Jumper Wire 		
NPN transistorx1 (S8050) 	Active buzzer x1 	Push Button Switch x1 	Resistor 1kΩ x1 	Resistor 10kΩ x2 

Component knowledge

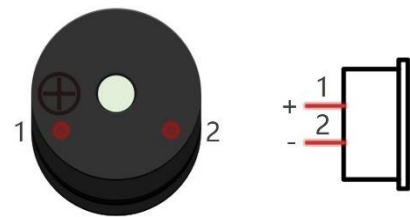
Buzzer

A buzzer is an audio component. They are widely used in electronic devices such as calculators, electronic alarm clocks, automobile fault indicators, etc. There are both active and passive types of buzzers. Active buzzers have oscillator inside, these will sound as long as power is supplied. Passive buzzers require an external oscillator signal (generally using PWM with different frequencies) to make a sound.

Active buzzer



Passive buzzer



Active buzzers are easier to use. Generally, they only make a specific sound frequency. Passive buzzers require an external circuit to make sounds, but passive buzzers can be controlled to make sounds of various frequencies. The resonant frequency of the passive buzzer in this Kit is 2kHz, which means the passive buzzer is the loudest when its resonant frequency is 2kHz.

How to identify active and passive buzzer?

1. As a rule, there is a label on an active buzzer covering the hole where sound is emitted, but there are exceptions to this rule.
2. Active buzzers are more complex than passive buzzers in their manufacture. There are many circuits and crystal oscillator elements inside active buzzers; all of this is usually protected with a waterproof coating (and a housing) exposing only its pins from the underside. On the other hand, passive buzzers do not have protective coatings on their underside. From the pin holes, view of a passive buzzer, you can see the circuit board, coils, and a permanent magnet (all or any combination of these components depending on the model).



Active buzzer bottom



Passive buzzer bottom

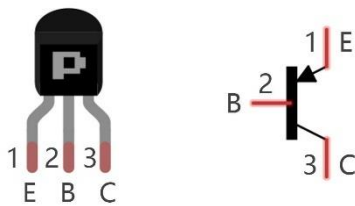
Transistors

A transistor is required in this project due to the buzzer's current being so great that GPIO of RPi's output capability cannot meet the power requirement necessary for operation. A NPN transistor is needed here to

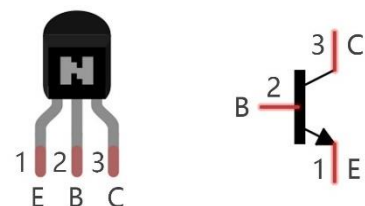
amplify the current.

Transistors, full name: semiconductor transistor, is a semiconductor device that controls current (think of a transistor as an electronic “amplifying or switching device”). Transistors can be used to amplify weak signals, or to work as a switch. Transistors have three electrodes (PINs): base (b), collector (c) and emitter (e). When there is current passing between “be” then “ce” will have a several-fold current increase (transistor magnification), in this configuration the transistor acts as an amplifier. When current produced by “be” exceeds a certain value, “ce” will limit the current output. at this point the transistor is working in its saturation region and acts like a switch. Transistors are available as two types as shown below: PNP and NPN,

PNP transistor



NPN transistor



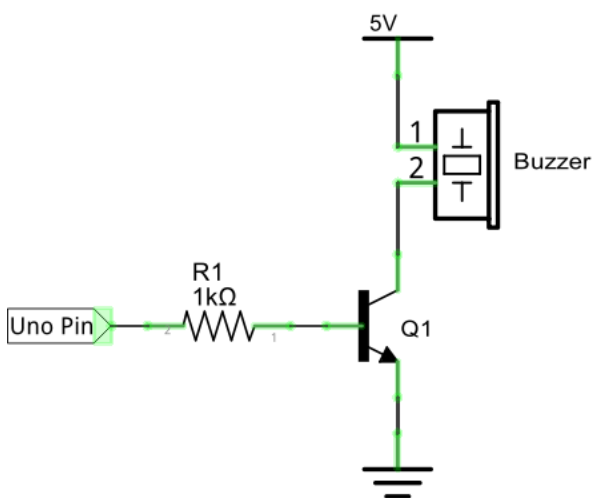
In our kit, the PNP transistor is marked with 8550, and the NPN transistor is marked with 8050.

Thanks to the transistor's characteristics, they are often used as switches in digital circuits. As micro-controllers output current capacity is very weak, we will use a transistor to amplify its current in order to drive components requiring higher current.

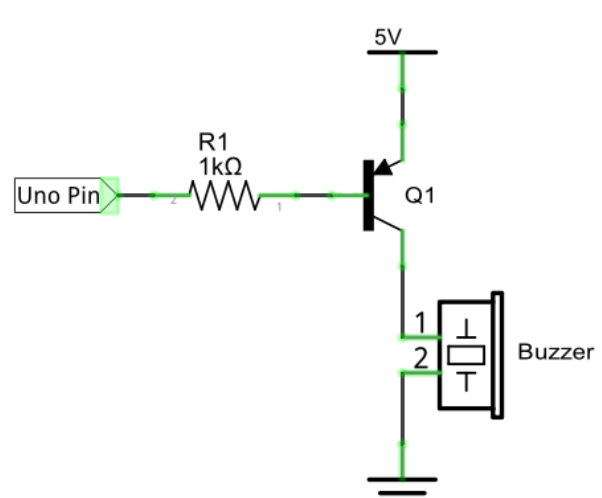
When we use a NPN transistor to drive a buzzer, we often use the following method. If GPIO outputs high level, current will flow through R1 (Resistor 1), the transistor conducts current and the buzzer will make sounds. If GPIO outputs low level, no current will flow through R1, the transistor will not conduct current and buzzer will remain silent (no sounds).

When we use a PNP transistor to drive a buzzer, we often use the following method. If GPIO outputs low level, current will flow through R1. The transistor conducts current and the buzzer will make sounds. If GPIO outputs high level, no current flows through R1, the transistor will not conduct current and buzzer will remain silent (no sounds). Below are the circuit schematics for both a NPN and PNP transistor to power a buzzer.

NPN transistor to drive buzzer

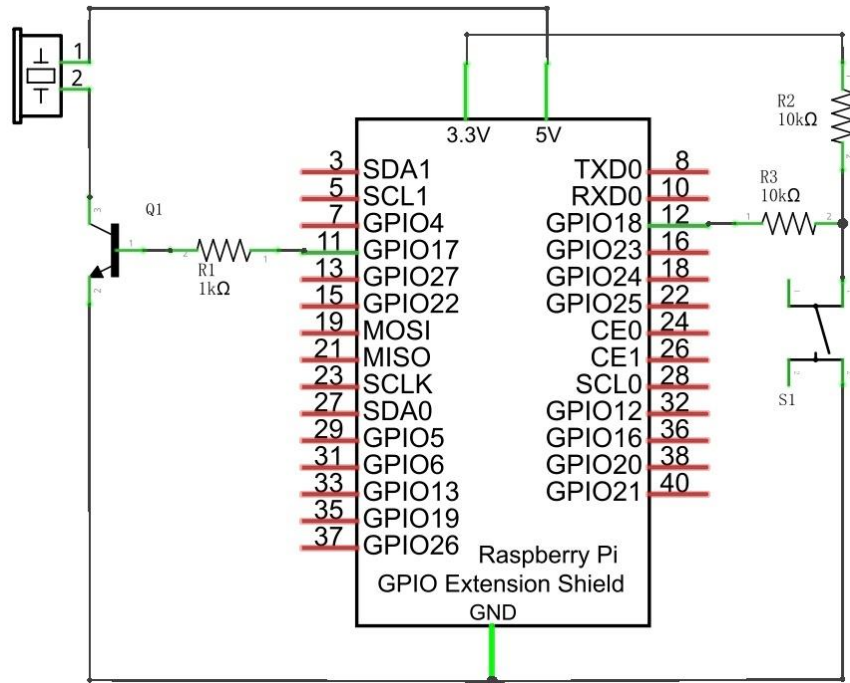


PNP transistor to drive buzzer

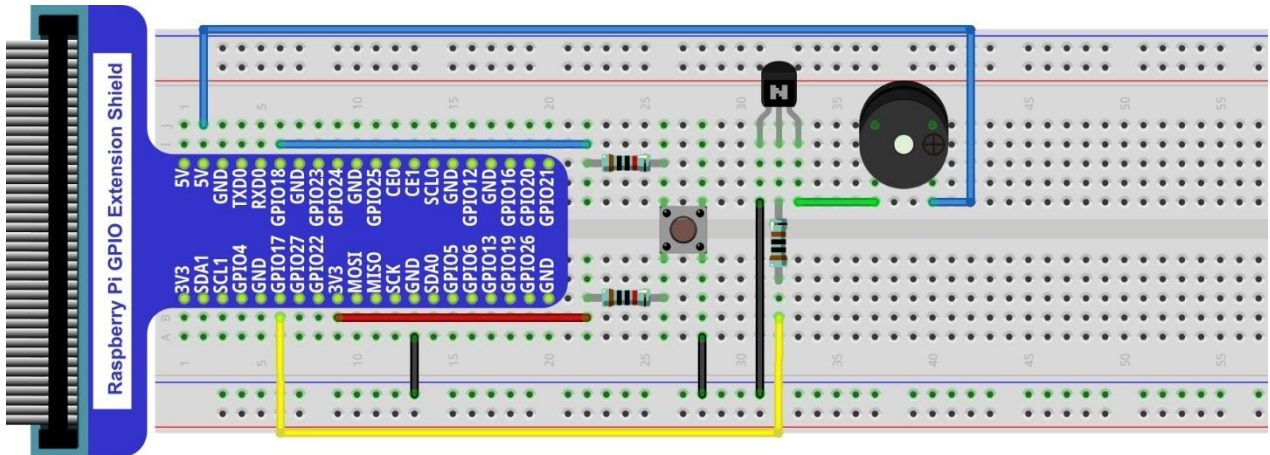


Circuit

Schematic diagram with RPi GPIO Extension Shield.



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://youtu.be/R_dmi3YwY-U

Note: in this circuit, the power supply for the buzzer is 5V, and pull-up resistor of the push button switch is connected to the 3.3V power feed. Actually, the buzzer can work when connected to the 3.3V power feed but this will produce a weak sound from the buzzer (not very loud).

Code

In this project, a buzzer will be controlled by a push button switch. When the button switch is pressed, the buzzer sounds and when the button is released, the buzzer stops. It is analogous to our earlier project that controlled an LED ON and OFF.

Python Code 6.1.1 Doorbell

First, observe the project result, then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 06.1.1_Doorbell directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless_Code/06.1.1_Doorbell
```

2. Use python command to execute python code "Doorbell.py".

```
python Doorbell.py
```

After the program is executed, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.

The following is the program code:

```
1  from gpiozero import Buzzer, Button
2  import time
3
4  buzzer = Buzzer(17)
5  button = Button(18)
6
7  def onButtonPressed():
8      buzzer.on()
9      print("Button is pressed, buzzer turned on >>>")
10
11 def onButtonReleased():
12     buzzer.off()
13     print("Button is released, buzzer turned on <<<")
14
15 def loop():
16     button.when_pressed = onButtonPressed
17     button.when_released = onButtonReleased
18     while True :
19         time.sleep(1)
20
21 def destroy():
22     buzzer.close()
23     button.close()
24
25 if __name__ == '__main__':    # Program entrance
26     print ('Program is starting ... ')
27     try:
28         loop()
29     except KeyboardInterrupt: # Press ctrl-c to end the program.
30         destroy()
31     print("Ending program")
```

The code is exactly the same as when we used a push button switch to control an LED. You can also try using

the PNP transistor to achieve the same results.

Import the Buzzer class that controls Buzzer from the gpiozero library.

```
from gpiozero import Buzzer, Button
```

Create the Buzzer class for controlling the Buzzer.

```
buzzer = Buzzer(17)
```

In GPIO Zero, you assign the when_pressed and when_released properties to set up callbacks on those actions.

Once it detects that the button is pressed, it executes the specified function onButtonPressed(). Once it detects that the button is released, it executes the specified function onButtonReleased()

```
def loop():  
    button.when_pressed = onButtonPressed  
    button.when_released = onButtonReleased
```

For more information about the methods used by the Buzzer class in the GPIO Zero library, please refer to:

https://gpiozero.readthedocs.io/en/stable/api_output.html#buzzer

Project 6.2 Alertor

Next, we will use a passive buzzer to make an alarm.

The list of components and the circuit is similar to the doorbell project. We only need to take the Doorbell circuit and replace the active buzzer with a passive buzzer.

Code

In this project, our buzzer alarm is controlled by the push button switch. Press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.

As stated before, it is analogous to our earlier project that controlled an LED ON and OFF.

To control a passive buzzer requires PWM of certain sound frequency.

Python Code 6.2.1 Alertor

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 06.2.1_Alertor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless_Code/06.2.1_Alertor
```

2. Use the python command to execute the Python code "Alertor.py".

```
python Alertor.py
```

After the program is executed, press the push button switch and the buzzer will sound. Release the push button switch and the buzzer will stop.

The following is the program code:

```
1  from gpiozero import TonalBuzzer, Button
2  from gpiozero.tones import Tone
3  import time
4  import math
5
6  buzzer = TonalBuzzer(17)
7  button = Button(18) # define Button pin according to BCM Numbering
8
9  def loop():
10     while True:
11         if button.is_pressed: # if button is pressed
12             alertor()
13             print ('alertor turned on >>> ')
14         else :
15             stopAlertor()
16             print ('alertor turned off <<<')
17
18  def alertor():
19     for x in range(0,361): # Make frequency of the alertor consistent with the sine wave
20         sinVal = math.sin(x * (math.pi / 180.0)) # calculate the sine value
```

```

20         toneVal = 2000 + sinVal * 500    # Add to the resonant frequency with a Weighted
21         buzzer.play(Tone(toneVal))    # Change Frequency of PWM to toneVal
22         time.sleep(0.001)
23
24     def stopAlertor():
25         buzzer.stop()
26
27     def destroy():
28         buzzer.close()
29
30     if __name__ == '__main__':    # Program entrance
31         print('Program is starting...')
32         try:
33             loop()
34         except KeyboardInterrupt:    # Press ctrl-c to end the program.
35             destroy()
36             print("Ending program")

```

Define GPIO17 as the buzzer control pin, and GPIO18 as the button control pin to control the passive buzzer. It requires a certain frequency of PWM to control a passive buzzer, so the TonalBuzzer class is needed.

```

buzzer = TonalBuzzer(17)
button = Button(18) # define Button pin according to BCM Numbering

```

In the while loop loop of the main function, when the push button switch is pressed the subfunction alertor() will be called and the alarm will issue a warning sound. The frequency curve of the alarm is based on a sine curve. We need to calculate the sine value from 0 to 360 degrees and multiplied by a certain value (here this value is 500) plus the resonant frequency of buzzer. We can set the PWM frequency through Tone(toneVal).

```

def alertor():
    buzzer.play(Tone(220.0))
    time.sleep(0.1)

```

When the push button switch is released, the buzzer (in this case our Alarm) will stop.

```

def stopAlertor():
    buzzer.stop()

```

For more information about the methods used by the TonalBuzzer class in the GPIO Zero library, please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#tonalbuzzer

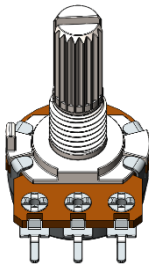
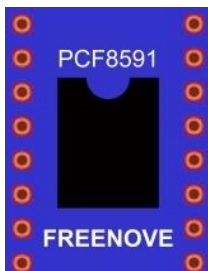
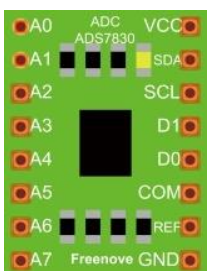
(Important) Chapter 7 ADC

We have learned how to control the brightness of an LED through PWM and that PWM is not a real analog signal. In this chapter, we will learn how to read analog values via an ADC Module and convert these analog values into digital.

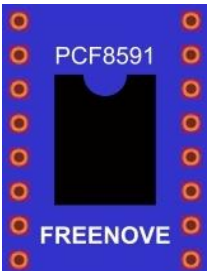
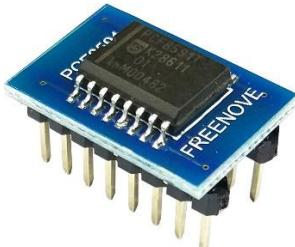
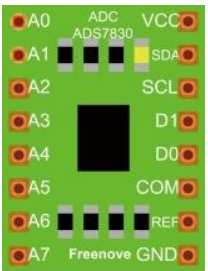
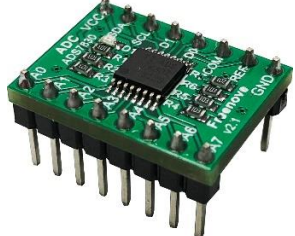
Project 7.1 Read the Voltage of Potentiometer

In this project, we will use the ADC function of an ADC Module to read the voltage value of a potentiometer.

Component List

Raspberry Pi x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1		Jumper Wire M/M x16 
Rotary potentiometer x1 	ADC module x1  or 	Resistor 10kΩ x2 

This product contains **only one ADC module**, there are two types, PCF8591 and ADS7830. For the projects described in this tutorial, they function the same. Please build corresponding circuits according to the ADC module found in your Kit.

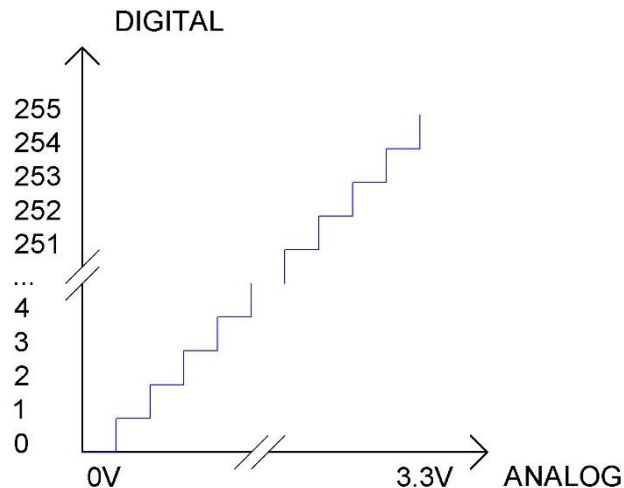
ADC module: PCF8591		ADC module: ADS7830	
Model diagram	Actual Picture	Model diagram	Actual Picture
			

Circuit knowledge

ADC

An ADC is an electronic integrated circuit used to convert analog signals such as voltages to digital or binary form consisting of 1s and 0s. The range of our ADC module is 8 bits, that means the resolution is $2^8=256$, so that its range (at 3.3V) will be divided equally to 256 parts.

Any analog value can be mapped to one digital value using the resolution of the converter. So the more bits the ADC has, the denser the partition of analog will be and the greater the precision of the resulting conversion.



Subsection 1: the analog in range of $0V-3.3/256$ V corresponds to digital 0;

Subsection 2: the analog in range of $3.3/256$ V- $2*3.3/256$ V corresponds to digital 1;

...

The resultant analog signal will be divided accordingly.

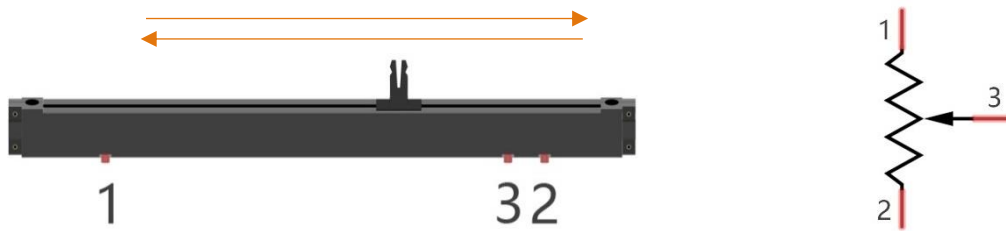
DAC

The reversing this process requires a DAC, Digital-to-Analog Converter. The digital I/O port can output high level and low level (0 or 1), but cannot output an intermediate voltage value. This is where a DAC is useful. The DAC module PCF8591 has a DAC output pin with 8-bit accuracy, which can divide VDD (here is 3.3V) into $2^8=256$ parts. For example, when the digital quantity is 1, the output voltage value is $3.3/256 * 1$ V, and when the digital quantity is 128, the output voltage value is $3.3/256 * 128=1.65$ V, the higher the accuracy of DAC, the higher the accuracy of output voltage value will be.

Component knowledge

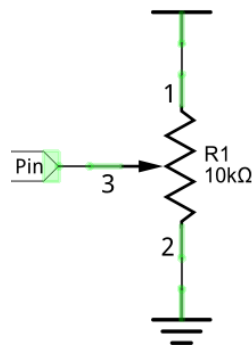
Potentiometer

Potentiometer is a resistive element with three Terminal parts. Unlike the resistors that we have used thus far in our project which have a fixed resistance value, the resistance value of a potentiometer can be adjusted. A potentiometer is often made up by a resistive substance (a wire or carbon element) and movable contact brush. When the brush moves along the resistor element, there will be a change in the resistance of the potentiometer's output side (3) (or change in the voltage of the circuit that is a part). The illustration below represents a linear sliding potentiometer and its electronic symbol on the right.



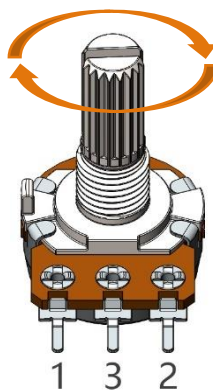
Between potentiometer pin 1 and pin 2 is the resistive element (a resistance wire or carbon) and pin 3 is connected to the brush that makes contact with the resistive element. In our illustration, when the brush moves from pin 1 to pin 2, the resistance value between pin 1 and pin 3 will increase linearly (until it reaches the highest value of the resistive element) and at the same time the resistance between pin 2 and pin 3 will decrease linearly and conversely down to zero. At the midpoint of the slider the measured resistance values between pin 1 and 3 and between pin 2 and 3 will be the same.

In a circuit, both sides of resistive element are often connected to the positive and negative electrodes of power. When you slide the brush "pin 3", you can get variable voltage within the range of the power supply.



Rotary potentiometer

Rotary potentiometers and linear potentiometers have the same function; the only difference being the physical action being a rotational rather than a sliding movement.



PCF8591

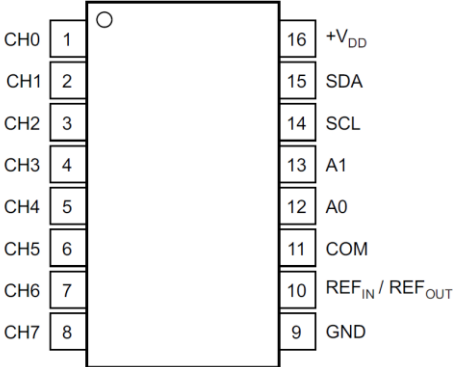
The PCF8591 is a single-chip, single-supply low power 8-bit CMOS data acquisition device with four analog inputs, one analog output and a serial I2C-bus interface. The following table is the pin definition diagram of PCF8591.

SYMBOL	PIN	DESCRIPTION	TOP VIEW
AIN0	1	Analog inputs (A/D converter)	AIN0 1 16 V_{DD} AIN1 2 15 AOUT AIN2 3 14 V_{REF} AIN3 4 13 AGND A0 5 12 EXT A1 6 11 OSC A2 7 10 SCL V_{SS} 8 9 SDA
AIN1	2		
AIN2	3		
AIN3	4		
A0	5	Hardware address	
A1	6		
A2	7		
Vss	8	Negative supply voltage	
SDA	9	I2C-bus data input/output	
SCL	10	I2C-bus clock input	
OSC	11	Oscillator input/output	
EXT	12	external/internal switch for oscillator input	
AGND	13	Analog ground	
Vref	14	Voltage reference input	
AOUT	15	Analog output(D/A converter)	
Vdd	16	Positive supply voltage	

For more details about PCF8591, please refer to the datasheet which can be found on the Internet.

ADS7830

The ADS7830 is a single-supply, low-power, 8-bit data acquisition device that features a serial I2C interface and an 8-channel multiplexer. The following table is the pin definition diagram of ADS7830.

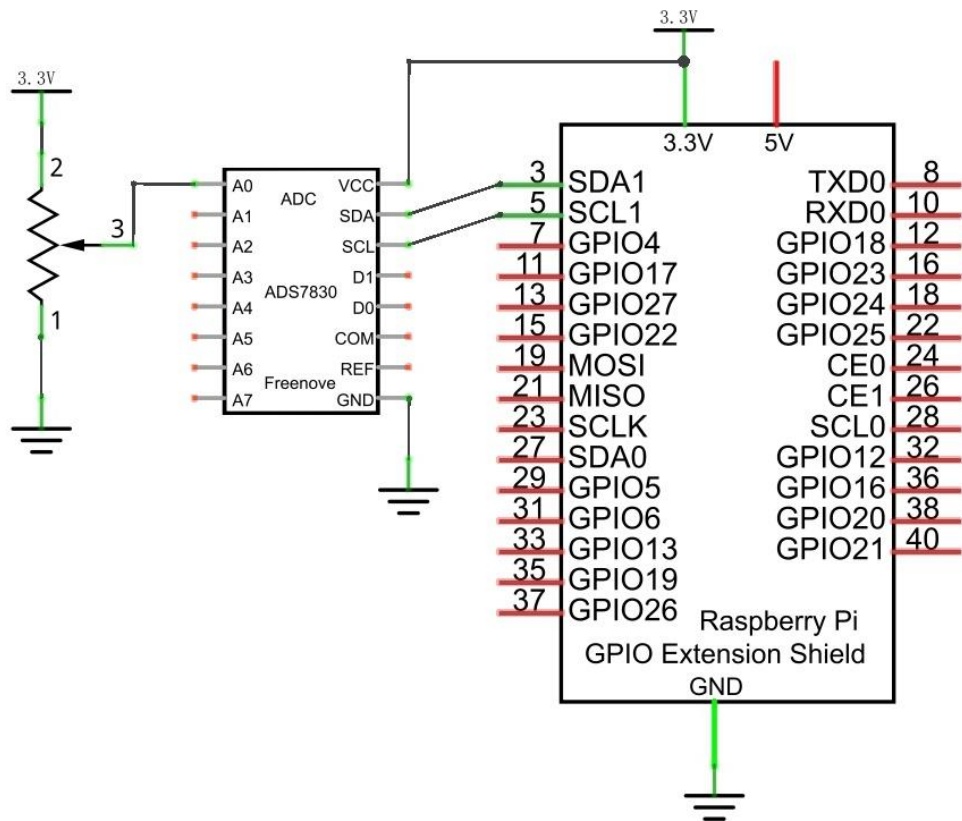
SYMBOL	PIN	DESCRIPTION	TOP VIEW
CH0	1	Analog input channels (A/D converter)	
CH1	2		
CH2	3		
CH3	4		
CH4	5		
CH5	6		
CH6	7		
CH7	8		
GND	9	Ground	
REF in/out	10	Internal +2.5V Reference, External Reference Input	
COM	11	Common to Analog Input Channel	
A0	12	Hardware address	
A1	13		
SCL	14		
SDA	15	Serial Sata	
+VDD	16	Power Supply, 3.3V Nominal	

I2C communication

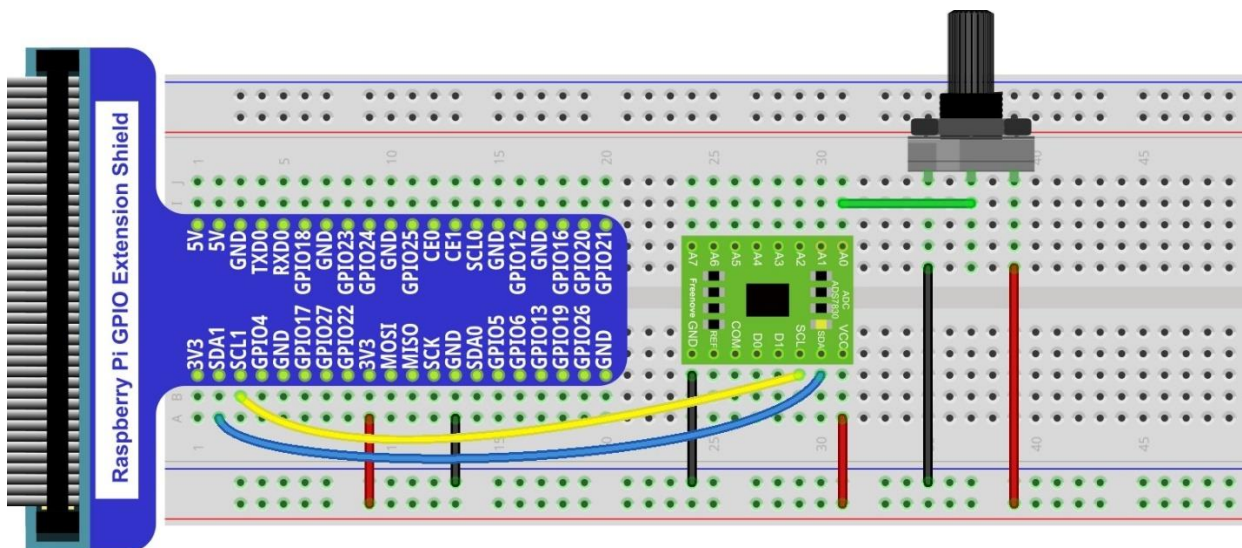
I2C (Inter-Integrated Circuit) has a two-wire serial communication mode, which can be used to connect a micro-controller and its peripheral equipment. Devices using I2C communications must be connected to the serial data line (SDA), and serial clock line (SCL) (called I2C bus). Each device has a unique address which can be used as a transmitter or receiver to communicate with devices connected via the bus.

Circuit with ADS7830

Schematic diagram



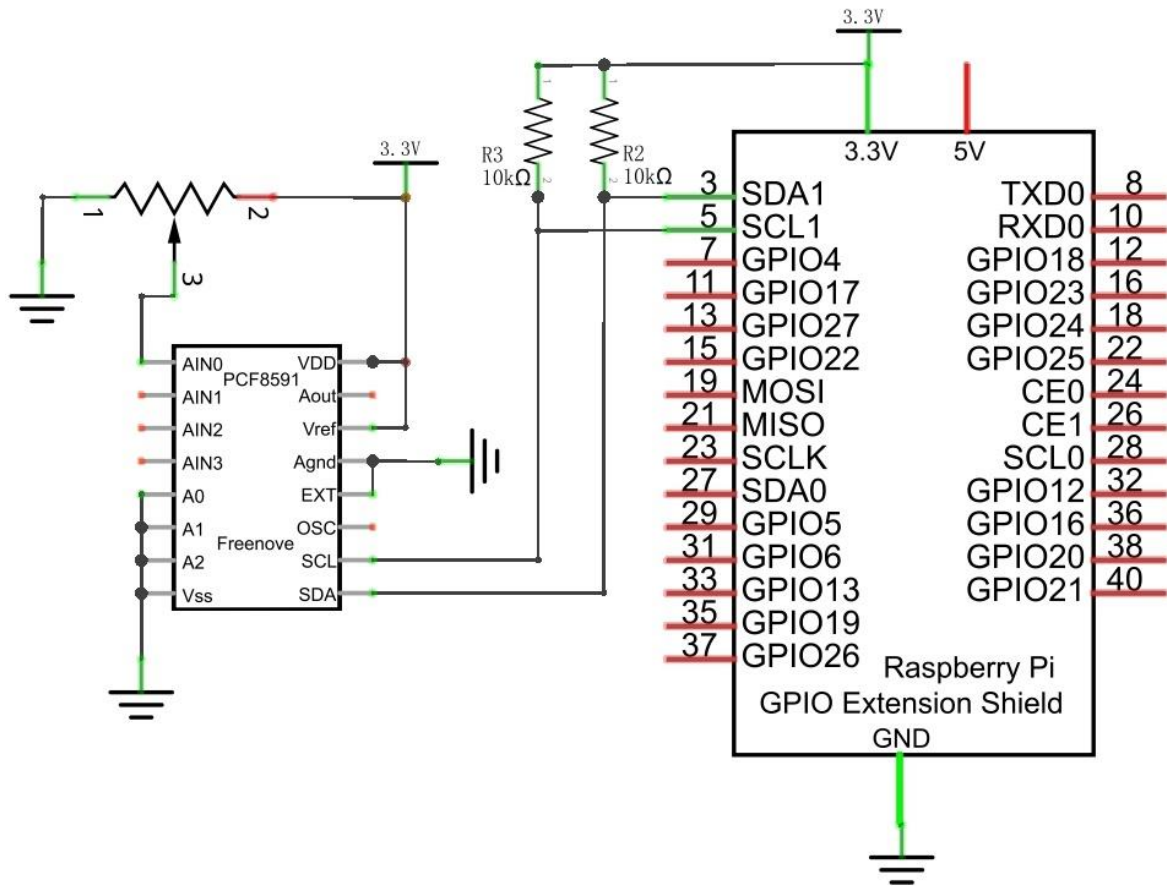
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com
 This product contains **only one ADC module**.



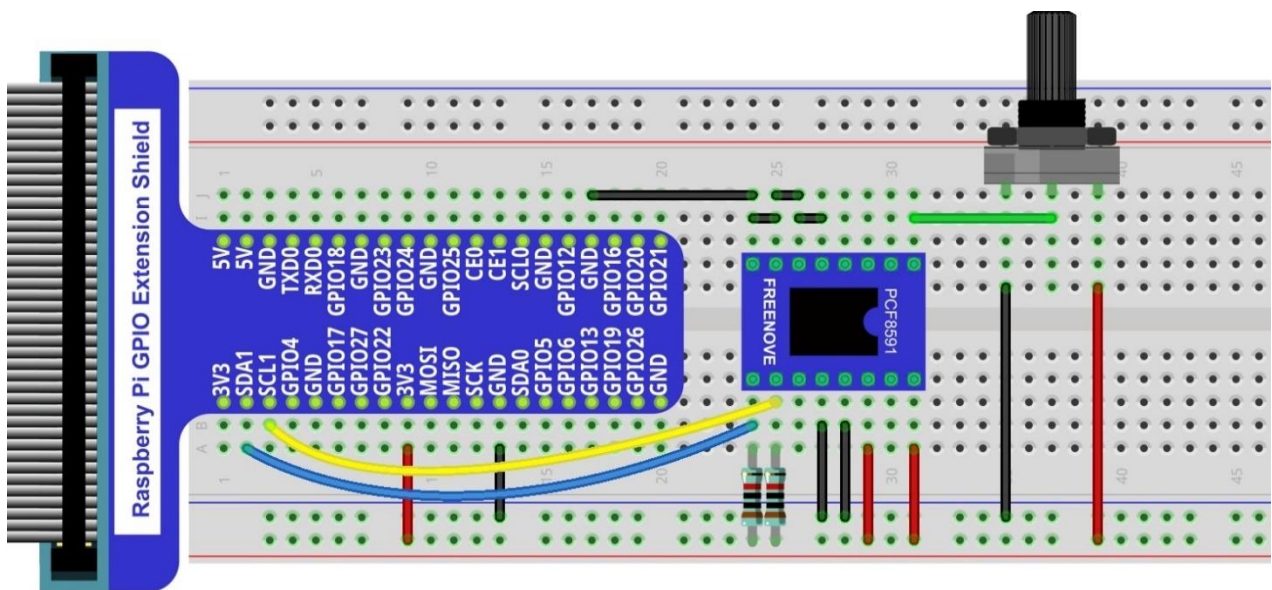
Video: https://youtu.be/PSUCctu_DqA

Circuit with PCF8591

Schematic diagram



Hardware connection



Please keep the **chip mark** consistent to make the chips under right direction and position.

Configure I2C and Install Smbus

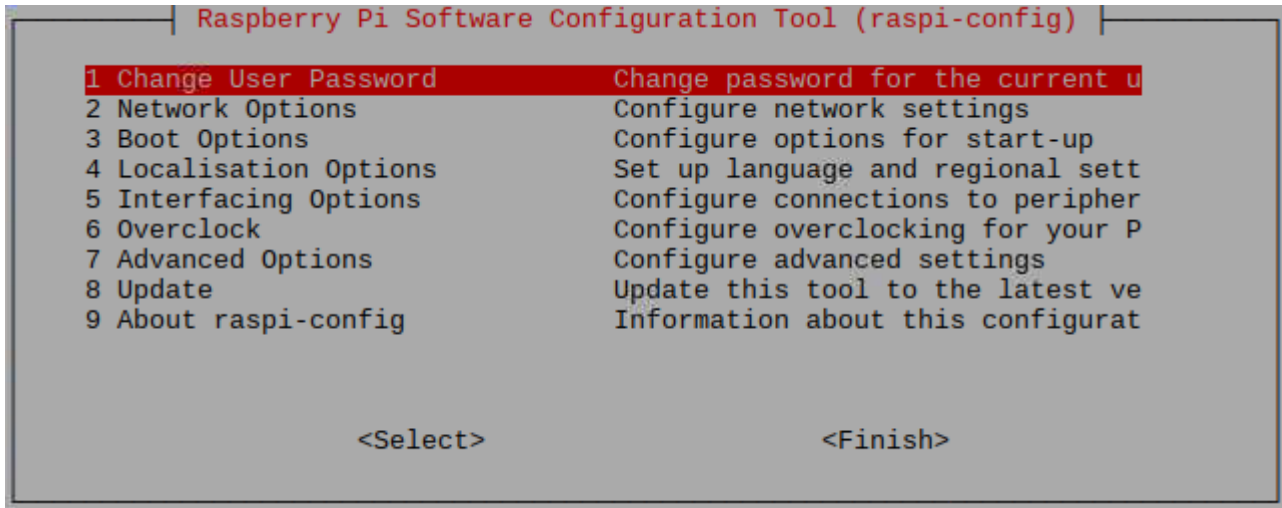
Enable I2C

The I2C interface in Raspberry Pi is disabled by default. You will need to open it manually and enable the I2C interface as follows:

Type command in the Terminal:

```
sudo raspi-config
```

Then open the following dialog box:



Choose “5 Interfacing Options” then “P5 I2C” then “Yes” and then “Finish” in this order and restart your RPi. The I2C module will then be started.

Type a command to check whether the I2C module is started:

```
lsmod | grep i2c
```

If the I2C module has been started, the following content will be shown. “bcm2708” refers to the CPU model. Different models of Raspberry Pi display different contents depending on the CPU installed:

```
pi@raspberrypi:~$ lsmod | grep i2c
i2c_bcm2708      4770  0
i2c_dev         5859  0
pi@raspberrypi:~$
```


Install I2C-Tools

Next, type the command to install I2C-Tools. It is available with the Raspberry Pi OS by default.

```
sudo apt-get install i2c-tools
```

I2C device address detection:

```
i2cdetect -y 1
```

When you are using the PCF8591 Module, the result should look like this:

```
pi@raspberrypi:~ $ i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
10:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
20:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
40:  --  --  --  --  --  --  --  48  --  --  --  --  --  --  --
50:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
60:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
70:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
```

Here, 48 (HEX) is the I2C address of ADC Module (PCF8591).

When you are using ADS, the result should look like this:

```
pi@raspberrypi:~ $ i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
10:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
20:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
40:  --  --  --  --  --  --  --  4b  --  --  --  --  --  --  --
50:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
60:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
70:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
```

Here, 4b (HEX) is the I2C address of ADC Module (ADS7830).

Install Smbus Module

```
sudo apt-get install python-smbus
sudo apt-get install python3-smbus
```

Code

Python Code 7.1.1 ADC

For Python code, ADCDevice requires a custom module which needs to be installed.

1. Use cd command to enter folder of ADCDevice.

```
cd ~/Freenove_Kit/Libs/Python-Libs/
```

2. Unzip the file.

```
tar -zxvf ADCDevice-1.0.3.tar.gz
```

3. Open the unzipped folder.

```
cd ADCDevice-1.0.3
```

4. Install library for python3 and python2.

```
sudo python3 setup.py install
sudo python2 setup.py install
```

A successful installation, without error prompts, is shown below:

```
Installed /usr/local/lib/python3.7/dist-packages/ADCDevice-1.0.2-py3.7.egg
Processing dependencies for ADCDevice==1.0.2
Finished processing dependencies for ADCDevice==1.0.2
```

Execute the following command. Observe the project result and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 07.1.1_ADC directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless/07.1.1_ADC
```

2. Use the Python command to execute the Python code "ADC.py".

```
python ADC.py
```

After the program is executed, adjusting the potentiometer will produce a readout display of the potentiometer voltage values in the Terminal and the converted digital content.

```
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
ADC Value : 169, Voltage : 2.19
ADC Value : 168, Voltage : 2.17
ADC Value : 168, Voltage : 2.17
```

The following is the code:

```
1  import time
2  from ADCDevice import *
3
4  adc = ADCDevice() # Define an ADCDevice class object
5
6  def setup():
7      global adc
8      if(adc.detectI2C(0x48)): # Detect the pcf8591.
9          adc = PCF8591()
10         elif(adc.detectI2C(0x4b)): # Detect the ads7830
11             adc = ADS7830()
12         else:
13             print("No correct I2C address found, \n"
14                 "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
15                 "Program Exit. \n");
```

```

16         exit(-1)
17
18     def loop():
19         while True:
20             value = adc.analogRead(0)    # read the ADC value of channel 0
21             voltage = value / 255.0 * 3.3 # calculate the voltage value
22             print ('ADC Value : %d, Voltage : %.2f'%(value, voltage))
23             time.sleep(0.1)
24
25     def destroy():
26         adc.close()
27
28     if __name__ == '__main__': # Program entrance
29         print ('Program is starting ... ')
30         try:
31             setup()
32             loop()
33         except KeyboardInterrupt: # Press ctrl-c to end the program.
34             destroy()

```

In this code, a custom Python module "ADCDevice" is used. It contains the method of utilizing the ADC Module in this project, through which the ADC Module can easily and quickly be used. In the code, you need to first create an ADCDevice object adc.

```
adc = ADCDevice() # Define an ADCDevice class object
```

Then in setup(), use detectI2C(addr), the member function of ADCDevice, to detect the I2C module in the circuit. Different modules have different I2C addresses. Therefore, according to the address, we can determine which ADC Module is in the circuit. When the correct module is detected, a device specific class object is created and assigned to adc. The default address of PCF8591 is 0x48, and that of ADS7830 is 0x4b.

```

def setup():
    global adc
    if(adc.detectI2C(0x48)): # Detect the pcf8591.
        adc = PCF8591()
    elif(adc.detectI2C(0x4b)): # Detect the ads7830
        adc = ADS7830()
    else:
        print("No correct I2C address found, \n"
              "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
              "Program Exit. \n");
        exit(-1)

```

When you have a class object of a specific device, you can get the ADC value of the specified channel by calling the member function of this class, analogRead(ch). In loop(), get the ADC value of potentiometer.

```
value = adc.analogRead(0)    # read the ADC value of channel 0
```

Then according to the formula, the voltage value is calculated and displayed on the terminal monitor.

```
voltage = value / 255.0 * 3.3 # calculate the voltage value
print ('ADC Value : %d, Voltage : %.2f'%(value, voltage))
time.sleep(0.1)
```

Reference

About smbus Module:

smbus Module

The System Management Bus Module defines an object type that allows SMBus transactions on hosts running the Linux kernel. The host kernel must support I2C, I2C device interface support, and a bus adapter driver. All of these can be either built-in to the kernel, or loaded from modules.

In Python, you can use `help(smbus)` to view the relevant functions and their descriptions.

bus=smbus.SMBus(1): Create an SMBus class object.

bus.read_byte_data(address,cmd+chn): Read a byte of data from an address and return it.

bus.write_byte_data(address,cmd,value): Write a byte of data to an address.

class ADCDevice(object)

This is a base class.

```
int detectI2C(int addr);
```

This is a member function, which is used to detect whether the device with the given I2C address exists. If it exists, it returns true. Otherwise, it returns false.

```
class PCF8591(ADCDevice)
```

```
class ADS7830(ADCDevice)
```

These two classes are derived from the ADCDevice and the main function is `analogRead(chn)`.

```
int analogRead(int chn);
```

This returns the value read on the supplied analog input pin.

Parameter chn: For PCF8591, the range of chn is 0, 1, 2, 3. For ADS7830, the range is 0, 1, 2, 3, 4, 5, 6, 7.

You can find the source file of this library in the folder below:

~/Freenove_Kit/Libs/Python-Libs/ADCDevice-1.0.3/src/ADCDevice/ADCdevice.py

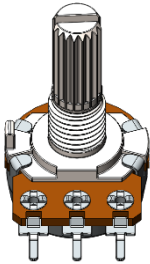
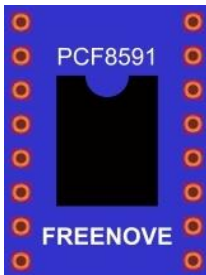
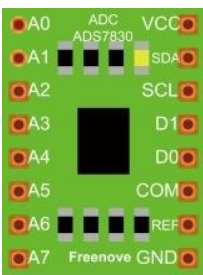
Chapter 8 Potentiometer & LED

Earlier we learned how to use ADC and PWM. In this chapter, we learn to control the brightness of an LED by using a potentiometer.

Project 8.1 Soft Light

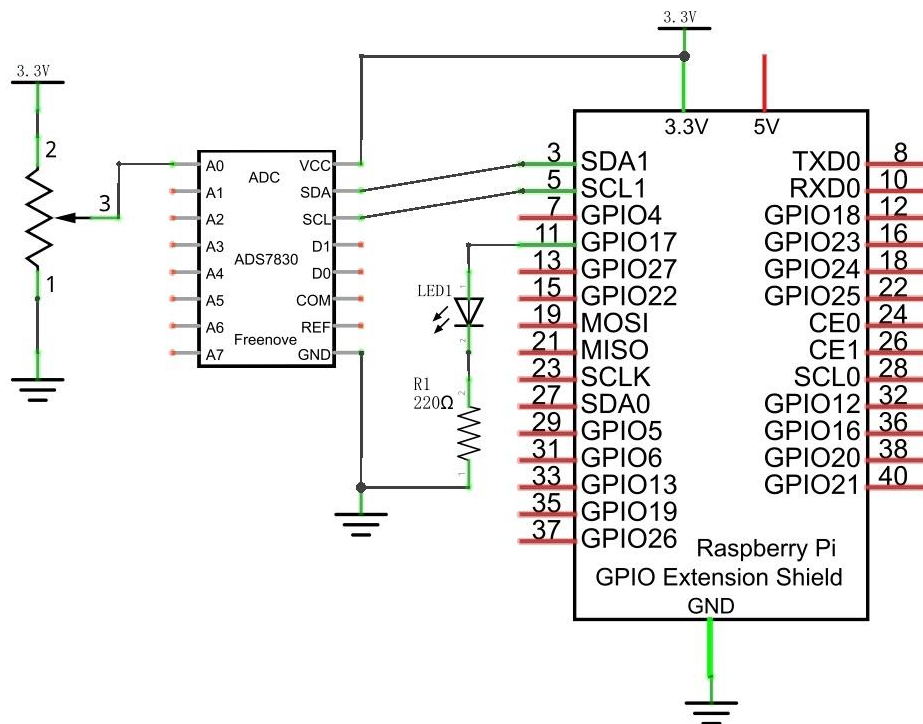
In this project, we will make a soft light. We will use an ADC Module to read ADC values of a potentiometer and map it to duty cycle ratio of the PWM used to control the brightness of an LED. Then you can change the brightness of an LED by adjusting the potentiometer.

Component List

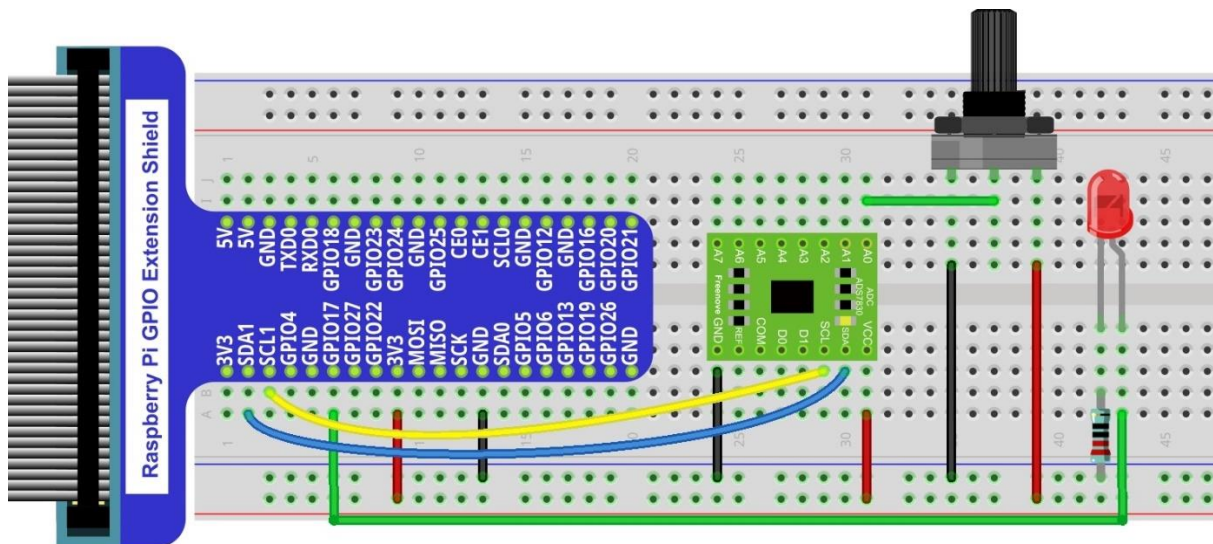
Raspberry Pi x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1		Jumper Wire M/M x17 			
Rotary Potentiometer x1 	ADC Module x1 (Only one)  or 		10kΩ x2 	220Ω x1 	LED x1 

Circuit with ADS7830

Schematic diagram



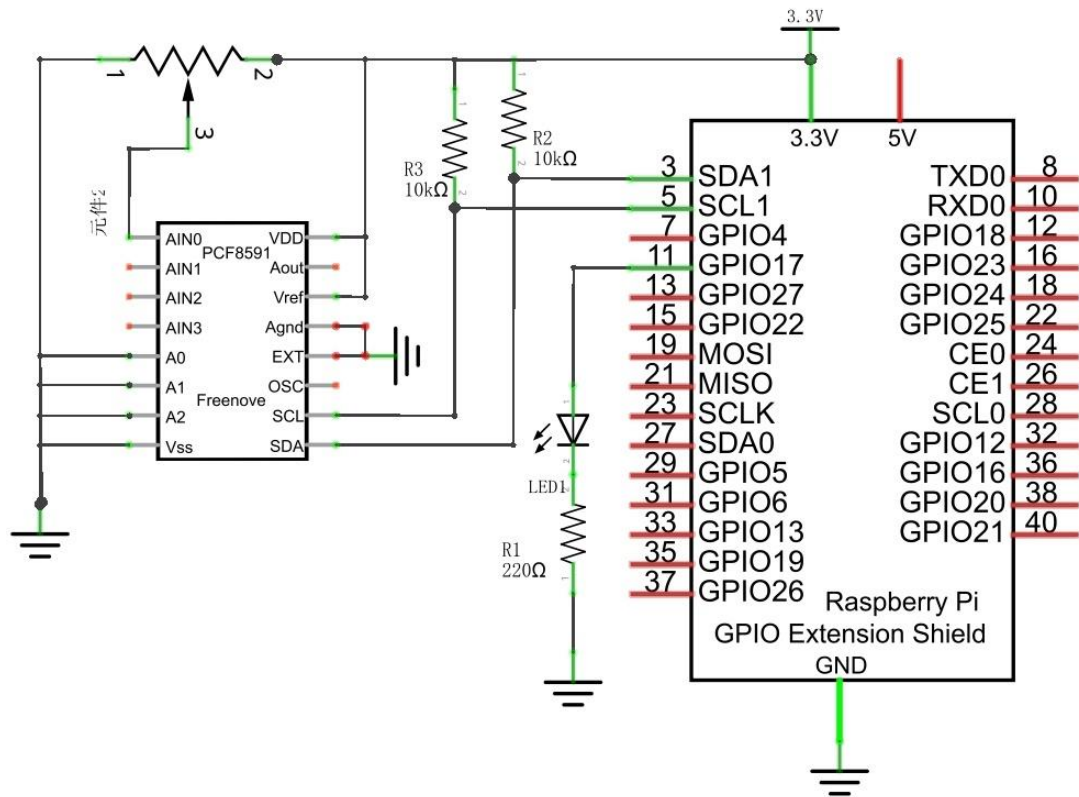
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



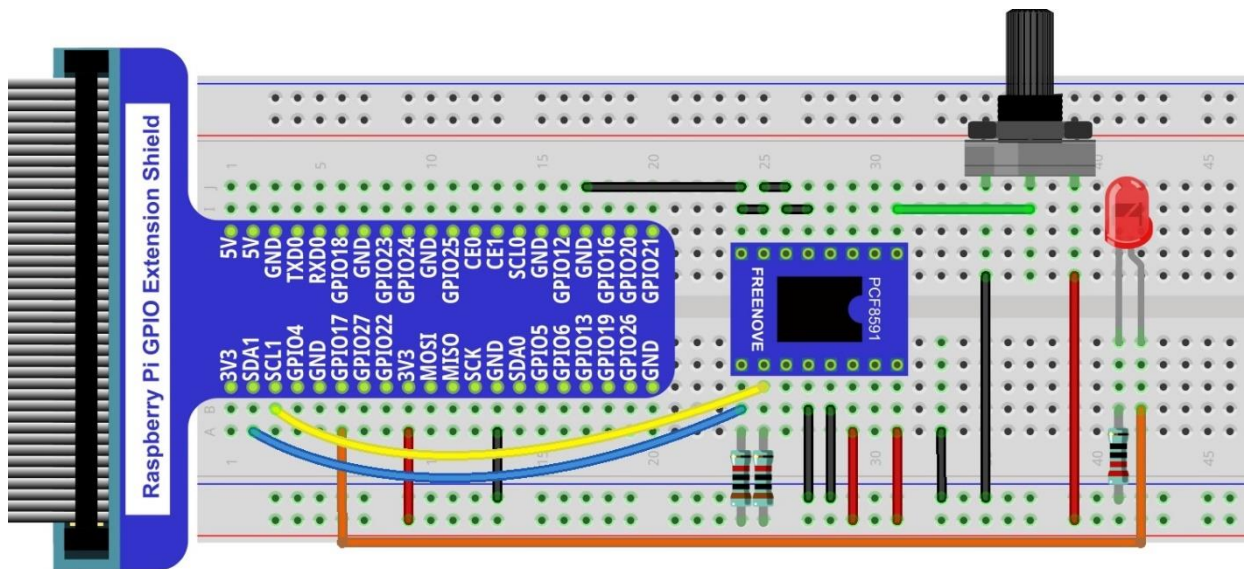
Video: <https://youtu.be/YMEfe9IWU6I>

Circuit with PCF8591

Schematic diagram



Hardware connection



Code

Python Code 8.1.1 Softlight

If you did not [configure I2C](#), please refer to [Chapter 7](#). If you did, please continue. First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 08.1.1_Softlight directory of Python code

```
cd ~/Freenove_Kit/Code/Python_GPIOWZero_Code/08.1.1_Softlight
```

2. Use the python command to execute the Python code "Softlight.py".

```
python Softlight.py
```

After the program is executed, adjusting the potentiometer will display the voltage values of the potentiometer in the Terminal window and the converted digital quantity. As a consequence, the brightness of LED will be changed.

The following is the code:

```

1  from gpiozero import PWMLED
2  import time
3  from ADCDevice import *
4
5  led = PWMLED(17, frequency=1000)      # define LED pin according to BCM Numbering
6  adc = ADCDevice() # Define an ADCDevice class object
7
8  def setup():
9      global adc
10     if(adc.detectI2C(0x48)): # Detect the pcf8591.
11         adc = PCF8591()
12     elif(adc.detectI2C(0x4b)): # Detect the ads7830
13         adc = ADS7830()
14     else:
15         print("No correct I2C address found, \n"
16             "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
17             "Program Exit. \n");
18         exit(-1)
19
20 def loop():
21     while True:
22         value = adc.analogRead(0)      # read the ADC value of channel 0
23         led.value = value / 255.0      # Mapping to PWM duty cycle
24         voltage = value / 255.0 * 3.3  # calculate the voltage value
25         print (' ADC Value : %d, Voltage : %.2f'%(value,voltage))
26         time.sleep(0.03)

```

```
27
28 def destroy():
29     led.close()
30     adc.close()
31
32 if __name__ == '__main__': # Program entrance
33     print ('Program is starting ... ')
34     try:
35         setup()
36         loop()
37     except KeyboardInterrupt: # Press ctrl-c to end the program.
38         destroy()
39     print("Ending program")
```

In the code, read ADC value of potentiometers and map it to the duty cycle of the PWM to control LED brightness.

```
value = adc.analogRead(0) # read the ADC value of channel 0
led.value = value / 255.0 # Mapping to PWM duty cycle
```

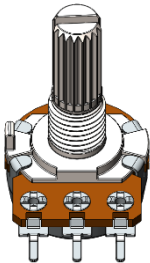
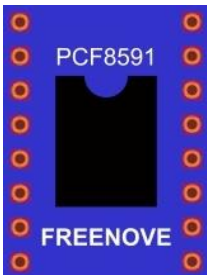
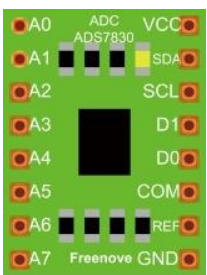
Chapter 9 Potentiometer & RGBLED

In this chapter, we will use 3 potentiometers to control the brightness of 3 LEDs of RGBLED to create multiple colors.

Project 9.1 Colorful Light

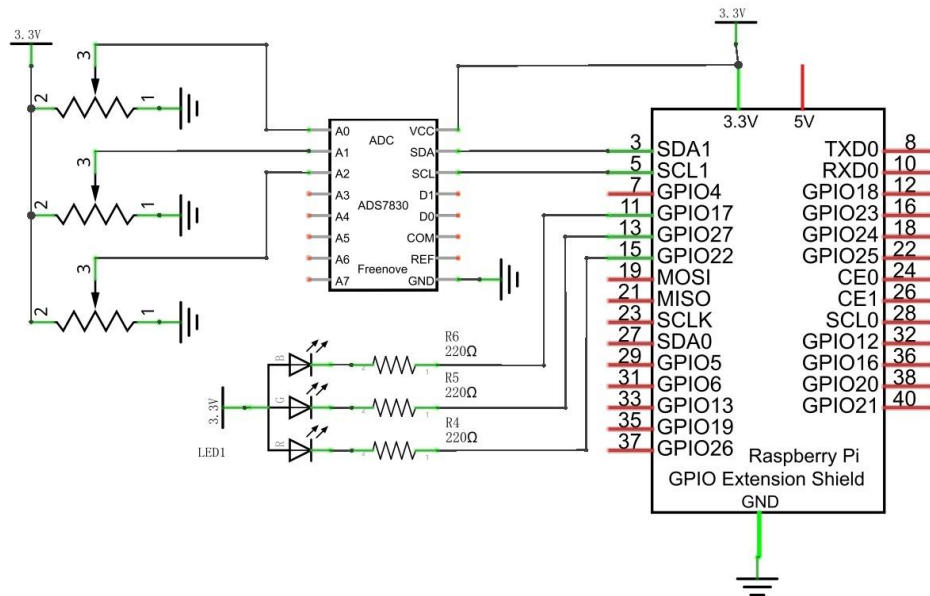
In this project, 3 potentiometers are used to control the RGB LED and in principle it is the same as with the Soft Light. project. Namely, read the voltage value of the potentiometer and then convert it to PWM used to control LED brightness. Difference is that the previous soft light project needed only one LED while this one required (1) RGB LEDs.

Component List

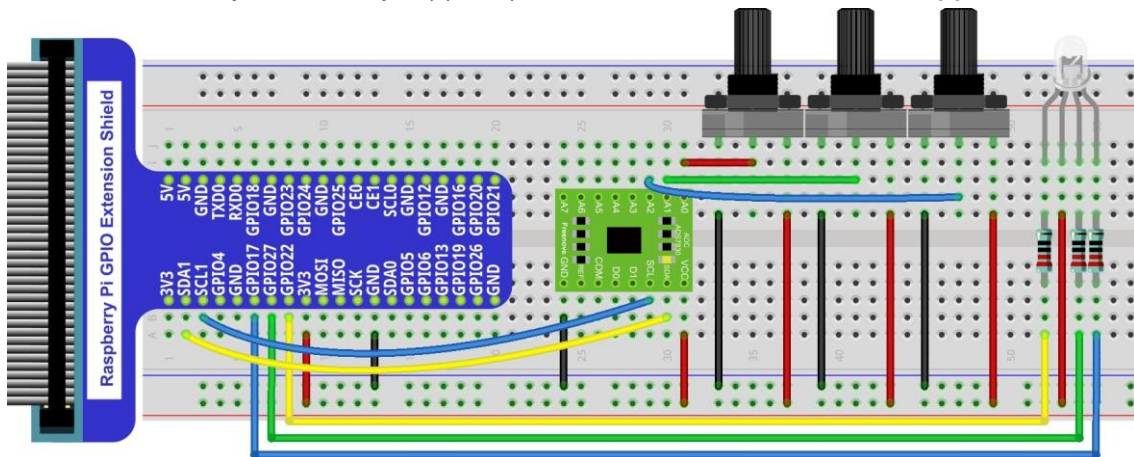
Raspberry Pi x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1		Jumper Wires M/M x17 			
Rotary potentiometer x3 	ADC module x1  or 	10kΩ x2 	220Ω x3 	RGB LEDx1 	

Circuit with ADS7830

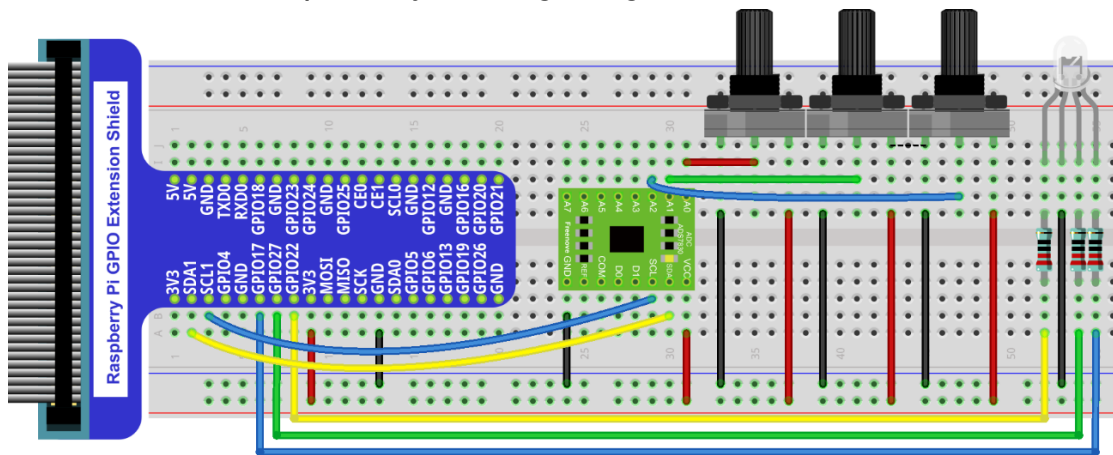
Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

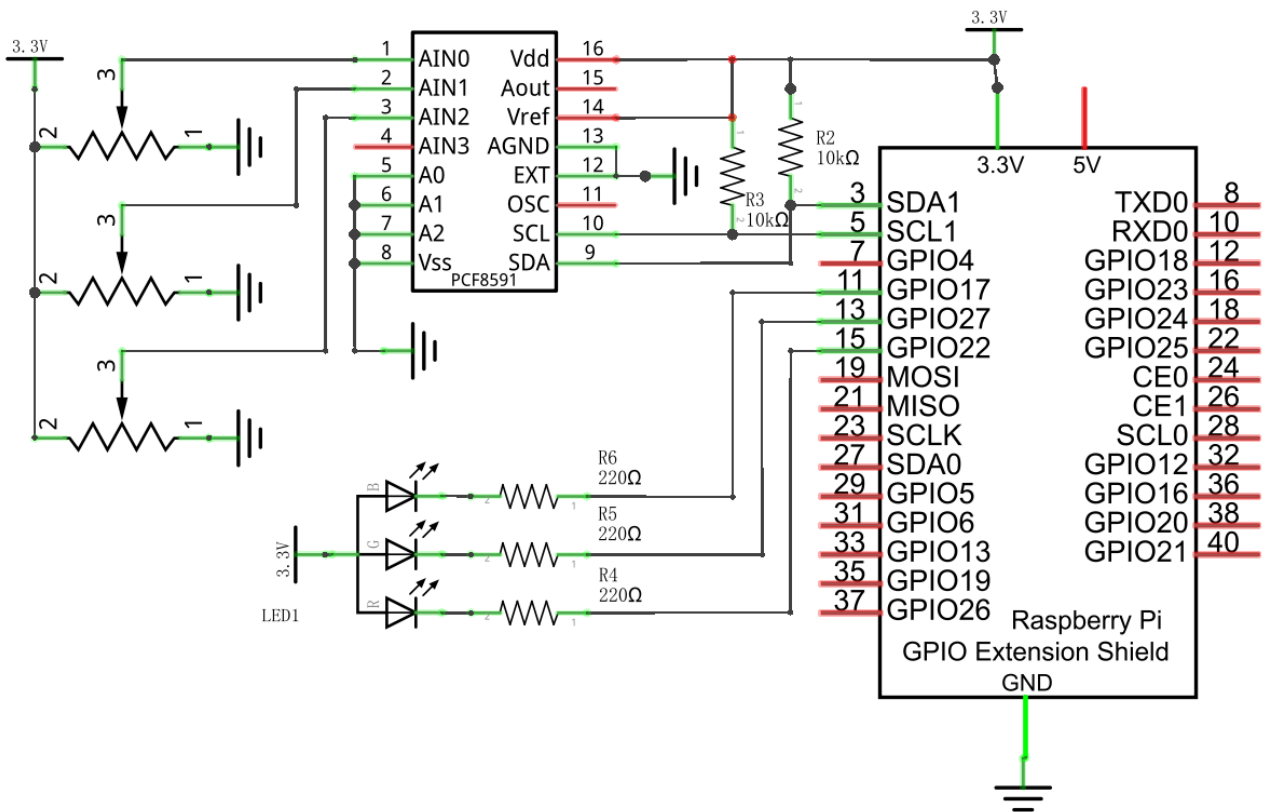


If circuit above doesn't work, please try following wiring.

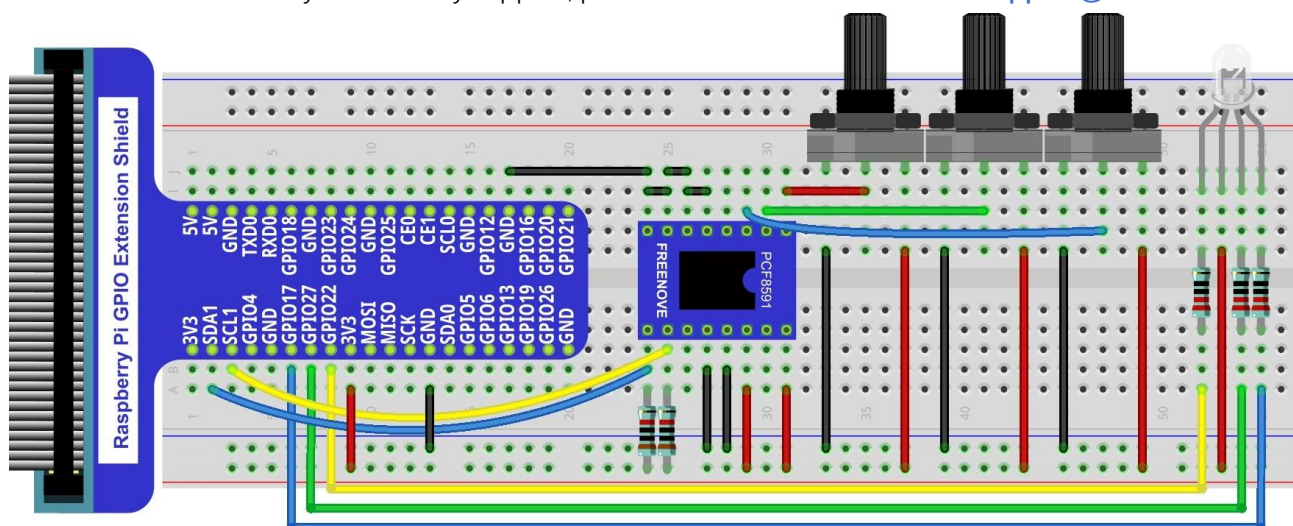


Circuit with PCF8591

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: <https://www.youtube.com/watch?v=CmLLNsBMN2U>

Code

Python Code 9.1.1 ColorfulSoftlight

If you did not [configure I2C](#), please refer to [Chapter 7](#). If you did, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 09.1.1_ColorfulSoftlight directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless/09.1.1_ColorfulSoftlight
```

2. Use python command to execute python code "ColorfulSoftlight.py".

```
python ColorfulSoftlight.py
```

After the program is executed, rotate one of the potentiometers, then the color of RGB LED will change. The Terminal window will display the ADC value of each potentiometer.

The following is the program code:

```

1  from gpiozero import RGBLED
2  import time
3  from ADCDevice import *
4
5  led = RGBLED(red=22, green=27, blue=17, active_high=False) # define the pins for
   R:GPIO22,G:GPIO27,B:GPIO17
6  #led = RGBLED(red="J8:15", green="J8:13", blue="J8:11") # according to BOARD Numbering define
   the pins for R:11,G:12,B:13
7  adc = ADCDevice() # Define an ADCDevice class object
8
9  def setup():
10     global adc
11     if(adc.detectI2C(0x48)): # Detect the pcf8591.
12         adc = PCF8591()
13     elif(adc.detectI2C(0x4b)): # Detect the ads7830
14         adc = ADS7830()
15     else:
16         print("No correct I2C address found, \n"
17             "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
18             "Program Exit. \n");
19         exit(-1)
20
21  def loop():
22     while True:
23         value_Red = adc.analogRead(0)    # read ADC value of 3 potentiometers
24         value_Green = adc.analogRead(1)
25         value_Blue = adc.analogRead(2)
26

```

```
27         led.red    =value_Red/255 # map the read value of potentiometers into PWM value and
output it
28         led.green  =value_Green/255
29         led.blue   =value_Blue/255
30         # print read ADC value
31         print ('ADC Value
value_Red: %d , \tvblue_Green: %d , \tvalue_Blue: %d'%(value_Red,value_Green,value_Blue))
32         time.sleep(0.01)
33
34     def destroy():
35         adc.close()
36         led.close()
37
38     if __name__ == '__main__': # Program entrance
39         print ('Program is starting ... ')
40         setup()
41         try:
42             loop()
43         except KeyboardInterrupt: # Press ctrl-c to end the program.
44             destroy()
45         print("Ending program")
```

In the code you can read the ADC values of the 3 potentiometers and map it into a PWM duty cycle to control the 3 LED elements to vary the color of their respective RGB LED.

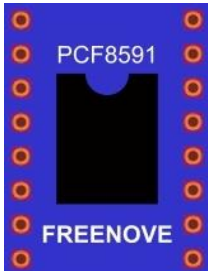
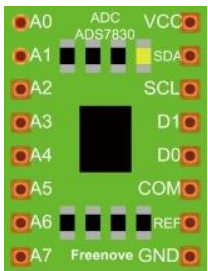
Chapter 10 Photoresistor & LED

In this chapter, we will learn how to use a photoresistor to make an automatic dimming nightlight.

Project 10.1 NightLamp

A Photoresistor is very sensitive to the amount of light present. We can take advantage of the characteristic to make a nightlight with the following function. When the ambient light is less (darker environment), the LED will automatically become brighter to compensate and when the ambient light is greater (brighter environment) the LED will automatically dim to compensate.

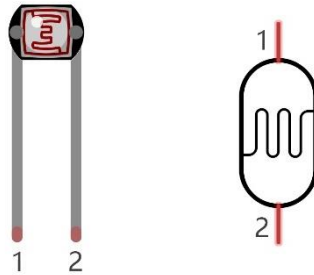
Component List

Raspberry Pi x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1		Jumper Wires M/M x15 			
Photoresistor x1 	ADC module x1  or 		10kΩ x3 	220Ω x1 	LED x1 

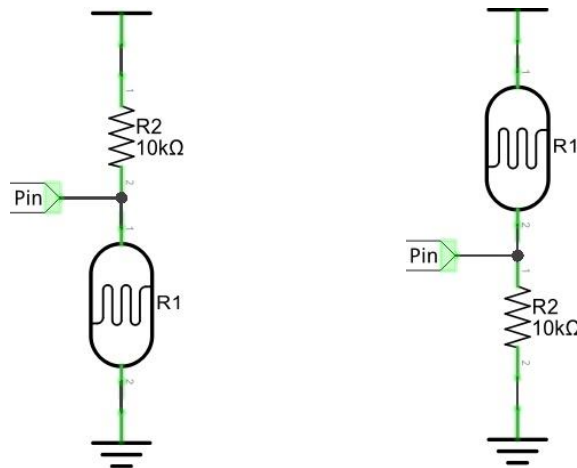
Component knowledge

Photoresistor

A Photoresistor is simply a light sensitive resistor. It is an active component that decreases resistance with respect to receiving luminosity (light) on the component's light sensitive surface. A Photoresistor's resistance value will change in proportion to the ambient light detected. With this characteristic, we can use a Photoresistor to detect light intensity. The Photoresistor and its electronic symbol are as follows.



The circuit below is used to detect the change of a Photoresistor's resistance value:

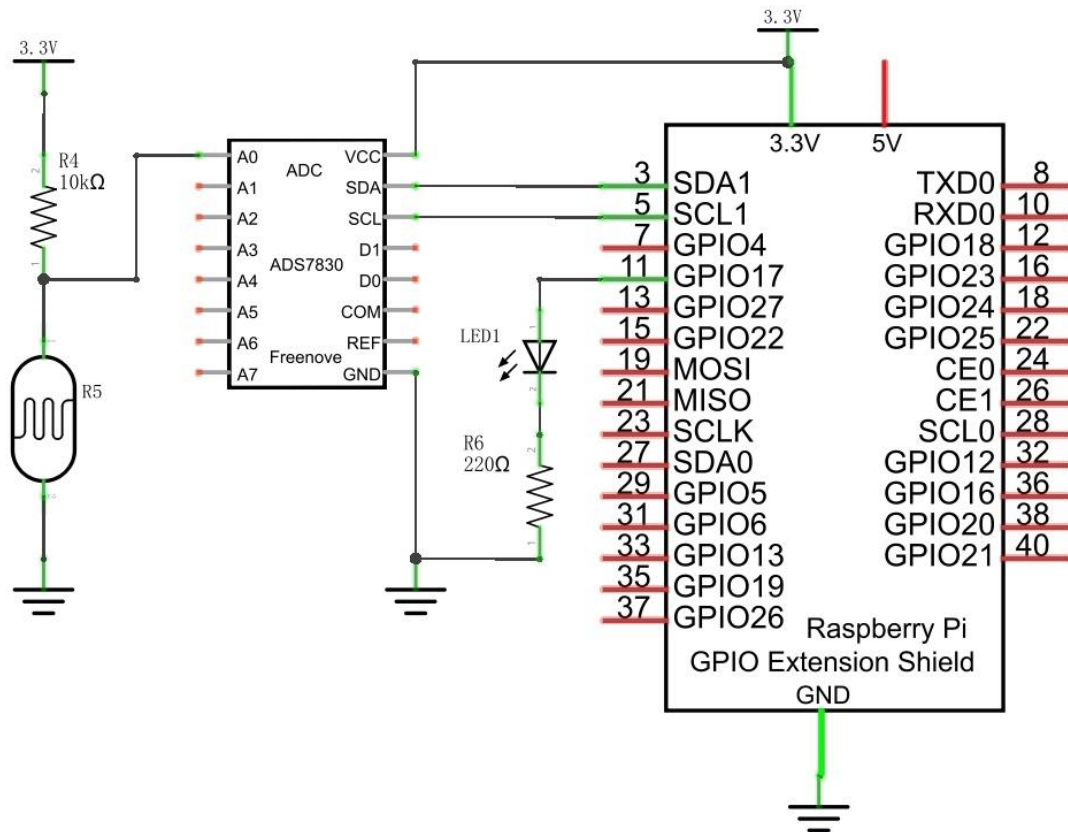


In the above circuit, when a Photoresistor's resistance value changes due to a change in light intensity, the voltage between the Photoresistor and Resistor R1 will also change. Therefore, the intensity of the light can be obtained by measuring this voltage.

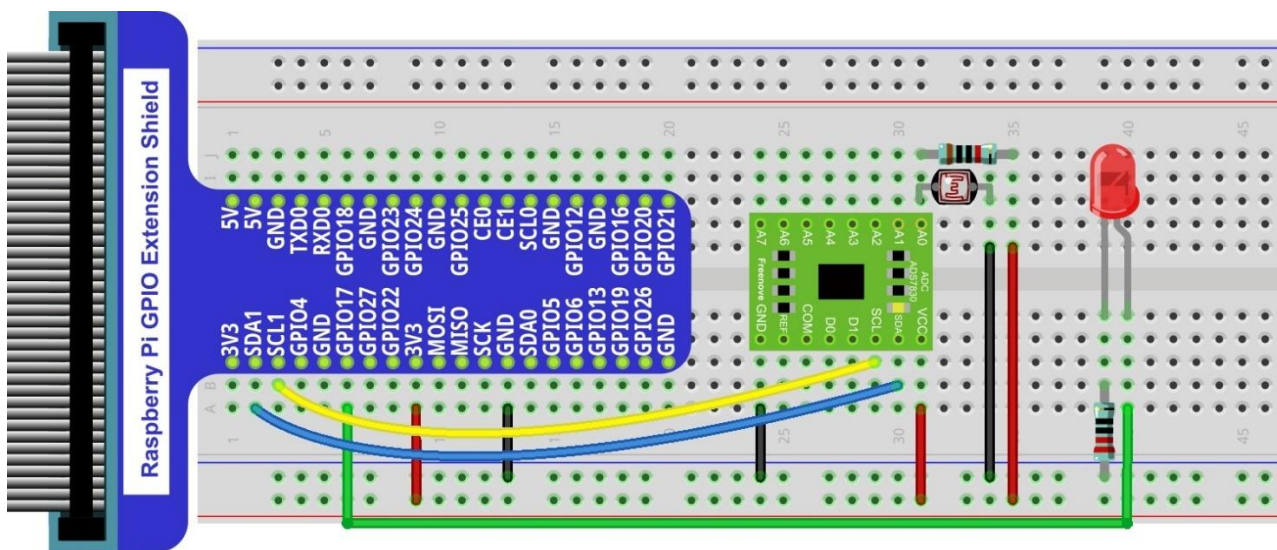
Circuit with ADS7830

The circuit used is similar to the Soft light project. The only difference is that the input signal of the AIN0 pin of ADC changes from a Potentiometer to a combination of a Photoresistor and a Resistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

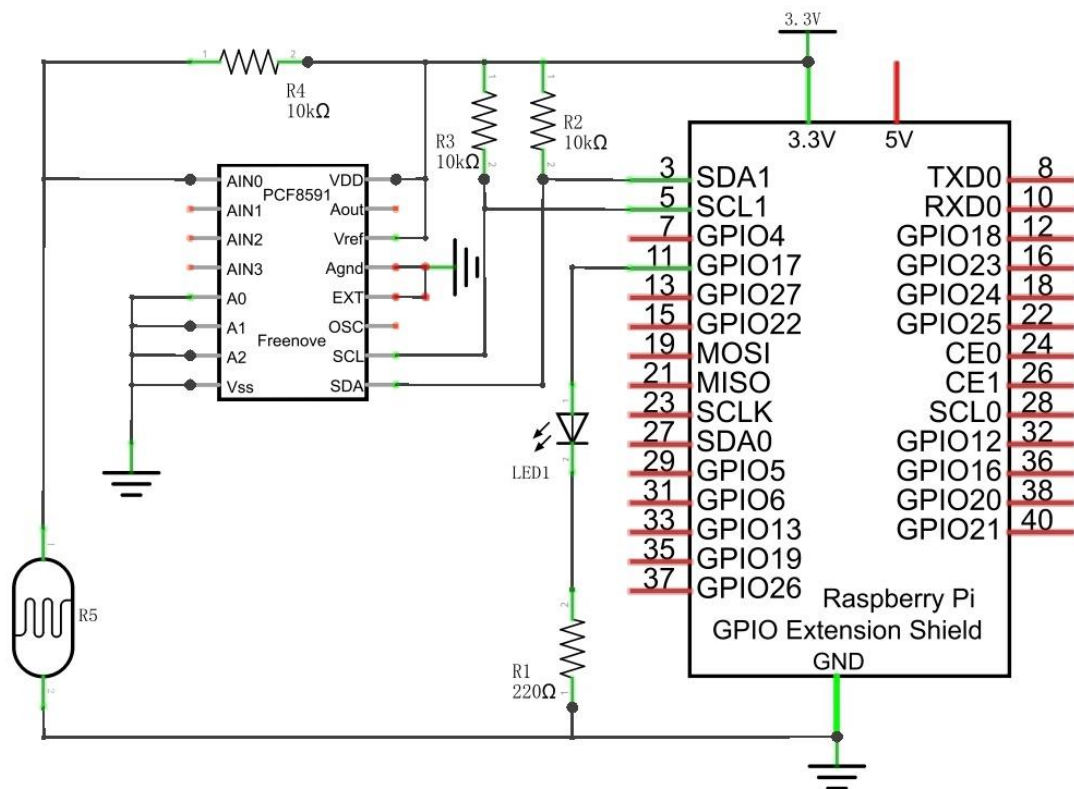


Video: <https://youtu.be/r6p3zhXsyko>

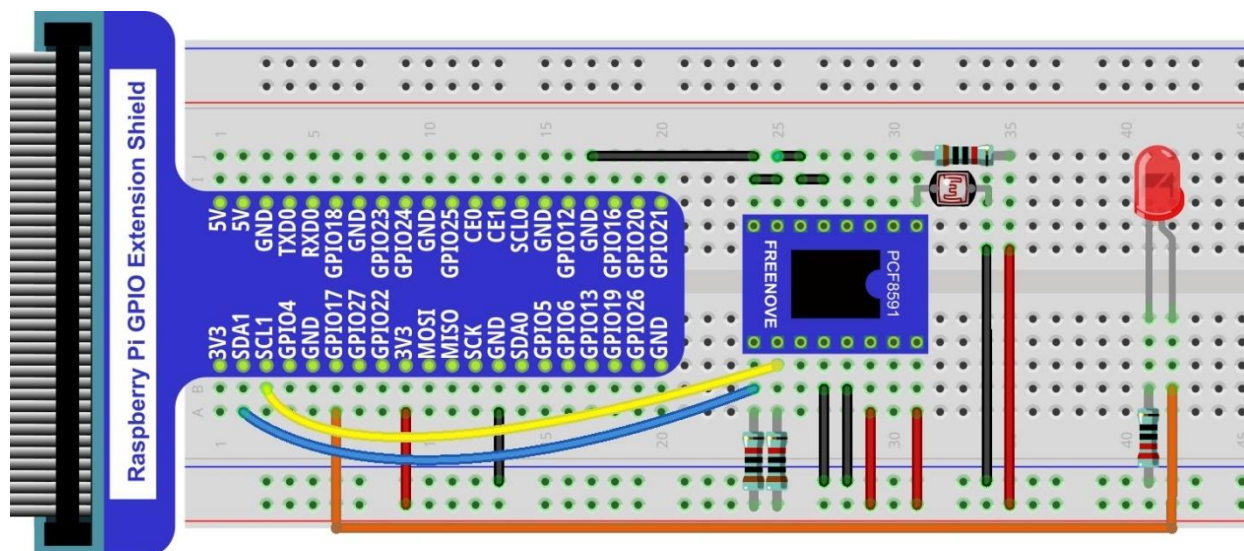
Circuit with PCF8591

The circuit used is similar to the Soft light project. The only difference is that the input signal of the AIN0 pin of ADC changes from a Potentiometer to a combination of a Photoresistor and a Resistor.

Schematic diagram



Hardware connection



Code

The code used in this project is identical with what was used in the last chapter.

Python Code 10.1.1 Nightlamp

If you did not [configure I2C](#), please refer to [Chapter 7](#). If you did, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 10.1_Nightlamp directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless/10.1.1_Nightlamp
```

2. Use the python command to execute the Python code "Nightlamp.py".

```
python Nightlamp.py
```

After the program is executed, if you cover the Photoresistor or increase the light shining on it, the brightness of the LED changes accordingly. As in previous projects the Terminal window will display the current input voltage value of ADC module A0 pin and the converted digital quantity.

The following is the program code:

```

1  from gpiozero import PWMLED
2  import time
3  from ADCDevice import *
4
5  ledPin = 17 # define ledPin
6  led = PWMLED(ledPin)
7  adc = ADCDevice() # Define an ADCDevice class object
8
9  def setup():
10     global adc
11     if(adc.detectI2C(0x48)): # Detect the pcf8591.
12         adc = PCF8591()
13     elif(adc.detectI2C(0x4b)): # Detect the ads7830
14         adc = ADS7830()
15     else:
16         print("No correct I2C address found, \n"
17             "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
18             "Program Exit. \n");
19         exit(-1)
20
21  def loop():
22     while True:
23         value = adc.analogRead(0)    # read the ADC value of channel 0
24         led.value = value / 255.0    # Mapping to PWM duty cycle
25         voltage = value / 255.0 * 3.3
26         print ('ADC Value : %d, Voltage : %.2f'%(value,voltage))
27         time.sleep(0.01)

```

```
28
29 def destroy():
30     led.close()
31     adc.close()
32
33 if __name__ == '__main__': # Program entrance
34     print ('Program is starting ... ')
35     setup()
36     try:
37         loop()
38     except KeyboardInterrupt: # Press ctrl-c to end the program.
39         destroy()
40     print("Ending program")
```

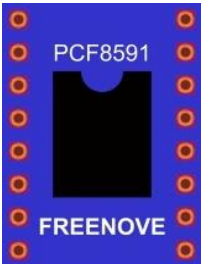
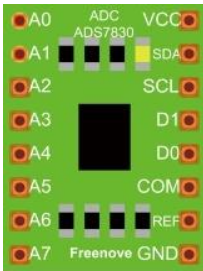
Chapter 11 Thermistor

In this chapter, we will learn about Thermistors which are another kind of Resistor.

Project 11.1 Thermometer

A Thermistor is a type of Resistor whose resistance value is dependent on temperature and changes in temperature. Therefore, we can take advantage of this characteristic to make a Thermometer.

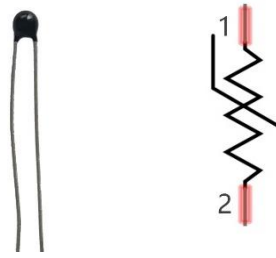
Component List

Raspberry Pi x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1		Jumper Wire M/M x14 
Thermistor x1 	ADC module x1  or 	
		Resistor 10kΩ x3 

Component knowledge

Thermistor

Thermistor is a temperature sensitive resistor. When it senses a change in temperature, the resistance of the Thermistor will change. We can take advantage of this characteristic by using a Thermistor to detect temperature intensity. A Thermistor and its electronic symbol are shown below.



The relationship between resistance value and temperature of a thermistor is:

$$R_t = R \cdot \text{EXP} [B \cdot (1/T_2 - 1/T_1)]$$

Where:

R_t is the thermistor resistance under T₂ temperature;

R is in the nominal resistance of thermistor under T₁ temperature;

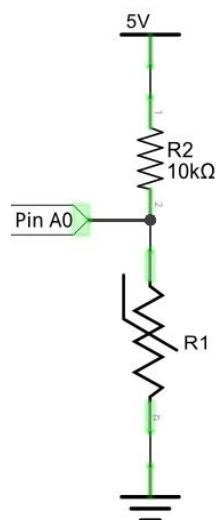
EXP[n] is nth power of e;

B is for thermal index;

T₁, T₂ is Kelvin temperature (absolute temperature). Kelvin temperature = 273.15 + Celsius temperature.

For the parameters of the Thermistor, we use: B=3950, R=10k, T₁=25.

The circuit connection method of the Thermistor is similar to photoresistor, as the following:



We can use the value measured by the ADC converter to obtain the resistance value of Thermistor, and then we can use the formula to obtain the temperature value.

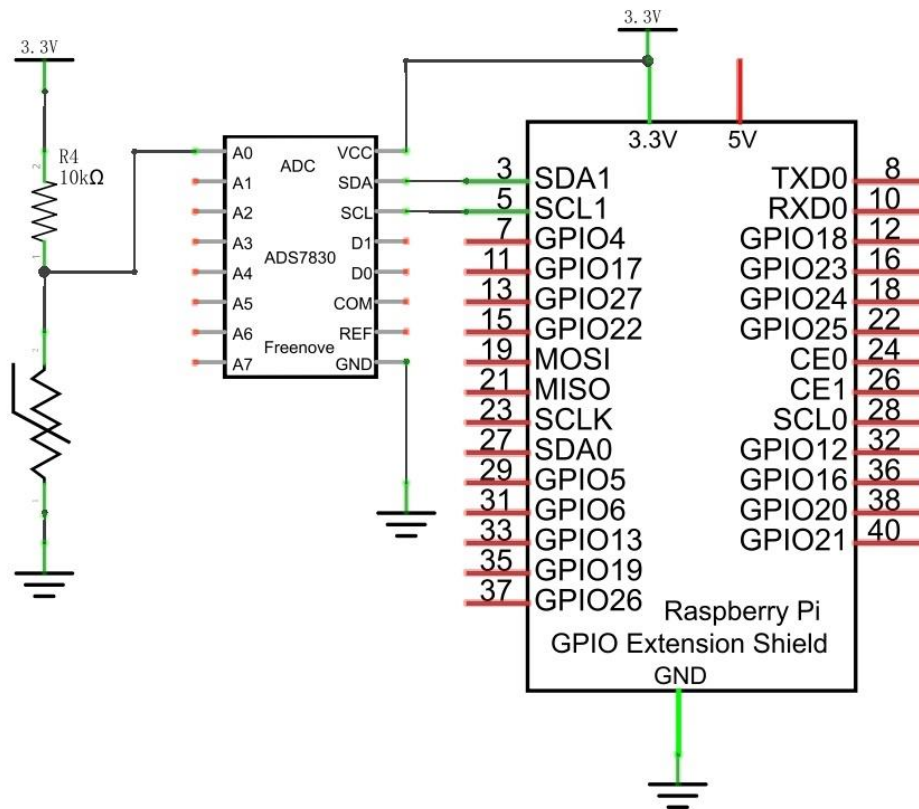
Therefore, the temperature formula can be derived as:

$$T_2 = 1 / (1/T_1 + \ln(R_t/R)/B)$$

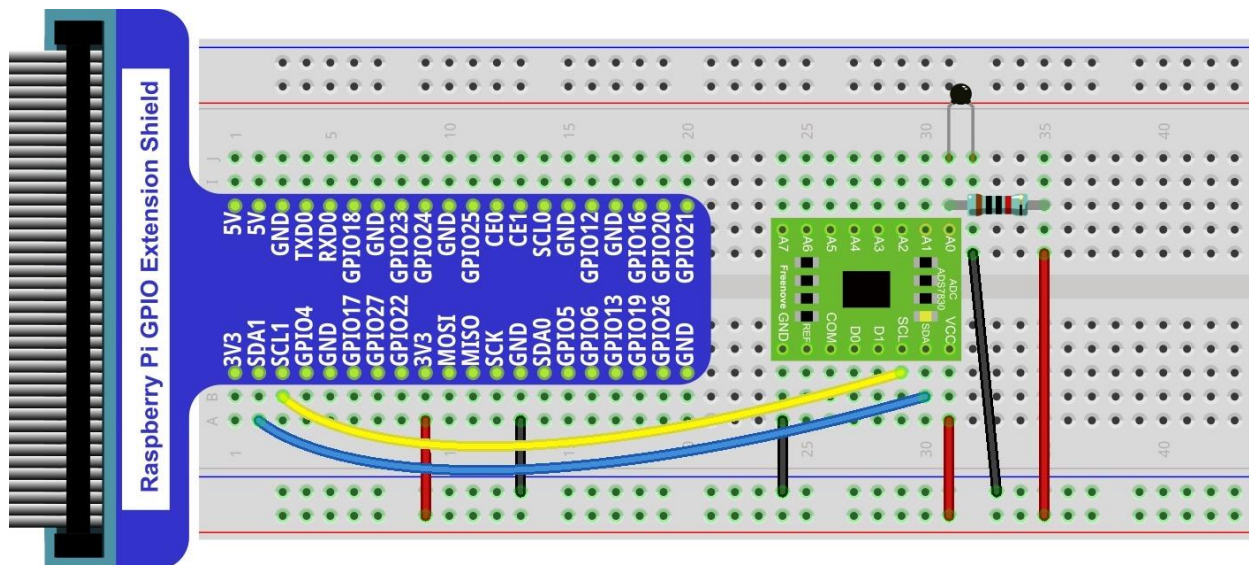
Circuit with ADS7830

The circuit of this project is similar to the one in last chapter. The only difference is that the Photoresistor is replaced by the Thermistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



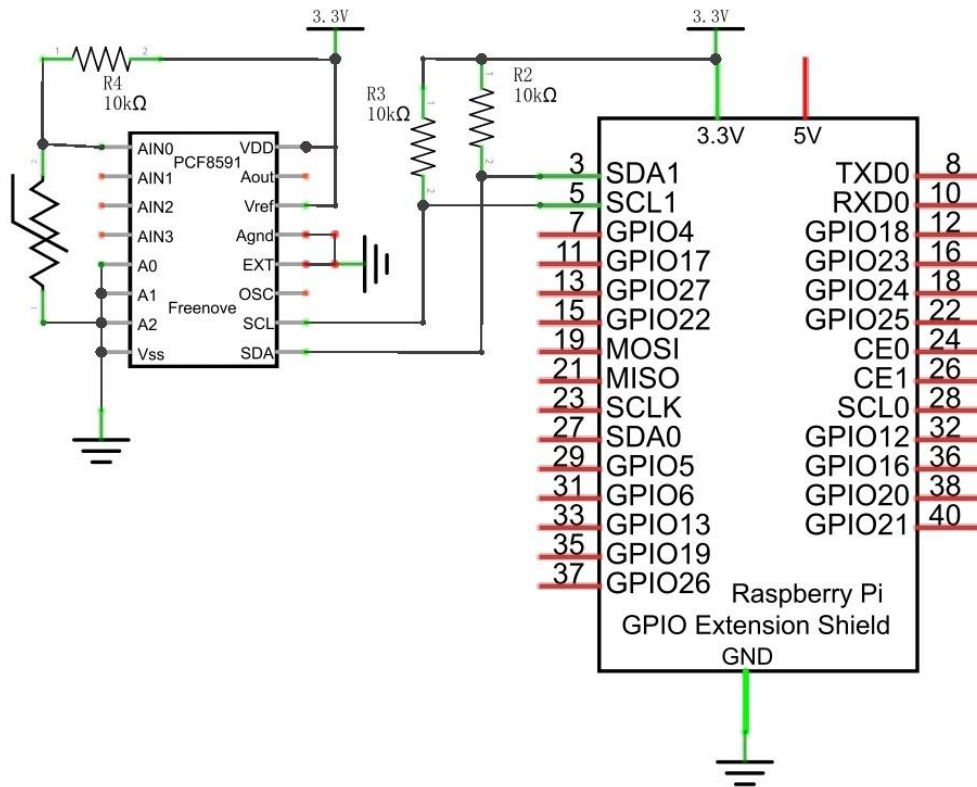
Thermistor has **longer pins** than the one shown in circuit.

Video: <https://youtu.be/spOalxanNMc>

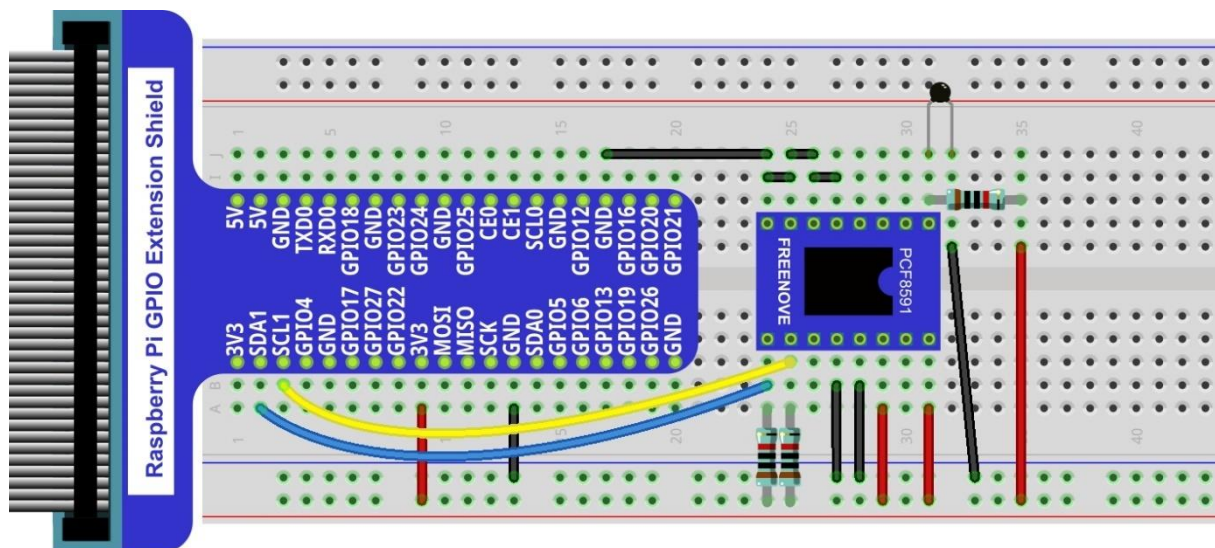
Circuit with PCF8591

The circuit of this project is similar to the one in the last chapter. The only difference is that the Photoresistor is replaced by the Thermistor.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Thermistor has longer pins than the one shown in circuit.

Code

In this project code, the ADC value still needs to be read, but the difference here is that a specific formula is used to calculate the temperature value.

Python Code 11.1.1 Thermometer

If you did not [configure I2C](#), please refer to [Chapter 7](#). If you did, please continue.
First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 11.1.1_Thermometer directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless/11.1.1_Thermometer
```

2. Use python command to execute Python code "Thermometer.py".

```
python Thermometer.py
```

After the program is executed, the Terminal window will display the current ADC value, voltage value and temperature value. Try to "pinch" the thermistor (without touching the leads) with your index finger and thumb for a brief time, you should see that the temperature value increases.

```
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
ADC Value : 107, Voltage : 1.38, Temperature : 32.48
```

The following is the code:

```
1  import time
2  import math
3  from ADCDevice import *
4
5  adc = ADCDevice() # Define an ADCDevice class object
6
7  def setup():
8      global adc
9      if(adc.detectI2C(0x48)): # Detect the pcf8591.
10         adc = PCF8591()
11         elif(adc.detectI2C(0x4b)): # Detect the ads7830
12             adc = ADS7830()
13     else:
```

```
14     print("No correct I2C address found, \n")
15     "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
16     "Program Exit. \n");
17     exit(-1)
18
19 def loop():
20     while True:
21         value = adc.analogRead(0)          # read ADC value A0 pin
22         voltage = value / 255.0 * 3.3      # calculate voltage
23         Rt = 10 * voltage / (3.3 - voltage) # calculate resistance value of thermistor
24         tempK = 1/(1/(273.15 + 25) + math.log(Rt/10)/3950.0) # calculate temperature (Kelvin)
25         tempC = tempK -273.15             # calculate temperature (Celsius)
26         print ('ADC Value : %d, Voltage : %.2f, Temperature : %.2f'%(value,voltage,tempC))
27         time.sleep(0.01)
28
29 def destroy():
30     adc.close()
31
32 if __name__ == '__main__': # Program entrance
33     print ('Program is starting ... ')
34     setup()
35     try:
36         loop()
37     except KeyboardInterrupt: # Press ctrl-c to end the program.
38         destroy()
39     print("Ending program")
```

In the code, the ADC value of ADC module A0 port is read, and then calculates the voltage and the resistance of Thermistor according to Ohms Law. Finally, it calculates the temperature sensed by the Thermistor, according to the formula.

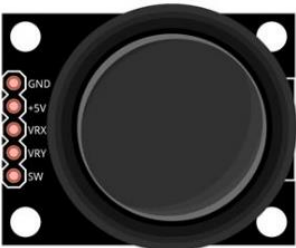
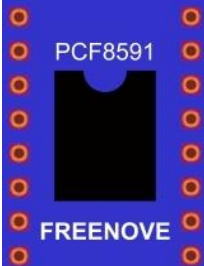
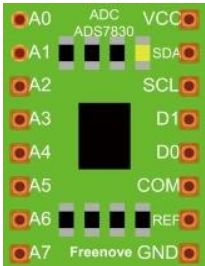
Chapter 12 Joystick

In an earlier chapter, we learned how to use Rotary Potentiometer. We will now learn about joysticks, which are electronic modules that work on the same principle as the Rotary Potentiometer.

Project 12.1 Joystick

In this project, we will read the output data of a joystick and display it to the Terminal screen.

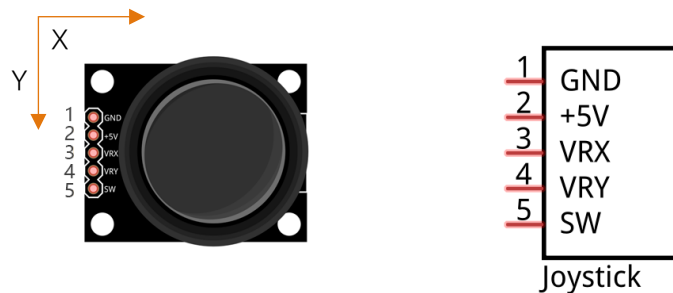
Component List

Raspberry Pi x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1		Jumper x18 
Joystick x1 	ADC module x1  or 	Resistor 10kΩ x3 

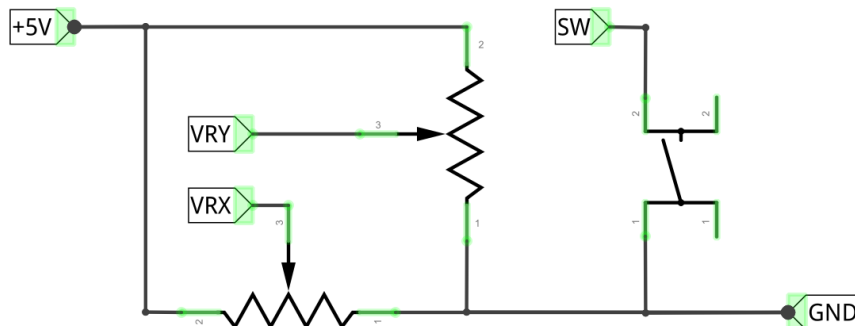
Component knowledge

Joystick

A Joystick is a kind of input sensor used with your fingers. You should be familiar with this concept already as they are widely used in gamepads and remote controls. It can receive input on two axes (Y and or X) at the same time (usually used to control direction on a two dimensional plane). And it also has a third direction capability by **pressing down (Z axis/direction)**.



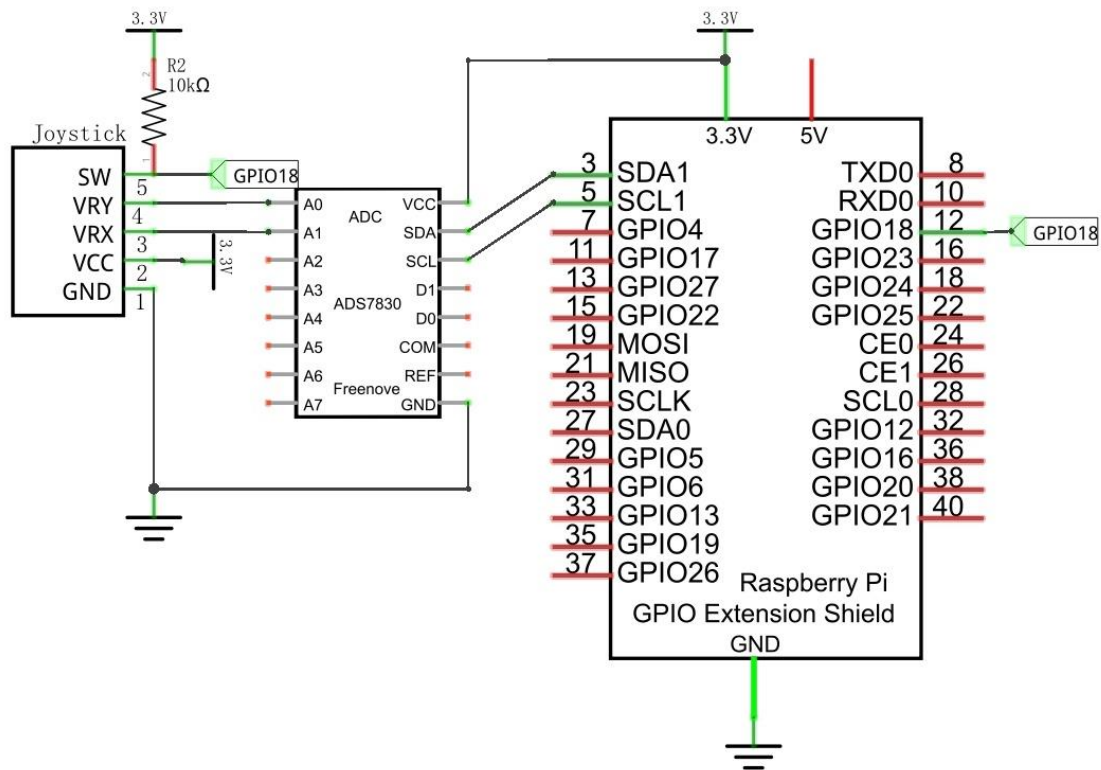
This is accomplished by incorporating two rotary potentiometers inside the Joystick Module at 90 degrees of each other, placed in such a manner as to detect shifts in direction in two directions simultaneously and with a Push Button Switch in the “vertical” axis, which can detect when a User presses on the Joystick.



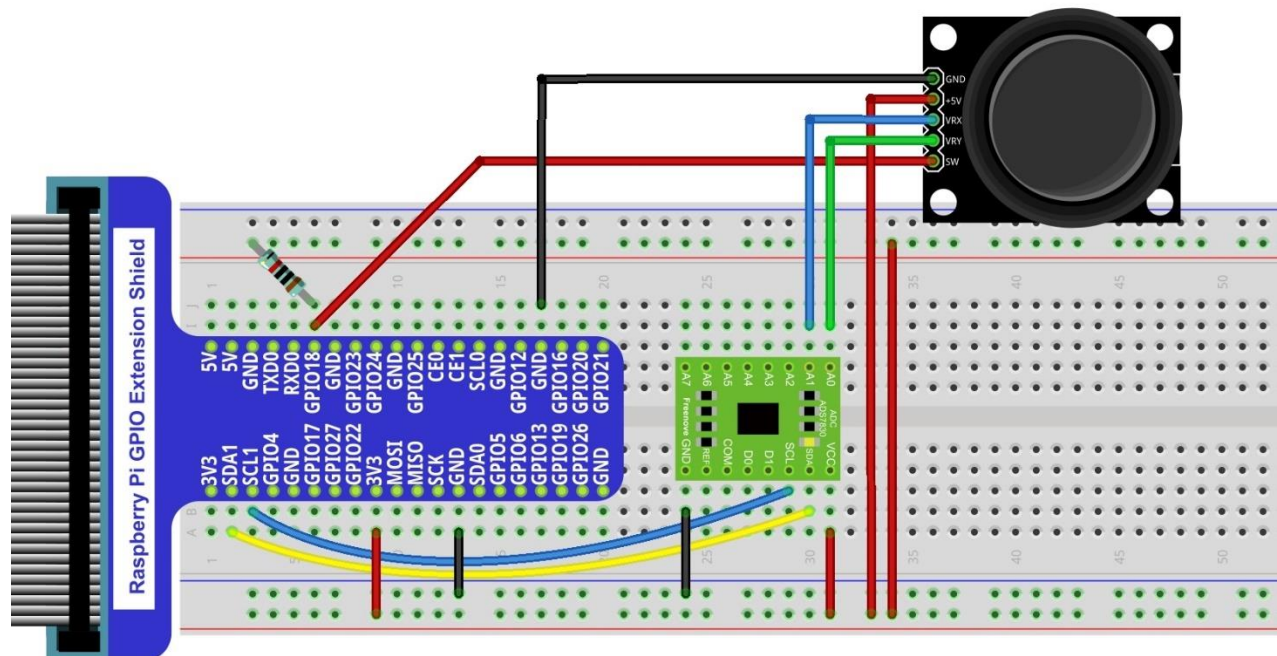
When the Joystick data is read, there are some differences between the axes: data of X and Y axes is analog, which needs to use the ADC. The data of the Z axis is digital, so you can directly use the GPIO to read this data or you have the option to use the ADC to read this.

Circuit with ADS7830

Schematic diagram



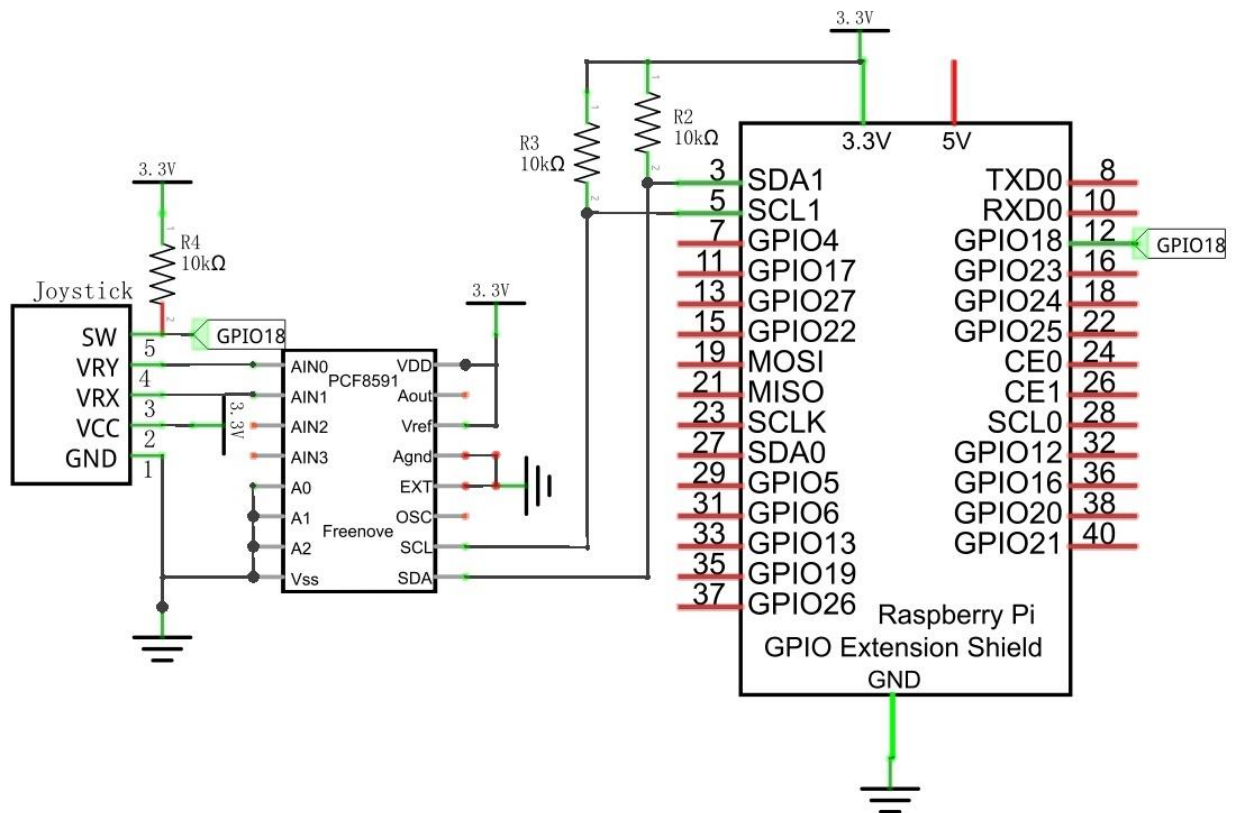
Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



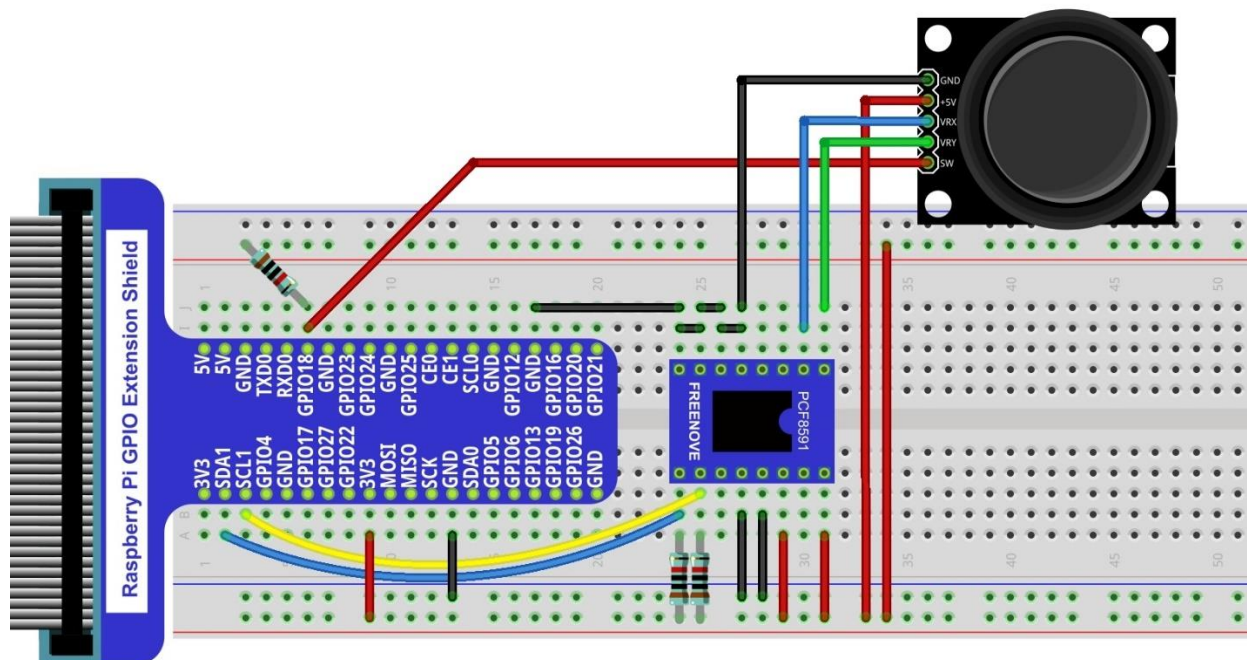
Video: <https://youtu.be/qjP3HpbPJTM>

Circuit with PCF8591

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

In this project's code, we will read the ADC values of X and Y axes of the Joystick, and read digital quality of the Z axis, then display these out in Terminal.

Python Code 12.1.1 Joystick

If you did not [configure I2C](#), please refer to [Chapter 7](#). If you did, please continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 12.1.1_Joystick directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless/12.1.1_Joystick
```

2. Use Python command to execute Python code "Joystick.py".

```
python Joystick.py
```

After the program is executed, the Terminal window will display the data of 3 axes X, Y and Z. Shifting (moving) the joystick or pressing it down will make the data change.

```
value_X: 128 , value_Y: 135 , value_Z: 1
value_X: 128 , value_Y: 135 , value_Z: 1
value_X: 128 , value_Y: 135 , value_Z: 1
value_X: 128 , value_Y: 135 , value_Z: 0
value_X: 128 , value_Y: 135 , value_Z: 0
value_X: 128 , value_Y: 135 , value_Z: 0
value_X: 128 , value_Y: 135 , value_Z: 0
value_X: 128 , value_Y: 135 , value_Z: 0
value_X: 128 , value_Y: 135 , value_Z: 0
value_X: 128 , value_Y: 135 , value_Z: 1
```

The following is the program code:

```
1  from gpiozero import Button
2  import time
3  from ADCDevice import *
4
5  Z_Pin = 18      # define Z_Pin
6  button = Button(Z_Pin) # define Button pin according to BCM Numbering
7  adc = ADCDevice() # Define an ADCDevice class object
8
9  def setup():
10     global adc
11     if(adc.detectI2C(0x48)): # Detect the pcf8591.
12         adc = PCF8591()
13     elif(adc.detectI2C(0x4b)): # Detect the ads7830
14         adc = ADS7830()
15     else:
16         print("No correct I2C address found, \n"
17             "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
18             "Program Exit. \n");
19         exit(-1)
20
```

```

21 def loop():
22     while True:
23         val_Z = not button.value           # read digital value of axis Z
24         val_Y = adc.analogRead(0)         # read analog value of axis X and Y
25         val_X = adc.analogRead(1)
26         print (' value_X: %d , \tvluue_Y: %d , \tvalue_Z: %d'%(val_X, val_Y, val_Z))
27         time.sleep(0.01)
28
29 def destroy():
30     adc.close()
31     button.close()
32
33 if __name__ == '__main__':
34     print ('Program is starting ... ') # Program entrance
35     setup()
36     try:
37         loop()
38     except KeyboardInterrupt: # Press ctrl-c to end the program.
39         destroy()
40         print("Ending program")

```

In the code, configure Z_Pin as pull-up input mode. In the while loop, use analogRead() to read the values of the axes X and Y and use the variable val_Z to save the value of the button.value variable for the Z axis, and then display them. When the button is pressed, the value of the variable button.value is 1, otherwise the value is 0.

```

def loop():
    while True:
        val_Z = not button.value           # read digital value of axis Z
        val_Y = adc.analogRead(0)         # read analog value of axis X and Y
        val_X = adc.analogRead(1)
        print (' value_X: %d , \tvluue_Y: %d , \tvalue_Z: %d'%(val_X, val_Y, val_Z))
        time.sleep(0.01)

```

For more information about the methods used by the Button class in the GPIO Zero library, please refer to: https://gpiozero.readthedocs.io/en/stable/api_input.html#button

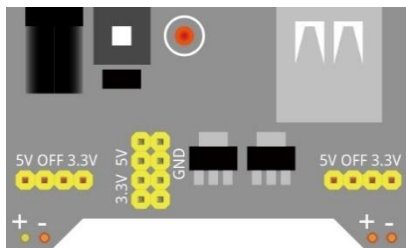
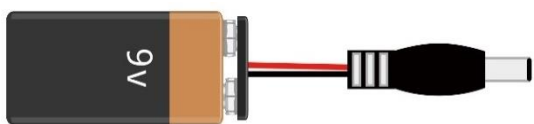
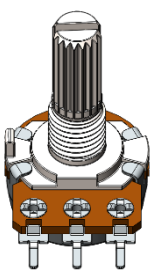
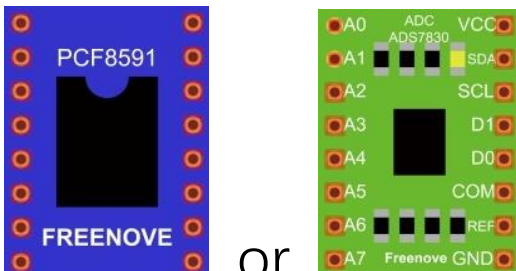
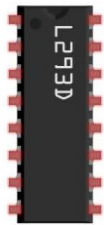
Chapter 13 Motor & Driver

In this chapter, we will learn about DC Motors and DC Motor Drivers and how to control the speed and direction of a DC Motor.

Project 13.1 Control a DC Motor with a Potentiometer

In this project, a potentiometer will be used to control a DC Motor. When the Potentiometer is at the midpoint position, the DC Motor will STOP, and when the Potentiometer is turned in either direction of this midpoint, the DC Motor speed increases until it reached the endpoint where the DC Motor achieves its maximum speed. When the Potentiometer is turned “Left” of the midpoint the DC Motor will ROTATE in one direction and when turned “Right” the DC Motor will ROTATE in the opposite direction.

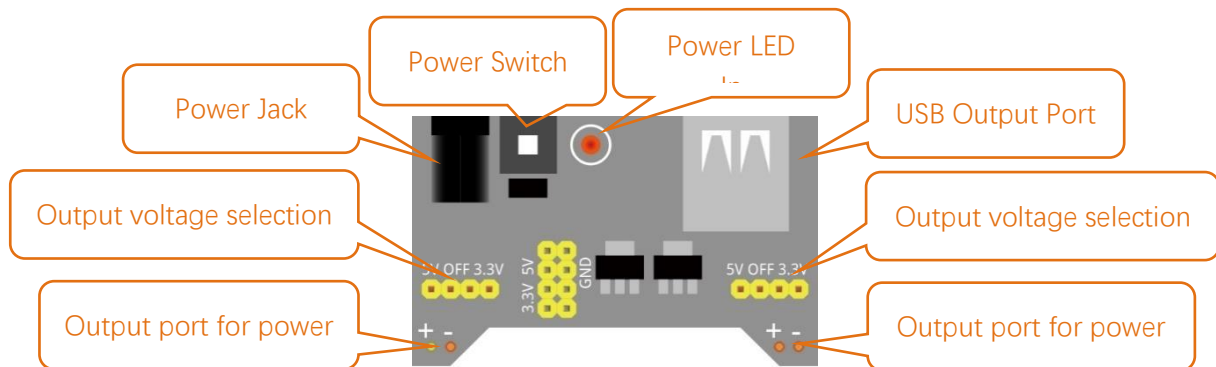
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1			Jumper Wires x23 	
Breadboard Power Module x1 			9V Battery (you provide) & 9V Battery Cable 	
Rotary Potentiometer x1 	DC Motor x1 	10kΩ x2 	ADC Module x1 	L293D IC Chip 

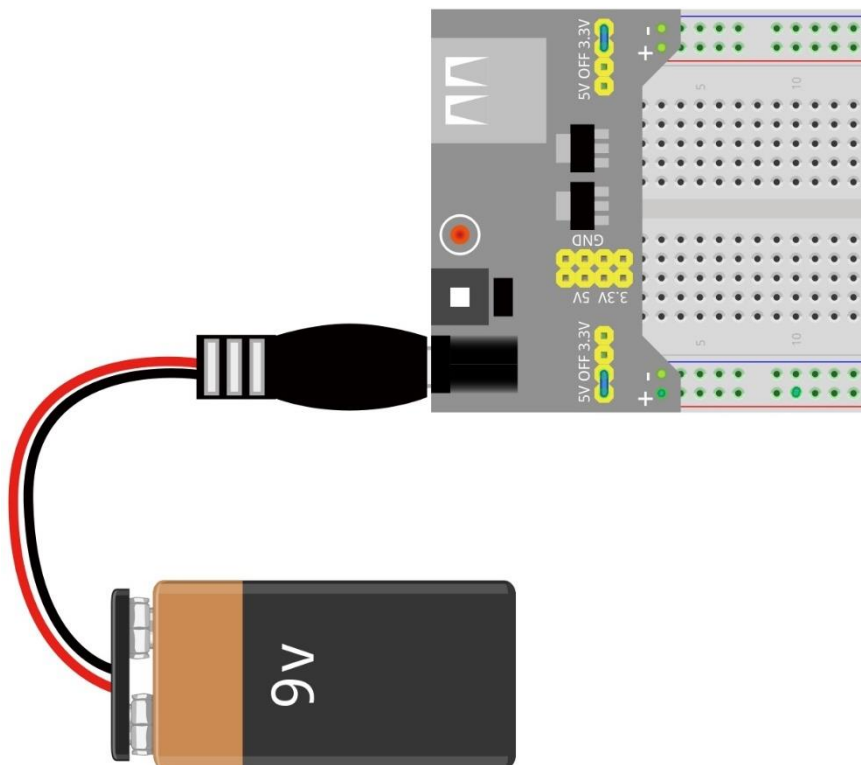
Component knowledge

Breadboard Power Module

Breadboard Power Module is an independent circuit board, which can provide independent 5V or 3.3V power to the breadboard when building circuits. It also has built-in power protection to avoid damaging your RPi module. The schematic diagram below identifies the important features of this Power Module:

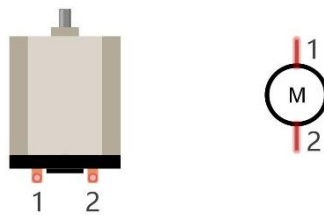


Here is an acceptable connection between Breadboard Power Module and Breadboard using a 9V battery and the provided power harness:



DC Motor

DC Motor is a device that converts electrical energy into mechanical energy. DC Motors consist of two major parts, a Stator and the Rotor. The stationary part of a DC Motor is the Stator and the part that Rotates is the Rotor. The Stator is usually part of the outer case of motor (if it is simply a pair of permanent magnets), and it has terminals to connect to the power if it is made up of electromagnet coils. Most Hobby DC Motors only use Permanent Magnets for the Stator Field. The Rotor is usually the shaft of motor with 3 or more electromagnets connected to a commutator where the brushes (via the terminals 1 & 2 below) supply electrical power, which can drive other mechanical devices. The diagram below shows a small DC Motor with two terminal pins.

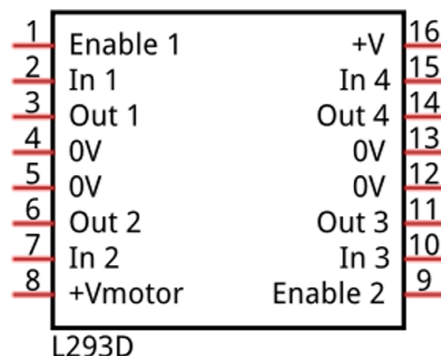
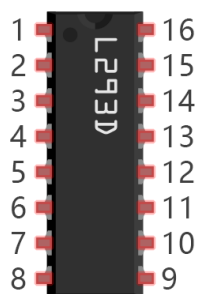


When a DC Motor is connected to a power supply, it will rotate in one direction. If you reverse the polarity of the power supply, the DC Motor will rotate in opposite direction. This is important to note.



L293D

L293D is an IC Chip (Integrated Circuit Chip) with a 4-channel motor drive. You can drive a Unidirectional DC Motor with 4 ports or a Bi-Directional DC Motor with 2 ports or a Stepper Motor (Stepper Motors are covered later in this Tutorial).



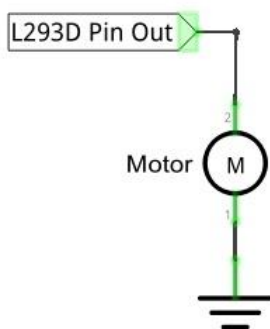
Port description of L293D module is as follows:

Pin name	Pin number	Description
In x	2, 7, 10, 15	Channel x digital signal input pin
Out x	3, 6, 11, 14	Channel x output pin, input high or low level according to In x pin, gets connected to +Vmotor or 0V
Enable1	1	Channel 1 and Channel 2 enable pin, high level enable
Enable2	9	Channel 3 and Channel 4 enable pin, high level enable
0V	4, 5, 12, 13	Power Cathode (GND)
+V	16	Positive Electrode (VCC) of power supply, supply voltage 4.5~36V
+Vmotor	8	Positive Electrode of load power supply, provide power supply for the Out pin x, the supply voltage is +V~36V

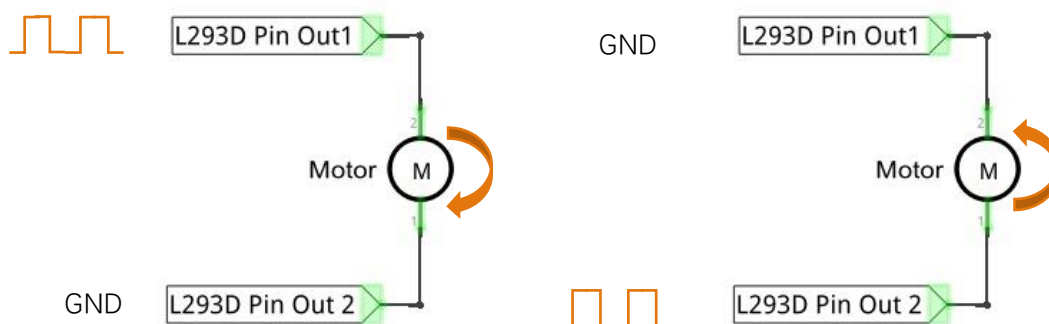
For more details, please see the datasheet for this IC Chip.

When using the L293D to drive a DC Motor, there are usually two connection options.

The following connection option uses one channel of the L293D, which can control motor speed through the PWM, However the motor then can only rotate in one direction.



The following connection uses two channels of the L293D: one channel outputs the PWM wave, and the other channel connects to GND. Therefore, you can control the speed of the motor. When these two channel signals are exchanged, not only controls the speed of motor, but also can control the direction of the motor.

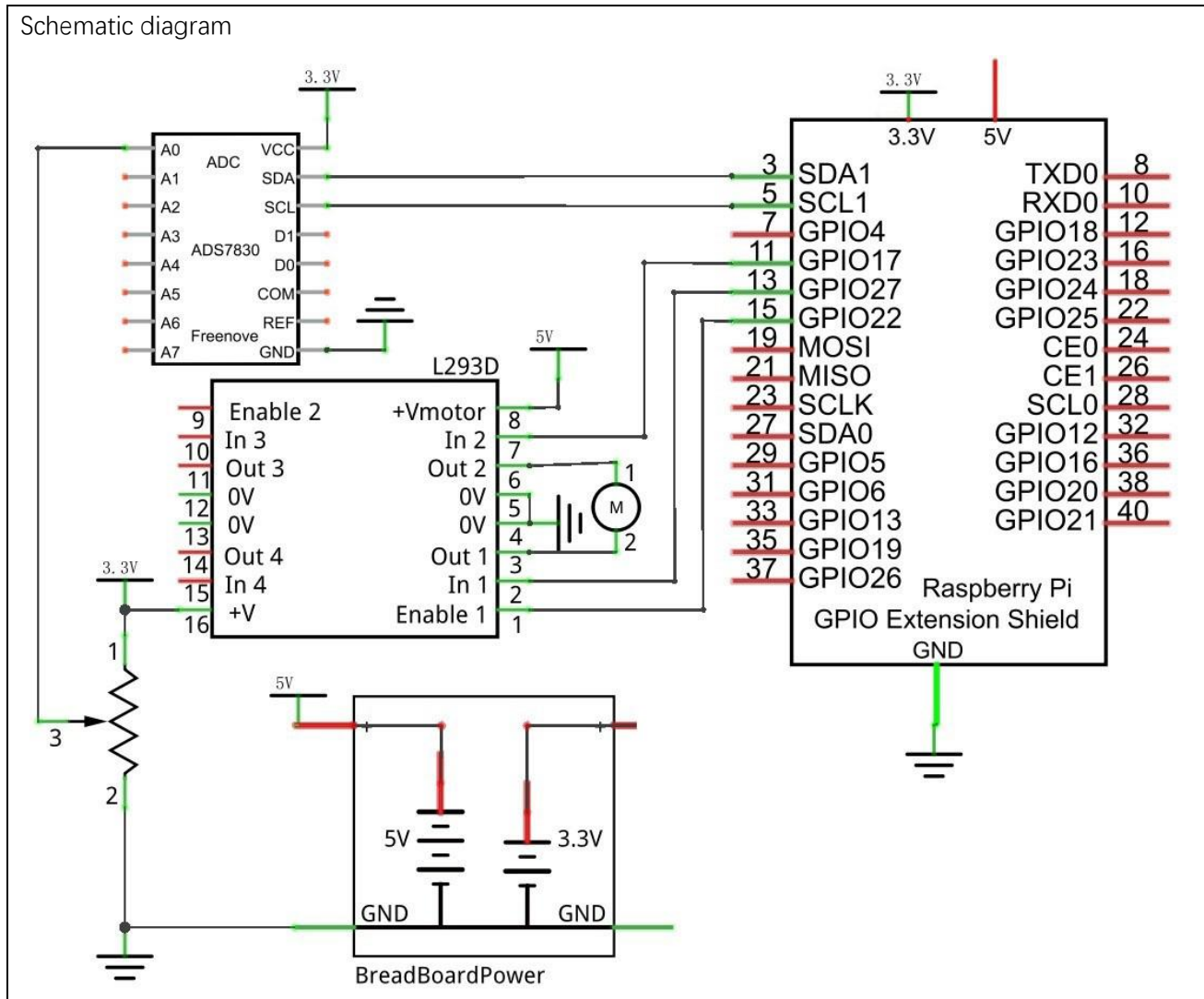


In practical use the motor is usually connected to channel 1 and by outputting different levels to in1 and in2 to control the rotational direction of the motor, and output to the PWM wave to Enable1 port to control the motor's rotational speed. If the motor is connected to channel 3 and 4 by outputting different levels to in3 and in4 to control the motor's rotation direction, and output to the PWM wave to Enable2 pin to control the motor's rotational speed.

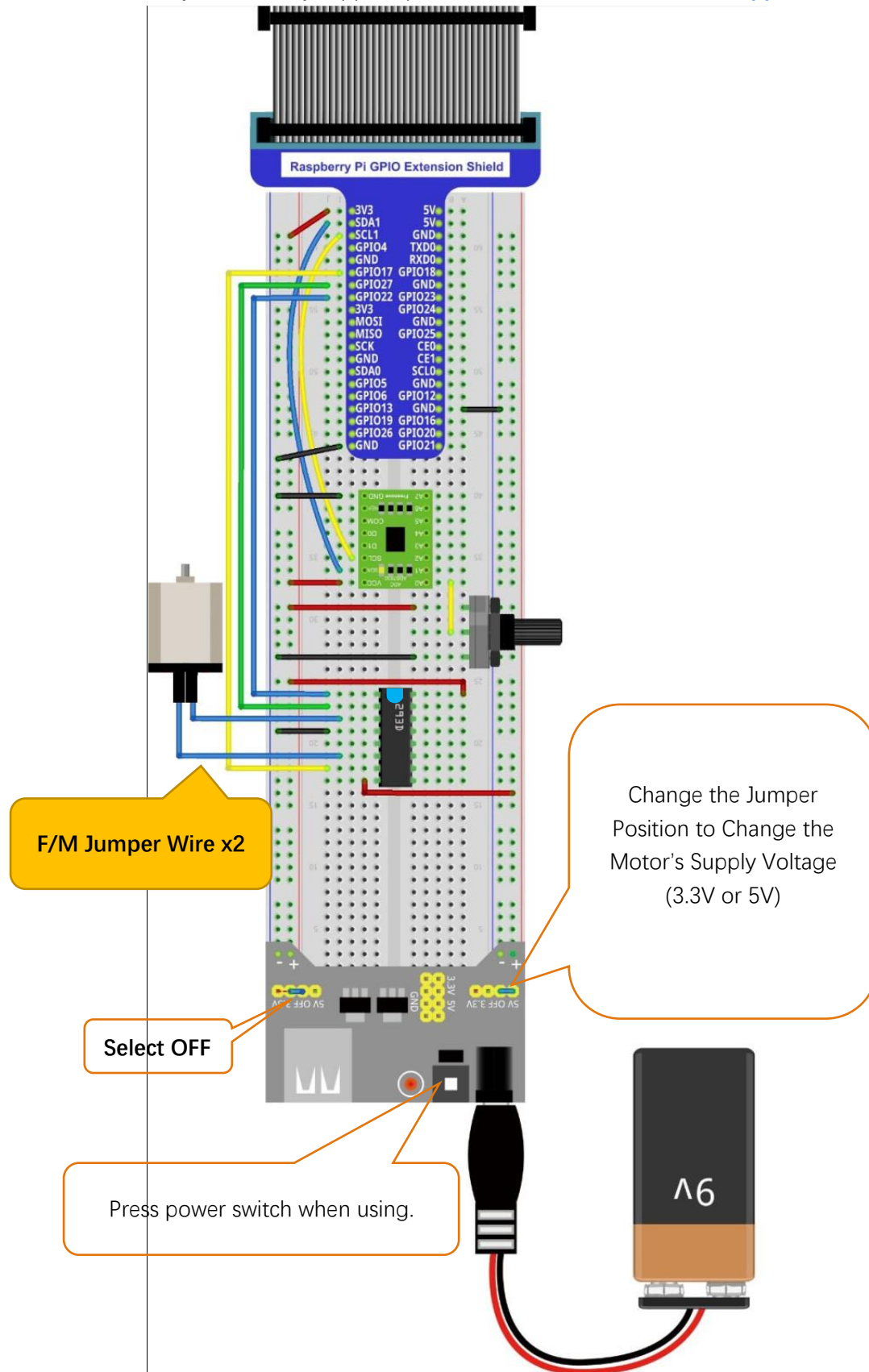
Circuit with ADS7830

Use caution when connecting this circuit because the DC Motor is a high-power component. **Do not use the power provided by the RPi to power the motor directly, as this may cause permanent damage to your RPi!** The logic circuit can be powered by the RPi's power or an external power supply, which should share a common ground with RPi.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

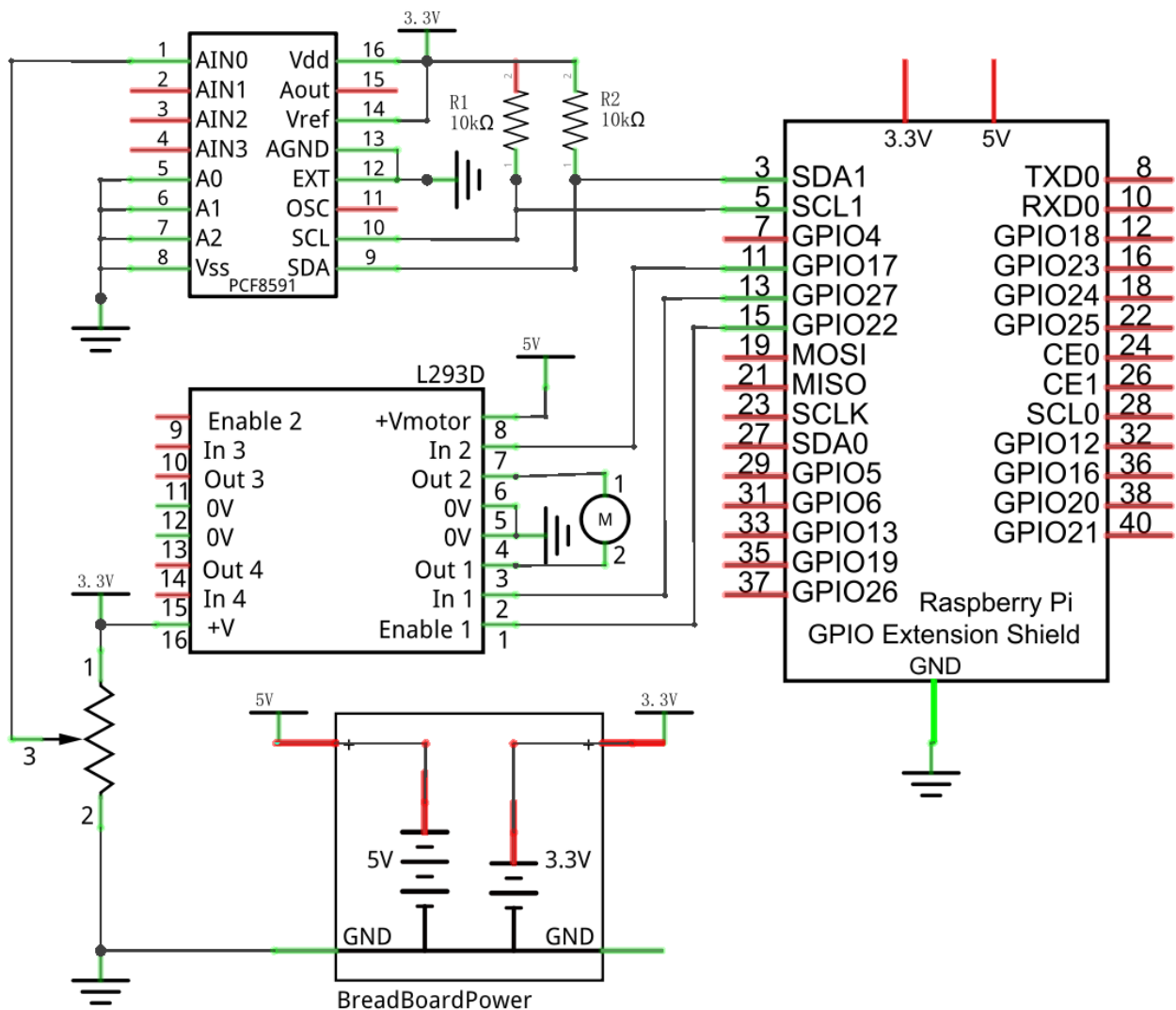


Video: <https://youtu.be/d5IRMTDK-wg>

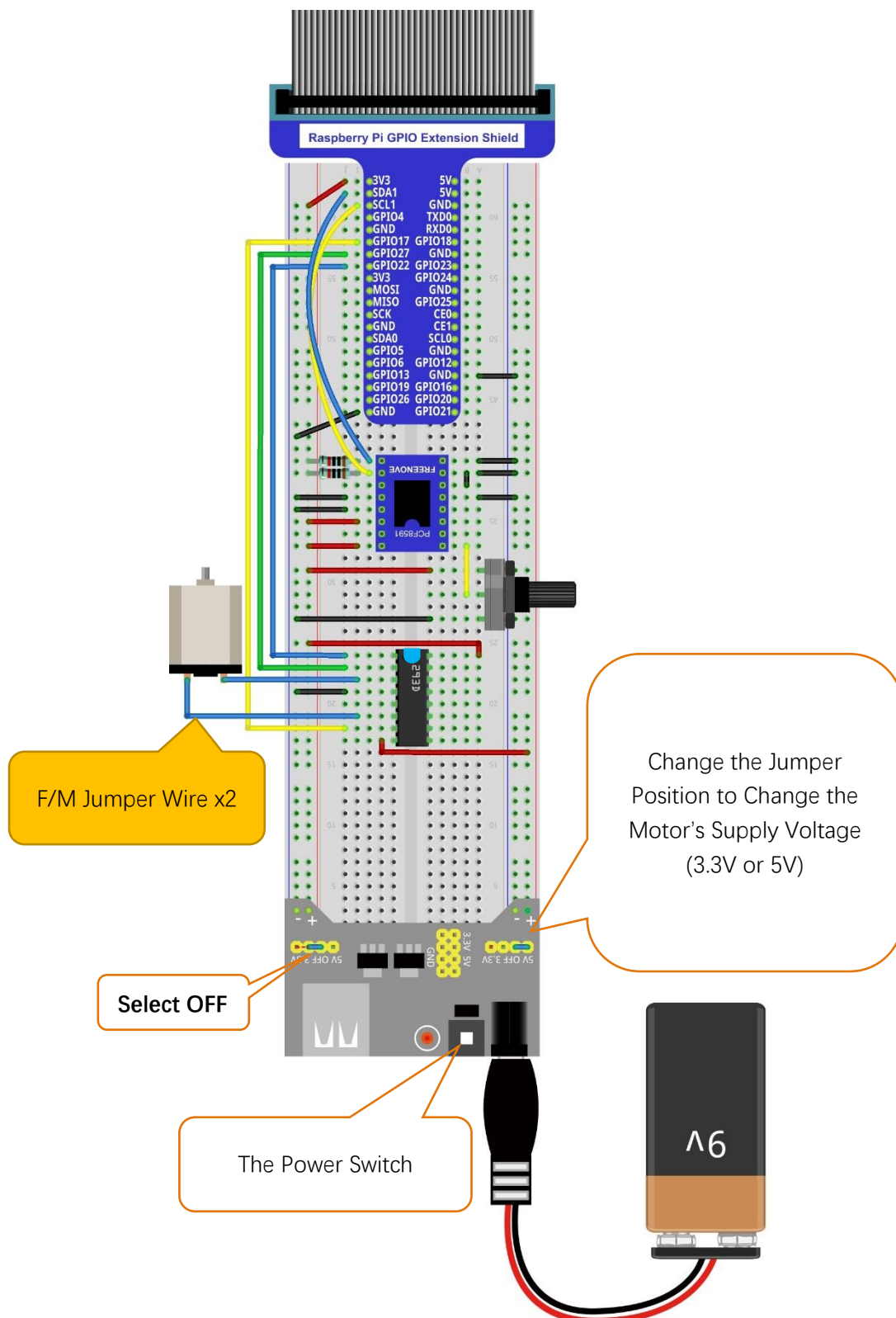
Circuit with PCF8591

Use caution when connecting this circuit because the DC Motor is a high-power component. **Do not use the power provided by the RPi to power the motor directly, as this may cause permanent damage to your RPi!** The logic circuit can be powered by the RPi's power or an external power supply, which should share a common ground with RPi.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Code

In code for this project, first read the ADC value and then control the rotation direction and speed of the DC Motor according to the value of the ADC.

Python Code 13.1.1 Motor

If you did not [configure I2C and install Smbus](#), please refer to [Chapter 7](#). If you did, please Continue. First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 13.1.1_Motor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless_Code/13.1.1_Motor
```

2. Use python command to execute the Python code "Motor.py".

```
python Motor.py
```

After the program is executed, you can use the Potentiometer to control the DC Motor. When the Potentiometer is at the midpoint position, the DC Motor will STOP, and when the Potentiometer is turned in either direction of this midpoint, the DC Motor speed increases until it reaches the endpoint where the DC Motor achieves its maximum speed. When the Potentiometer is turned "Left" of the midpoint the DC Motor will ROTATE in one direction and when turned "Right" the DC Motor will ROTATE in the opposite direction. You will also see the ADC value of the potentiometer displayed in the Terminal with the motor direction and the PWM duty cycle used to control the DC Motor's speed.

```
Turn Forward...
The PWM duty cycle is 100%

ADC Value : 255
Turn Forward...
The PWM duty cycle is 100%

ADC Value : 255
Turn Forward...
The PWM duty cycle is 100%

ADC Value : 255
Turn Forward...
The PWM duty cycle is 100%
```

The following is the code:

```
1  from gpiozero import DigitalOutputDevice, PWMOutputDevice
2  import time
3  from ADCDevice import *
4
5  # define the pins connected to L293D
6  motorPin1 = DigitalOutputDevice(27)          # define L293D pin according to BCM Numbering
7  motorPin2 = DigitalOutputDevice(17)          # define L293D pin according to BCM Numbering
8  enablePin = PWMOutputDevice(22, frequency=1000)
9  adc = ADCDevice() # Define an ADCDevice class object
```

```

10
11 def setup():
12     global adc
13     if(adc.detectI2C(0x48)): # Detect the pcf8591.
14         adc = PCF8591()
15     elif(adc.detectI2C(0x4b)): # Detect the ads7830
16         adc = ADS7830()
17     else:
18         print("No correct I2C address found, \n"
19             "Please use command 'i2cdetect -y 1' to check the I2C address! \n"
20             "Program Exit. \n");
21         exit(-1)
22 # mapNUM function: map the value from a range of mapping to another range.
23 def mapNUM(value,fromLow,fromHigh,toLow,toHigh):
24     return (toHigh-toLow)*(value-fromLow) / (fromHigh-fromLow) + toLow
25
26 # motor function: determine the direction and speed of the motor according to the input ADC
27 # value input
28 def motor(ADC):
29     value = ADC -128
30     if (value > 0): # make motor turn forward
31         motoRPin1.on()      # motoRPin1 output HIGH level
32         motoRPin2.off()     # motoRPin2 output LOW level
33         print (' Turn Forward...')
34     elif (value < 0): # make motor turn backward
35         motoRPin1.off()
36         motoRPin2.on()
37         print (' Turn Backward...')
38     else :
39         motoRPin1.off()
40         motoRPin2.off()
41         print (' Motor Stop...')
42     b=mapNUM(abs(value),0,128,0,100)
43     enablePin.value = b / 100.0      # set dc value as the duty cycle
44     print (' The PWM duty cycle is %d%%\n'%(abs(value)*100/127)) # print PMW duty cycle.
45
46 def loop():
47     while True:
48         value = adc.analogRead(0) # read ADC value of channel 0
49         print (' ADC Value : %d'%(value))
50         motor(value)
51         time.sleep(0.2)
52
53 def destroy():

```



```

52     motoRPin1.close()
53     motoRPin2.close()
54     enablePin.close()
55     adc.close()
56
57 if __name__ == '__main__': # Program entrance
58     print ('Program is starting ... ')
59     setup()
60     try:
61         loop()
62     except KeyboardInterrupt: # Press ctrl-c to end the program.
63         destroy()
64         print("Ending program")

```

Now that we have familiarity with reading ADC values, let's learn the subfunction void motor (int ADC): first, compare the ADC value with 128 (value corresponding to midpoint). When the current ADC value is higher, motoRPin1 outputs high level and motoRPin2 outputs low level to control the DC Motor to run in the "Forward" Rotational Direction. When the current ADC value is lower, motoRPin1 outputs low level and motoRPin2 outputs high level to control the DC Motor to run in the "Reverse" Rotational Direction. When the ADC value is equal to 128, motoRPin1 and motoRPin2 output low level, the motor STOPS. Then determine the PWM duty cycle according to the difference (delta) between ADC value and 128. Because the absolute delta value stays within 0-128. We need to use the map() subfunction mapping the delta value to a range of 0-255. Finally, we see a display of the duty cycle in Terminal.

```

def motor(ADC):
    value = ADC - 128
    if (value > 0): # make motor turn forward
        motoRPin1.on()      # motoRPin1 output HIGH level
        motoRPin2.off()     # motoRPin2 output LOW level
        print (' Turn Forward...')
    elif (value < 0): # make motor turn backward
        motoRPin1.off()
        motoRPin2.on()
        print (' Turn Backward...')
    else :
        motoRPin1.off()
        motoRPin2.off()
        print (' Motor Stop...')
    b=mapNUM(abs(value), 0, 128, 0, 100)
    enablePin.value = b / 100.0      # set dc value as the duty cycle
    print (' The PWM duty cycle is %d%%\n'%(abs(value)*100/127)) # print PMW duty cycle.

```

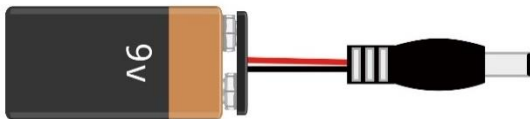
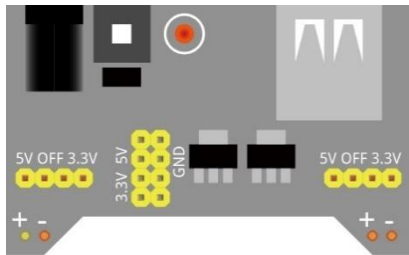
Chapter 14 Relay & Motor

In this chapter, we will learn a kind of special switch module, Relay Module.

Project 14.1.1 Relay & Motor

In this project, we will use a Push Button Switch indirectly to control the DC Motor via a Relay.

Component List

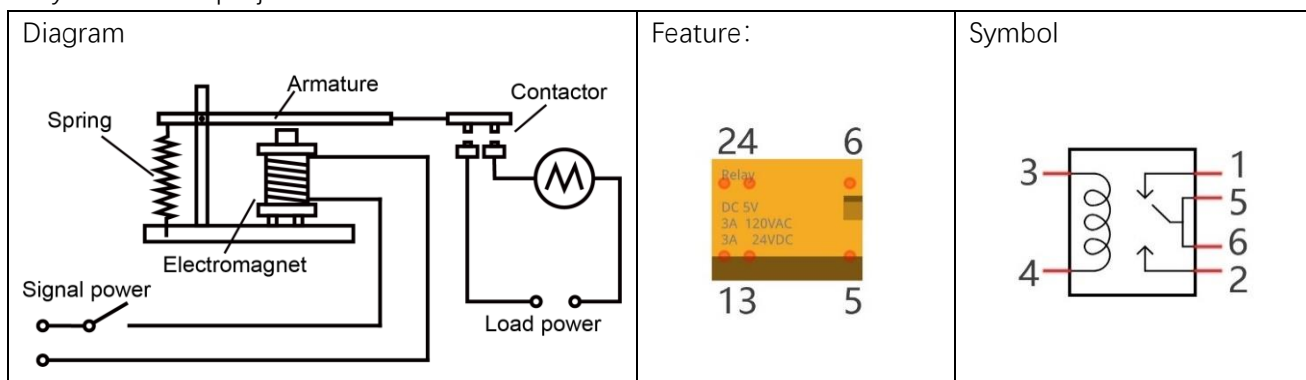
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1		Jumper Wire x11 			
9V battery (prepared by yourself) & battery line 					
Breadboard Power module x1 		Resistor 10kΩ x2 	Resistor 1kΩ x1 	Resistor 220Ω x1 	
NPN transistor x1 	Relay x1 	Motor x1 	Push button x1 	LED x1 	Diode x1 

Component knowledge

Relay

Relays are a type of Switch that open and close circuits electromechanically or electronically. Relays control one electrical circuit by opening and closing contacts in another circuit using an electromagnet to initiate the Switch action. When the electromagnet is energized (powered), it will attract internal contacts completing a circuit, which act as a Switch. Many times Relays are used to allow a low powered circuit (and a small low amperage switch) to safely turn ON a larger more powerful circuit. They are commonly found in automobiles, especially from the ignition to the starter motor.

The following is a basic diagram of a common Relay and the image and circuit symbol diagram of the 5V relay used in this project:



Pin 5 and pin 6 are internally connected to each other. When the coil pin3 and pin 4 are connected to a 5V power supply, pin 1 will be disconnected from pins 5 & 6 and pin 2 will be connected to pins 5 & 6. Pin 1 is called Closed End and pin 2 is called the Open End.

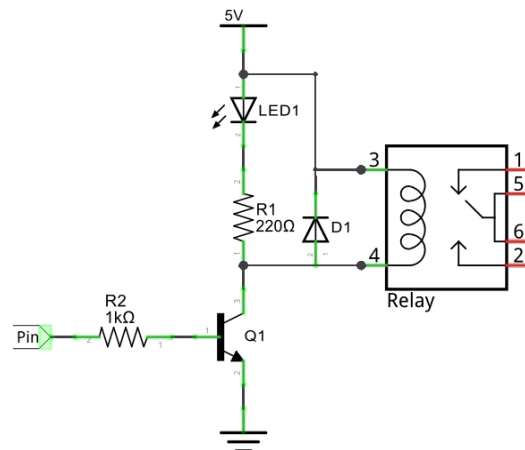
Inductor

The symbol of Inductance is "L" and the unit of inductance is the "Henry" (H). Here is an example of how this can be encountered: $1\text{H}=1000\text{mH}$, $1\text{mH}=1000\mu\text{H}$.

An Inductor is a passive device that stores energy in its Magnetic Field and returns energy to the circuit whenever required. An Inductor is formed by a Cylindrical Core with many Turns of conducting wire (usually copper wire). Inductors will hinder the changing current passing through it. When the current passing through the Inductor increases, it will attempt to hinder the increasing movement of current; and when the current passing through the inductor decreases, it will attempt to hinder the decreasing movement of current. So the current passing through an Inductor is not transient.



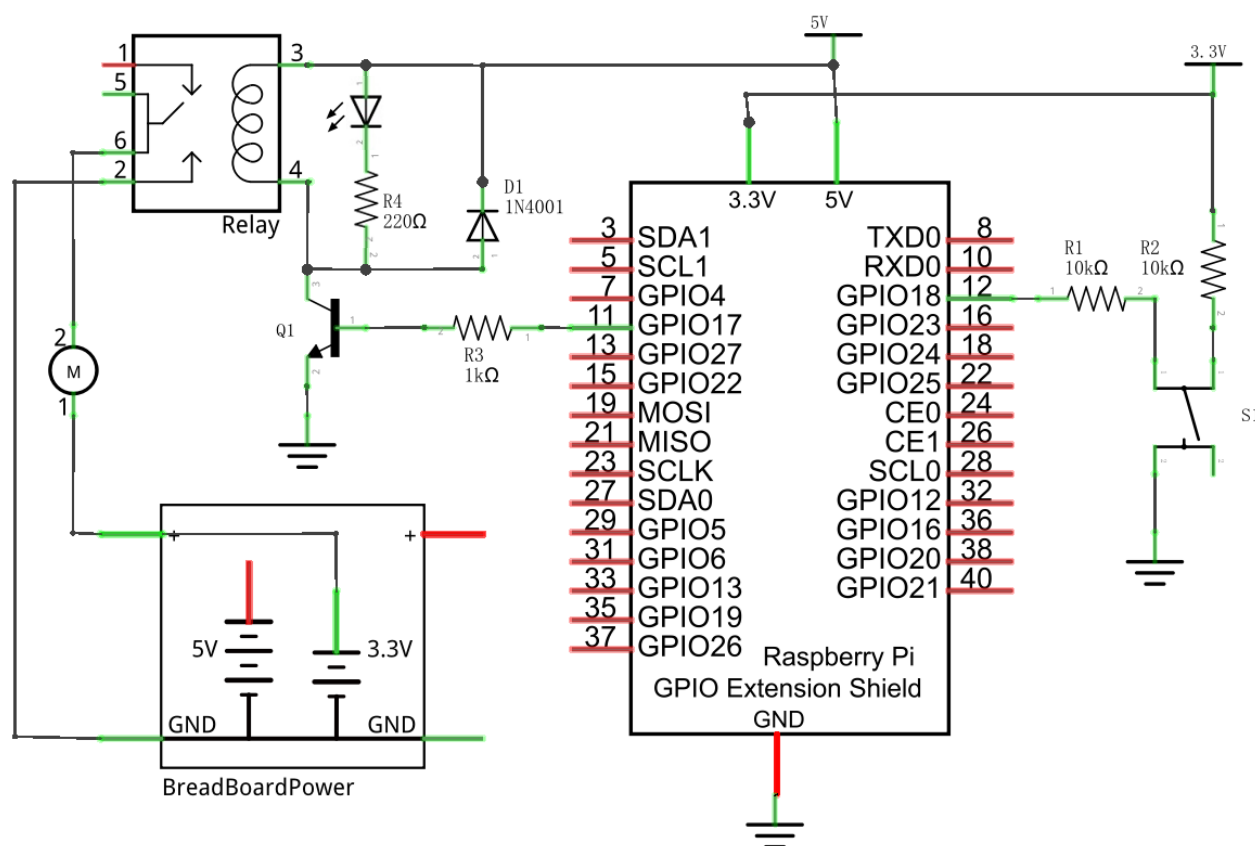
The circuit for a Relay is as follows: The coil of Relay can be equivalent to an Inductor, when a Transistor is present in this coil circuit it can disconnect the power to the relay, the current in the Relay's coil does not stop immediately, which affects the power supply adversely. To remedy this, diodes in parallel are placed on both ends of the Relay coil pins in opposite polar direction. Having the current pass through the diodes will avoid any adverse effect on the power supply.



Circuit

Use caution with the power supply voltage needed for the components in this circuit. The Relay requires a power supply voltage of 5V, and the DC Motor only requires 3.3V. Additionally, there is an LED present, which acts as an indicator (ON or OFF) for the status of the Relay's active status.

Schematic diagram





Code

The project code is in the same as we used earlier in the Table Lamp project. Pressing the Push Button Switch activates the transistor. Because the Relay and the LED are connected in parallel, they will be powered ON at the same time. Press the Push Button Switch again will turn them both OFF.

Python Code 14.1.1 Relay

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 14.1.1_Relay directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless_Code/14.1.1_Relay
```

2. Use Python command to execute code "Relay.py".

```
python Relay.py
```

After the program is executed, pressing the Push Button Switch activates the Relay (the internal switch is closed), which powers the DC Motor to rotate and simultaneously powers the LED to turn ON. If you press the Push Button Switch again, the Relay is deactivated (the internal switch opens), the Motor STOPS and the LED turns OFF.

The following is the program code:

```

1  from gpiozero import DigitalOutputDevice, Button
2  import time
3
4  relayPin = 17      # define the relayPin
5  buttonPin = 18     # define the buttonPin
6  relay = DigitalOutputDevice(relayPin)      # define LED pin according to BCM Numbering
7  button = Button(buttonPin) # define Button pin according to BCM Numbering
8
9  def onButtonPressed(): # When button is pressed, this function will be executed
10     relay.toggle()
11     if relay.value :
12         print("Turn on relay ...")
13     else :
14         print("Turn off relay ... ")
15
16  def loop():
17     button.when_pressed = onButtonPressed
18     while True:
19         time.sleep(1)
20
21  def destroy():
22     relay.close()
23     button.close()
24
25  if __name__ == '__main__':      # Program entrance
26     print ('Program is starting...')
```

```
27     try:
28         loop()
29     except KeyboardInterrupt: # Press ctrl-c to end the program.
30         destroy()
31         print("Ending program")
```

The project code is in the same as we used earlier in the Table Lamp project.

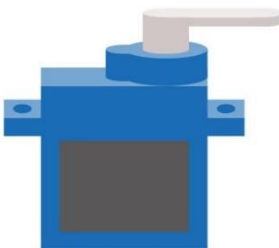
Chapter 15 Servo

Previously, we learned how to control the speed and rotational direction of a DC Motor. In this chapter, we will learn about Servos which are a rotary actuator type motor that can be controlled rotate to specific angles.

Project 15.1 Servo Sweep

First, we need to learn how to make a Servo rotate.

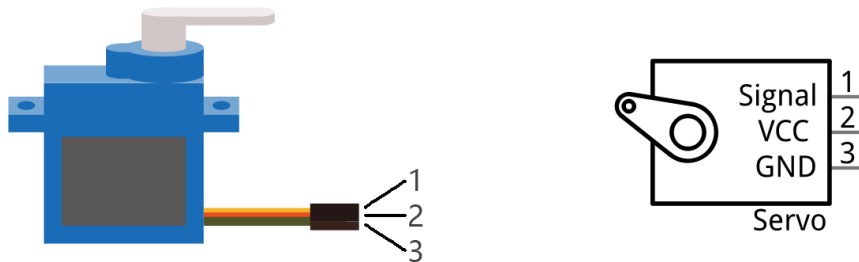
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper Wire x3 
Servo x1 	

Component knowledge

Servo

Servo is a compact package which consists of a DC Motor, a set of reduction gears to provide torque, a sensor and control circuit board. Most Servos only have a 180-degree range of motion via their “horn”. Servos can output higher torque than a simple DC Motor alone and they are widely used to control motion in model cars, model airplanes, robots, etc. Servos have three wire leads which usually terminate to a male or female 3-pin plug. Two leads are for electric power: Positive (2-VCC, Red wire), Negative (3-GND, Brown wire), and the signal line (1-Signal, Orange wire) as represented in the Servo provided in your Kit.



We will use a 50Hz PWM signal with a duty cycle in a certain range to drive the Servo. The lasting time 0.5ms-2.5ms of PWM single cycle high level corresponds to the Servo angle 0 degrees - 180 degree linearly. Part of the corresponding values are as follows:

Note: the lasting time of high level corresponding to the servo angle is absolute instead of accumulating. For example, the high level time lasting for 0.5ms correspond to the 0 degree of the servo. If the high level time lasts for another 1ms, the servo rotates to 45 degrees.

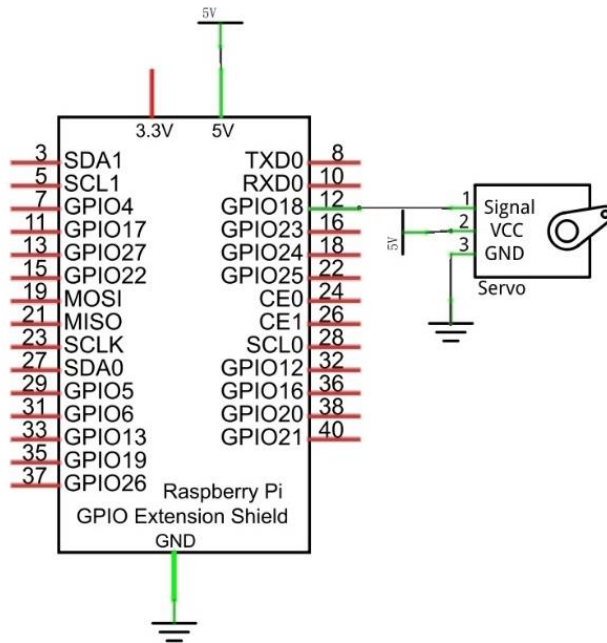
High level time	Servo angle
0.5ms	0 degree
1ms	45 degree
1.5ms	90 degree
2ms	135 degree
2.5ms	180 degree

When you change the Servo signal value, the Servo will rotate to the designated angle.

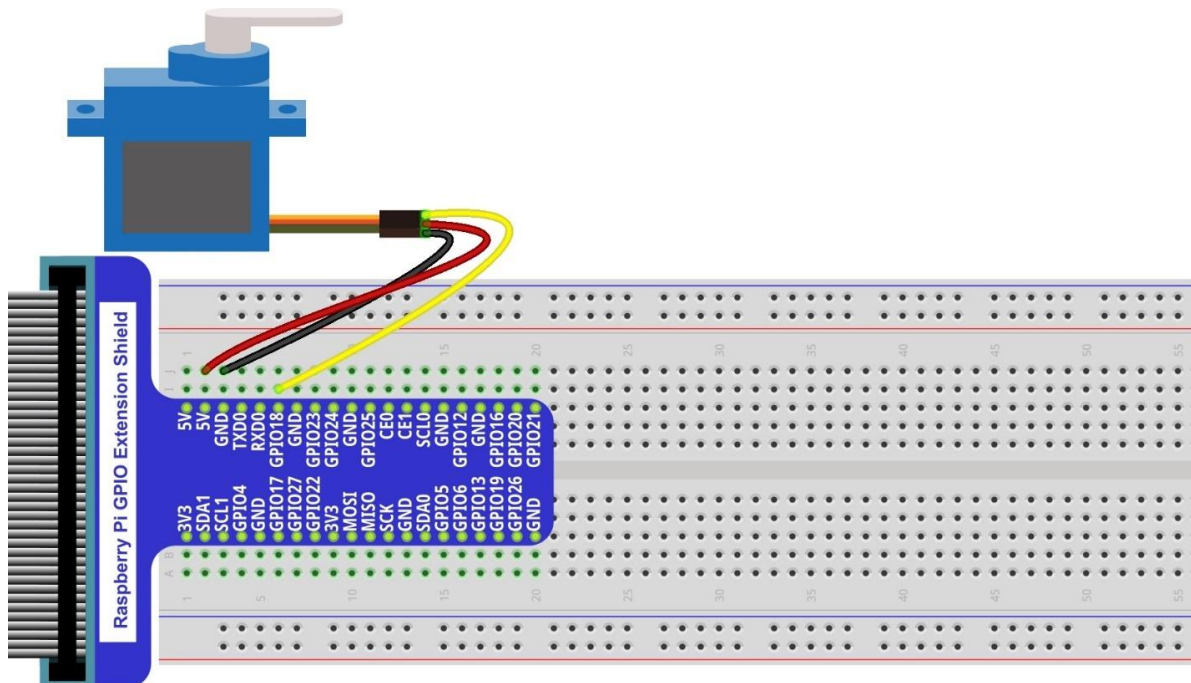
Circuit

Use caution when supplying power to the Servo it should be 5V. Make sure you do not make any errors when connecting the Servo to the power supply.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: <https://youtu.be/leptbJh32ZI>

Code

In this project, we will make a Servo rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop.

Python Code 15.1.1 Sweep

First observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 15.1.1_Sweep directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/15.1.1_Sweep
```

2. Use python command to execute code "Sweep.py".

```
python Sweep.py
```

After the program is executed, the Servo will rotate from 0 degrees to 180 degrees and then reverse the direction to make it rotate from 180 degrees to 0 degrees and repeat these actions in an endless loop.

The following is the program code:

```
1  from gpiozero import AngularServo
2  import time
3
4  myGPIO=18
5  SERVO_DELAY_SEC = 0.001
6  myCorrection=0.0
7  maxPW=(2.5+myCorrection)/1000
8  minPW=(0.5-myCorrection)/1000
9  servo = AngularServo(myGPIO, initial_angle=0, min_angle=0,
    max_angle=180, min_pulse_width=minPW, max_pulse_width=maxPW)
10
11 def loop():
12     while True:
13         for angle in range(0, 181, 1): # make servo rotate from 0 to 180 deg
14             servo.angle = angle
15             time.sleep(SERVO_DELAY_SEC)
16             time.sleep(0.5)
17         for angle in range(180, -1, -1): # make servo rotate from 180 to 0 deg
18             servo.angle = angle
19             time.sleep(SERVO_DELAY_SEC)
20             time.sleep(0.5)
21
22 if __name__ == '__main__': # Program entrance
23     print ('Program is starting...')
24     try:
25         loop()
26     except KeyboardInterrupt: # Press ctrl-c to end the program.
27         print("Ending program")
```

Import the AngularServo class that controls Servo from the gpiozero library.

```
from gpiozero import AngularServo
```

A 50 Hz pulse for a 20ms cycle is required to control the Servo. By default, the AngularServo class has set the control period to 20 milliseconds.

```
servo = AngularServo(myGPIO, initial_angle=0, min_angle=0,
max_angle=180, min_pulse_width=minPW, max_pulse_width=maxPW)
```

The 0-180 degree rotation of the servo corresponds to a PWM pulse width of 0.5-2.5ms at a period of 20ms and a duty cycle of 2.5%-12.5%. After setting the AngularServo class and passing in the corresponding angle parameters, the servo will turn to the corresponding position. However, in actual operation, as there is a deviation in the width of the servo pulse, we need to define minimum and maximum pulse width and error offset (this is essential in robotics).

```
myCorrection=0.0 #define pulse offset of servo
maxPW=(2.5+myCorrection)/1000 #define pulse duty cycle for minimum angle of servo
minPW=(0.5-myCorrection)/1000 #define pulse duty cycle for maximum angle of s
servo = AngularServo(myGPIO, initial_angle=0, min_angle=0,
max_angle=180, min_pulse_width=minPW, max_pulse_width=maxPW)
.....
OFFSE_DUTY = 0.5 #define pulse offset of servo
SERVO_MIN_DUTY = 2.5+OFFSE_DUTY #define pulse duty cycle for minimum angle of servo
SERVO_MAX_DUTY = 12.5+OFFSE_DUTY #define pulse duty cycle for maximum angle of s ervo
.....

for angle in range(0, 181, 1): # make servo rotate from 0 to 180 deg
    servo.angle = angle
    time.sleep(SERVO_DELAY_SEC)
    time.sleep(0.5)
```

Finally, in the "while" cycle of main function, we need to use two separate cycles to make servo rotate from 0 degrees to 180 degrees and then from 180 degrees to 0 degrees.

```
def loop():
    while True:
        for angle in range(0, 181, 1): # make servo rotate from 0 to 180 deg
            servo.angle = angle
            time.sleep(SERVO_DELAY_SEC)
        time.sleep(0.5)
        for angle in range(180, -1, -1): # make servo rotate from 180 to 0 deg
            servo.angle = angle
            time.sleep(SERVO_DELAY_SEC)
        time.sleep(0.5)
```

For more information about the methods used by the AngularServo class in the GPIO Zero library, please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#angularservo

In the above experiment, you can see that the servo jitter obviously when working.

Note: To reduce servo jitter, use the pigpio pin driver rather than the default RPi.GPIO driver (pigpio uses DMA sampling for much more precise edge timing).

You can refer to the code sweep2.py for more details.

1. Use cd command to enter 15.1.1_Sweep directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWZero_Code/15.1.1_Sweep
```

2. Use python command to execute code "Sweep.py".

```
python Sweep2.py
```

After the program executes, the servo will rotate from 0 degrees to 180 degrees, then reverse the direction so it rotates from 180 degrees to 0 degrees, and repeat these actions in an infinite loop. At this time, the steering gear can hardly feel the vibration.

This code is based on pigpio. In the latest Raspberry Pi OS, "pigpio" library has been installed. You only need to run the command to enable it.

```
sudo pigpiod
```

```
pi@raspberrypi:~ $ sudo pigpiod
pi@raspberrypi:~ $
```

If the "pigpio" library has not yet been installed, please follow the steps to install it.

Run the command to install "pigpio" library.

```
sudo apt-get update
```

```
sudo apt-get install pigpio python-pigpio python3-pigpio
```

The following is the program code:

```
1  import os
2  os.system("sudo pigpiod")
3  from gpiozero import AngularServo
4  from gpiozero.pins.pigpio import PiGPIOFactory
5  import time
6
7  my_factory = PiGPIOFactory()
8  myGPIO=18
9  SERVO_DELAY_SEC = 0.001
   myCorrection=0.0
10 maxPW=(2.5+myCorrection)/1000
11 minPW=(0.5-myCorrection)/1000
12 servo = AngularServo(myGPIO, initial_angle=0, min_angle=0,
13 max_angle=180, min_pulse_width=minPW, max_pulse_width=maxPW, pin_factory=my_factory)
14
15 def loop():
16     while True:
17         for angle in range(0, 181, 1): # make servo rotate from 0 to 180 deg
18             servo.angle = angle
19             time.sleep(SERVO_DELAY_SEC)
```

```

20     time.sleep(0.5)
21     for angle in range(180, -1, -1): # make servo rotate from 180 to 0 deg
22         servo.angle = angle
23         time.sleep(SERVO_DELAY_SEC)
24     time.sleep(0.5)
25
26 if __name__ == '__main__':    # Program entrance
27     print ('Program is starting...')
28     try:
29         loop()
30     except KeyboardInterrupt: # Press ctrl-c to end the program.
31         servo.close()
32         os.system("sudo killall pigpiod")
33     print("Ending program")

```

The following values, and the corresponding Factory and Pin classes are listed in the table below. Factories are listed in the order that they are tried by default.

Name	Factory class	Pin class
rpigpio	gpiozero.pins.rpigpio.RPiGPIOFactory	gpiozero.pins.rpigpio.RPiGPiOPin
lgpio	gpiozero.pins.lgpio.LGPIOFactory	gpiozero.pins.lgpio.LGPiOPin
rpio	gpiozero.pins.rpio.RPIOFactory	gpiozero.pins.rpio.RPiOPin
pigpio	gpiozero.pins.pigpio.PiGPIOFactory	gpiozero.pins.pigpio.PiGPiOPin
native	gpiozero.pins.native.NativeFactory	gpiozero.pins.native.NativePin

See Changing the pin factory for further information:

https://gpiozero.readthedocs.io/en/stable/api_pins.html#changing-pin-factory

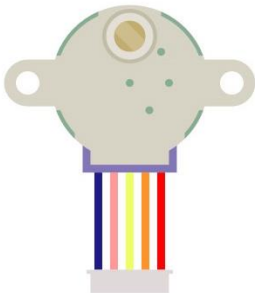
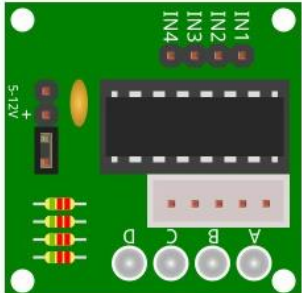
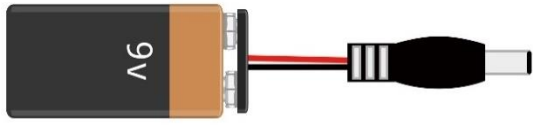
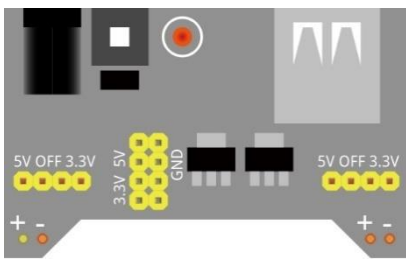
Chapter 16 Stepper Motor

Thus far, we have learned about DC Motors and Servos. A DC motor can rotate constantly in one direction but we cannot control the rotation to a specific angle. On the contrary, a Servo can rotate to a specific angle but cannot rotate constantly in one direction. In this chapter, we will learn about a Stepper Motor which is also a type of motor. A Stepper Motor can rotate constantly and also to a specific angle. Using a Stepper Motor can easily achieve higher accuracies in mechanical motion.

Project 16.1 Stepper Motor

In this project, we will learn how to drive a Stepper Motor, and understand its working principle.

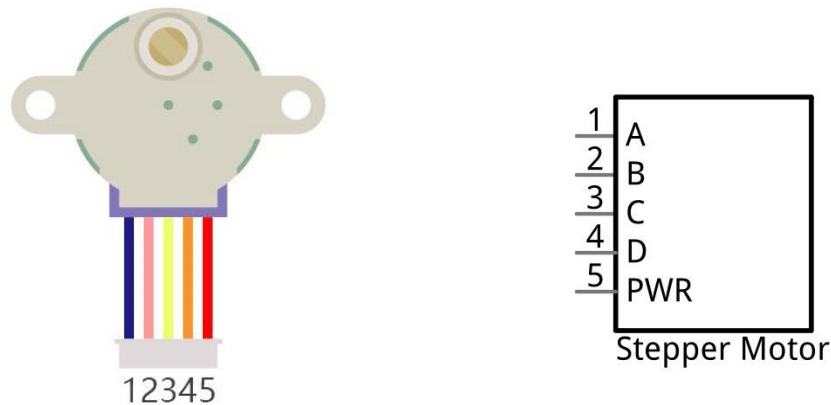
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Jumper Wire x12 
Stepper Motor x1 	ULN2003 Stepper Motor Driver x1 
9V battery (prepared by yourself) & battery line 	Breadboard Power module x1 

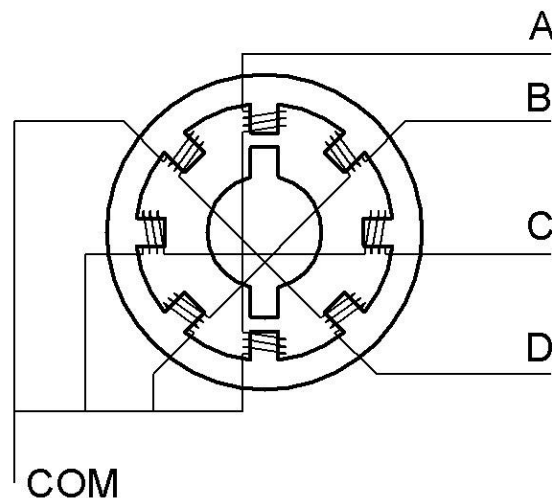
Component knowledge

Stepper Motor

Stepper Motors are an open-loop control device, which converts an electronic pulse signal into angular displacement or linear displacement. In a non-overload condition, the speed of the motor and the location of the stops depends only on the pulse signal frequency and number of pulses and is not affected by changes in load as with a DC Motor. A small Four-Phase Deceleration Stepper Motor is shown here:

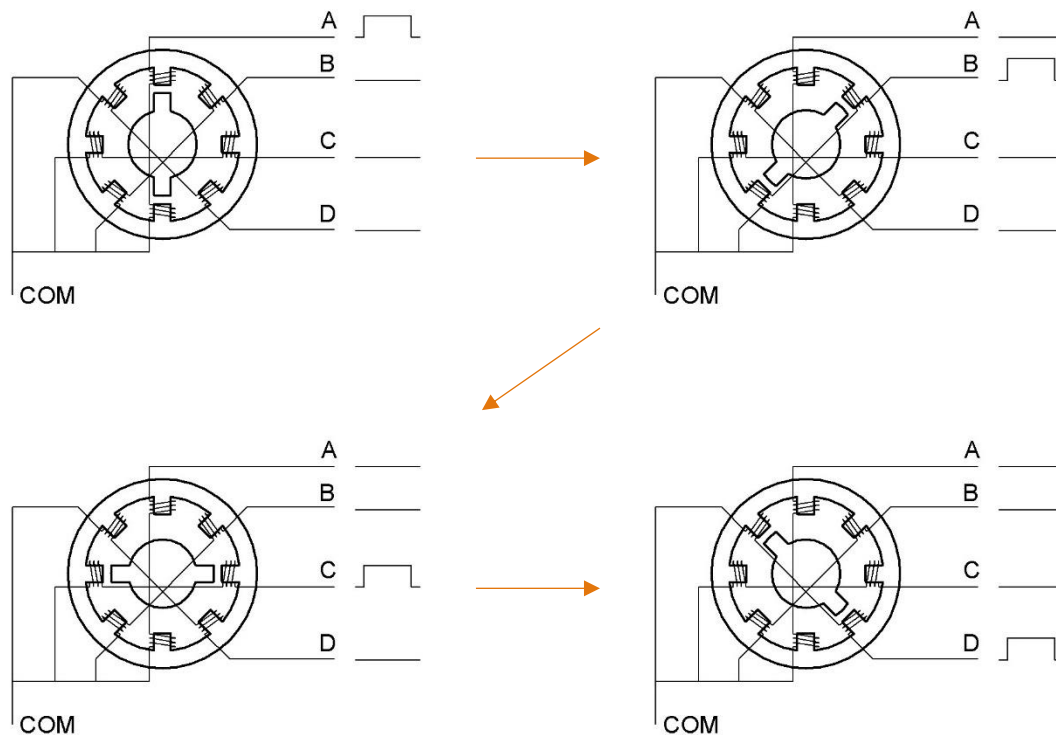


The electronic schematic diagram of a Four-Phase Stepper Motor is shown below:



The outside case or housing of the Stepper Motor is the Stator and inside the Stator is the Rotor. There is a specific number of individual coils, usually an integer multiple of the number of phases the motor has, when the Stator is powered ON, an electromagnetic field will be formed to attract a corresponding convex diagonal groove or indentation in the Rotor's surface. The Rotor is usually made of iron or a permanent magnet. Therefore, the Stepper Motor can be driven by powering the coils on the Stator in an ordered sequence (producing a series of "steps" or stepped movements).

A common driving sequence is shown here:



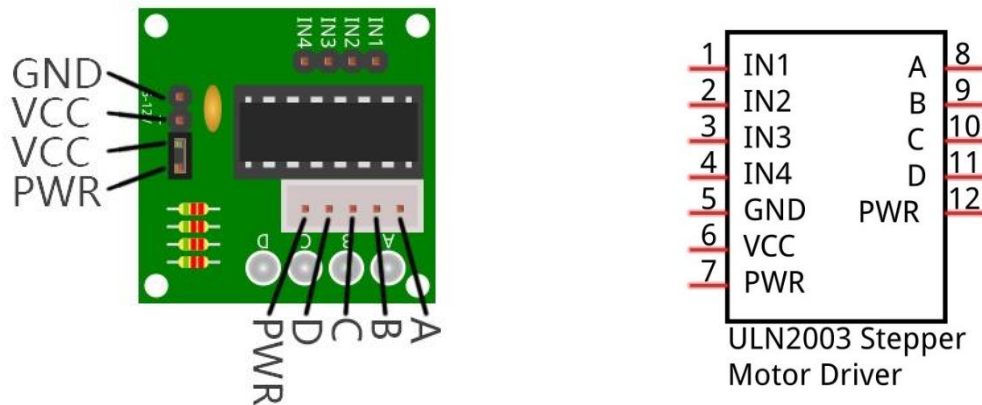
In the sequence above, the Stepper Motor rotates by a certain angle at once, which is called a “step”. By controlling the number of rotational steps, you can then control the Stepper Motor’s rotation angle. By defining the time between two steps, you can control the Stepper Motor’s rotation speed. When rotating clockwise, the order of coil powered on is: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A \rightarrow \dots$. And the rotor will rotate in accordance with this order, step by step, called four-steps, four-part. If the coils is powered ON in the reverse order, $D \rightarrow C \rightarrow B \rightarrow A \rightarrow D \rightarrow \dots$, the rotor will rotate in counter-clockwise direction.

There are other methods to control Stepper Motors, such as: connect A phase, then connect A B phase, the stator will be located in the center of A B, which is called a half-step. This method can improve the stability of the Stepper Motor and reduces noise. Tise sequence of powering the coils looks like this: $A \rightarrow AB \rightarrow B \rightarrow BC \rightarrow C \rightarrow CD \rightarrow D \rightarrow DA \rightarrow A \rightarrow \dots$, the rotor will rotate in accordance to this sequence ar, a half-step at a time, called four-steps, eight-part. Conversely, if the coils are powered ON in the reverse order the Stepper Motor will rotate in the opposite direction.

The stator in the Stepper Motor we have supplied has 32 magnetic poles. Therefore, to complete one full revolution requires 32 full steps. The rotor (or output shaft) of the Stepper Motor is connected to a speed reduction set of gears and the reduction ratio is 1:64. Therefore, the final output shaft (exiting the Stepper Motor’s housing) requires $32 \times 64 = 2048$ steps to make one full revolution.

ULN2003 Stepper Motor driver

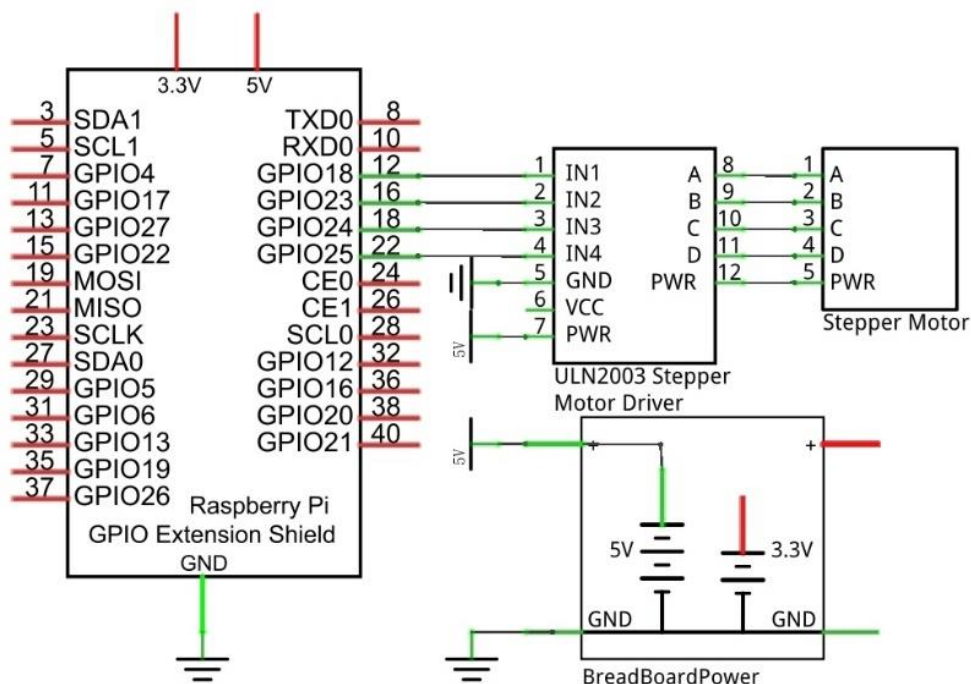
A ULN2003 Stepper Motor Driver is used to convert weak signals into more powerful control signals in order to drive the Stepper Motor. In the illustration below, the input signal IN1-IN4 corresponds to the output signal A-D, and 4 LEDs are integrated into the board to indicate the state of these signals. The PWR interface can be used as a power supply for the Stepper Motor. By default, PWR and VCC are connected.



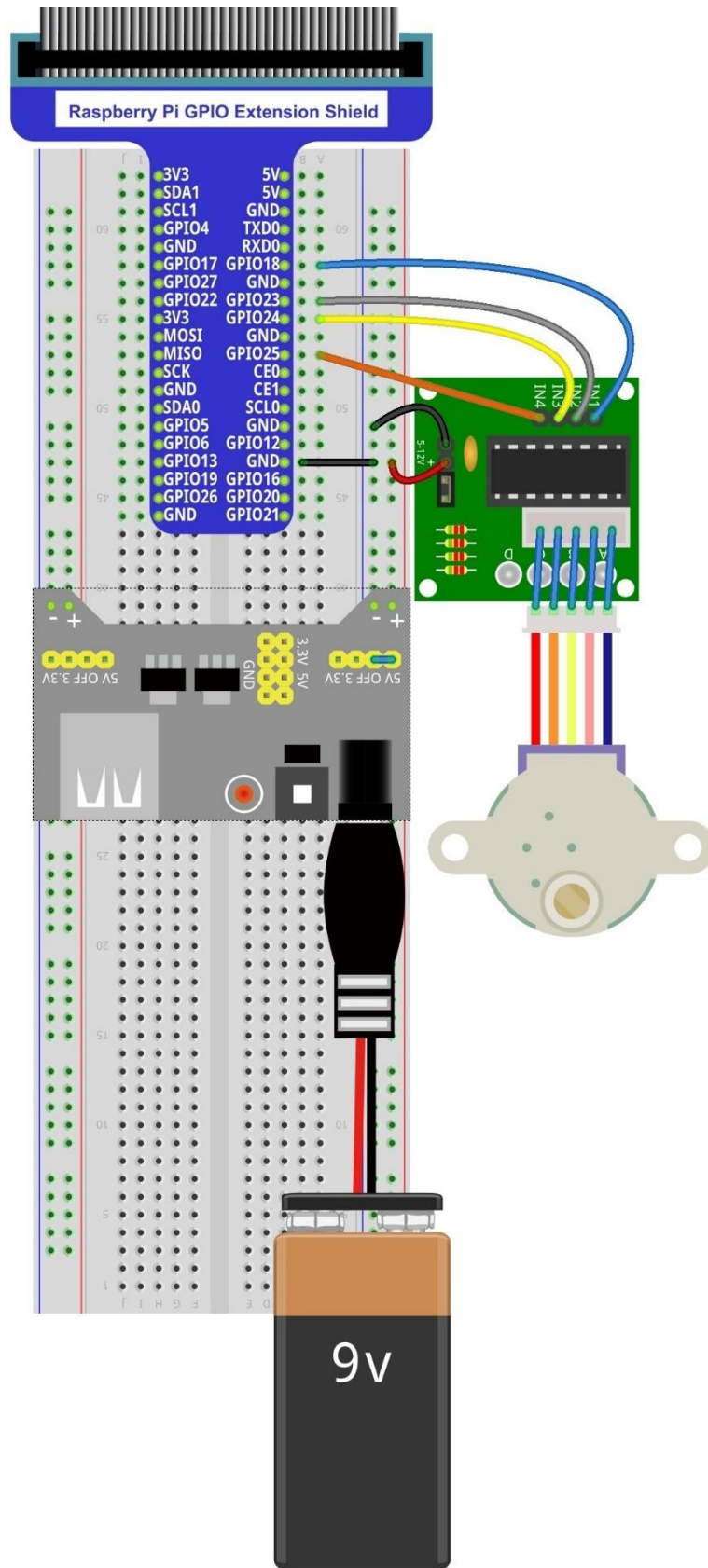
Circuit

When building the circuit, note that rated voltage of the Stepper Motor is 5V, and we need to use the breadboard power supply independently, (**Caution do not use the RPi power supply**). Additionally, the breadboard power supply needs to share Ground with Rpi.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: <https://youtu.be/g6R6ldqjQyw>

Code

Python Code 16.1.1 SteppingMotor

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 16.1.1_SteppingMotor directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless/16.1.1_SteppingMotor
```

2. Use Python command to execute code "SteppingMotor.py".

```
python SteppingMotor.py
```

After the program is executed, the Stepper Motor will rotate 360° clockwise and then 360° anticlockwise and repeat this action in an endless loop.

The following is the program code:

```

1  from gpiozero import OutputDevice
2  import time
3
4  motorPins = (18, 23, 24, 25) # define pins connected to four phase ABCD of stepper motor
5  # motorPins = ("J8:12", "J8:16", "J8:18", "J8:22") # define pins connected to four phase ABCD
  of stepper motor
6  motors = list(map(lambda pin: OutputDevice(pin), motorPins))
7  CCWStep = (0x01, 0x02, 0x04, 0x08) # define power supply order for rotating anticlockwise
8  CWStep = (0x08, 0x04, 0x02, 0x01) # define power supply order for rotating clockwise
9
10 # as for four phase stepping motor, four steps is a cycle. the function is used to drive the
  stepping motor clockwise or anticlockwise to take four steps
11 def moveOnePeriod(direction, ms):
12     for j in range(0, 4, 1): # cycle for power supply order
13         for i in range(0, 4, 1): # assign to each pin
14             if (direction == 1): # power supply order clockwise
15                 motors[i].on() if (CCWStep[j] == 1<<i) else motors[i].off()
16             else : # power supply order anticlockwise
17                 motors[i].on() if CWStep[j] == 1<<i else motors[i].off()
18             if(ms<3): # the delay can not be less than 3ms, otherwise it will exceed speed
  limit of the motor
19                 ms = 3
20                 time.sleep(ms*0.001)
21
22 # continuous rotation function, the parameter steps specifies the rotation cycles, every four
  steps is a cycle
23 def moveSteps(direction, ms, steps):
24     for i in range(steps):
25         moveOnePeriod(direction, ms)
26
27 # function used to stop motor

```

```

28 def motorStop():
29     for i in range(0,4,1):
30         motors.off()
31
32 def loop():
33     while True:
34         moveSteps(0,3,512) # rotating 360 deg clockwise, a total of 2048 steps in a circle,
512 cycles
35         time.sleep(0.5)
36         moveSteps(1,3,512) # rotating 360 deg anticlockwise
37         time.sleep(0.5)
38
39 if __name__ == '__main__': # Program entrance
40     print ('Program is starting...')
41     try:
42         loop()
43     except KeyboardInterrupt: # Press ctrl-c to end the program.
44         print("Ending program")

```

In the code we define the four pins of the Stepper Motor and the order to supply power to the coils for a four-step rotation mode.

```

motorPins = (18, 23, 24, 25) # define pins connected to four phase ABCD of stepper motor
# motorPins = ("J8:12", "J8:16", "J8:18", "J8:22") # define pins connected to four phase ABCD
of stepper motor
motors = list(map(lambda pin: OutputDevice(pin), motorPins))
CCWStep = (0x01, 0x02, 0x04, 0x08) # define power supply order for rotating anticlockwise
CWStep = (0x08, 0x04, 0x02, 0x01) # define power supply order for rotating clockwise

```

Subfunction **moveOnePeriod** (int dir, int ms) will drive the Stepper Motor rotating four-step clockwise or anticlockwise, four-step as a cycle. Where parameter "dir" indicates the rotation direction, if "dir" is 1, the servo will rotate clockwise, otherwise it rotates to anticlockwise. Parameter "ms" indicates the time between each two steps. The "ms" of Stepper Motor used in this project is 3ms (the shortest time period), a value of less than 3ms will exceed the limits of the Stepper Motor with a result that it does not rotate.

```

def moveOnePeriod(direction, ms):
    for j in range(0,4,1): # cycle for power supply order
        for i in range(0,4,1): # assign to each pin
            if (direction == 1):# power supply order clockwise
                motors[i].on() if (CCWStep[j] == 1<<i) else motors[i].off()
            else :
                # power supply order anticlockwise
                motors[i].on() if CWStep[j] == 1<<i else motors[i].off()
        if(ms<3): # the delay can not be less than 3ms, otherwise it will exceed speed
limit of the motor
            ms = 3
        time.sleep(ms*0.001)

```


Subfunction **moveSteps** (direction, ms, steps) is used to specify the cycle number of Stepper Motor.

```
def moveSteps(direction, ms, steps):  
    for i in range(steps):  
        moveOnePeriod(direction, ms)
```

Subfunction **motorStop** () is used to stop the Stepper Motor.

```
def motorStop():  
    for i in range(0, 4, 1):  
        motors.off()
```

Finally, in the while loop of main function, rotate one revolution clockwise, and then one revolution anticlockwise. According to the previous material covered, the Stepper Motor one revolution requires 2048 steps, that is, $2048/4=512$ cycle.

```
while True:  
    moveSteps(0, 3, 512) # rotating 360 deg clockwise, a total of 2048 steps in a circle,  
512 cycles  
    time.sleep(0.5)  
    moveSteps(1, 3, 512) # rotating 360 deg anticlockwise  
    time.sleep(0.5)
```

For more information about the methods used by the OutputDevice class in the GPIO Zero library, please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#outputdevice

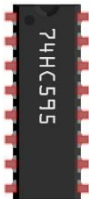
Chapter 17 74HC595 & Bar Graph LED

We have used LED Bar Graph to make a flowing water light, in which 10 GPIO ports of RPi are occupied. More GPIO ports mean that more peripherals can be connected to RPi, so GPIO resource is very precious. Can we make flowing water light with less GPIO ports? In this chapter, we will learn a component, 74HC595, which can achieve the target.

Project 17.1 Flowing Water Light

Now let us learn how to use the 74HC595 IC Chip to make a flowing water light using less GPIO.

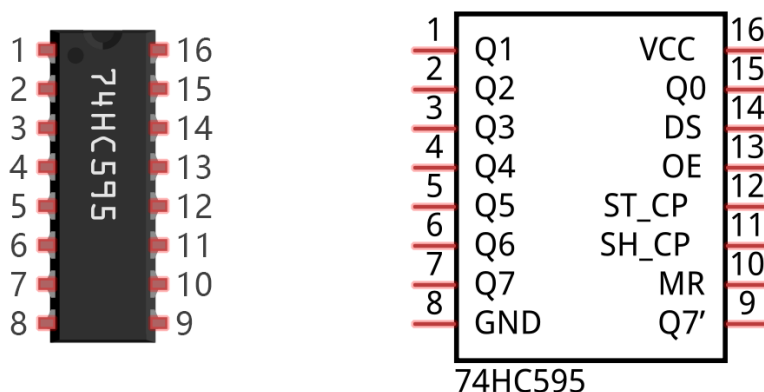
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1		Jumper x17 
74HC595 x1 	Bar Graph LED x1 	Resistor 220Ω x8 

Component knowledge

74HC595

A 74HC595 chip is used to convert serial data into parallel data. A 74HC595 chip can convert the serial data of one byte into 8 bits, and send its corresponding level to each of the 8 ports correspondingly. With this characteristic, the 74HC595 chip can be used to expand the IO ports of a Raspberry Pi. At least 3 ports on the RPI board are required to control the 8 ports of the 74HC595 chip.



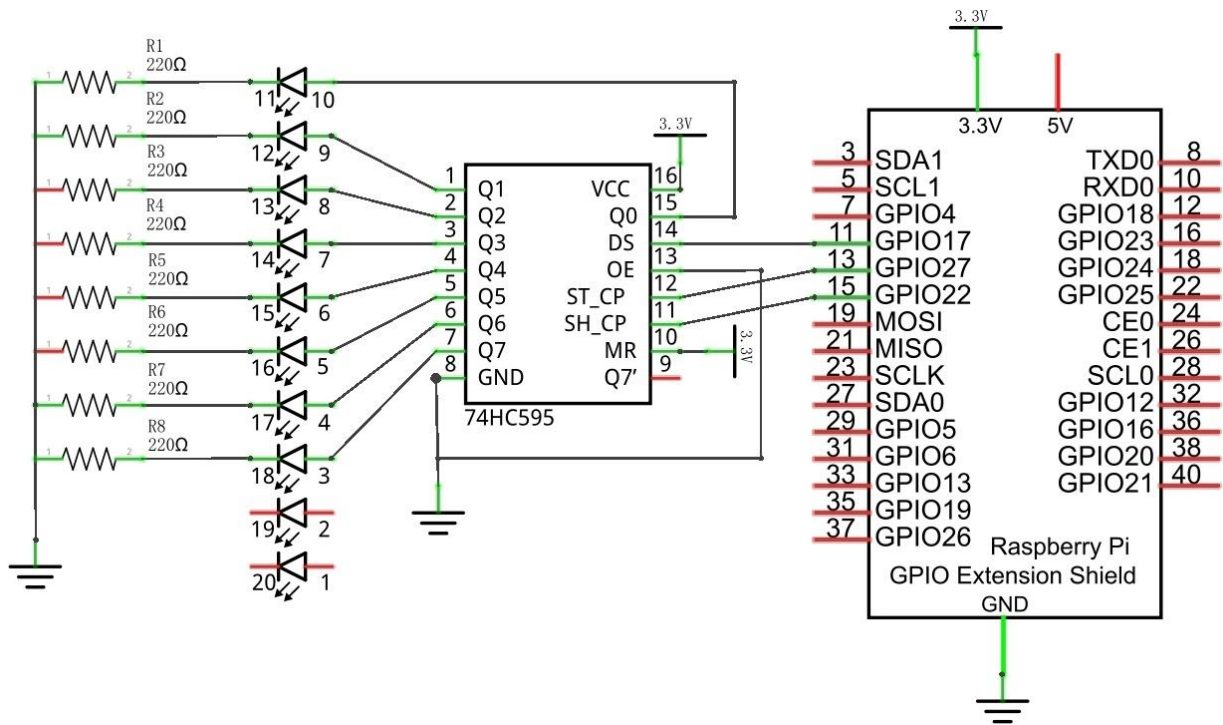
The ports of the 74HC595 chip are described as follows:

Pin name	Pin number	Description
Q0-Q7	15, 1-7	Parallel Data Output
VCC	16	The Positive Electrode of the Power Supply, the Voltage is 2~6V
GND	8	The Negative Electrode of Power Supply
DS	14	Serial Data Input
OE	13	Enable Output, When this pin is in high level, Q0-Q7 is in high resistance state When this pin is in low level, Q0-Q7 is in output mode
ST_CP	12	Parallel Update Output: when its electrical level is rising, it will update the parallel data output.
SH_CP	11	Serial Shift Clock: when its electrical level is rising, serial data input register will do a shift.
MR	10	Remove Shift Register: When this pin is in low level, the content in shift register will be cleared.
Q7'	9	Serial Data Output: it can be connected to more 74HC595 chips in series.

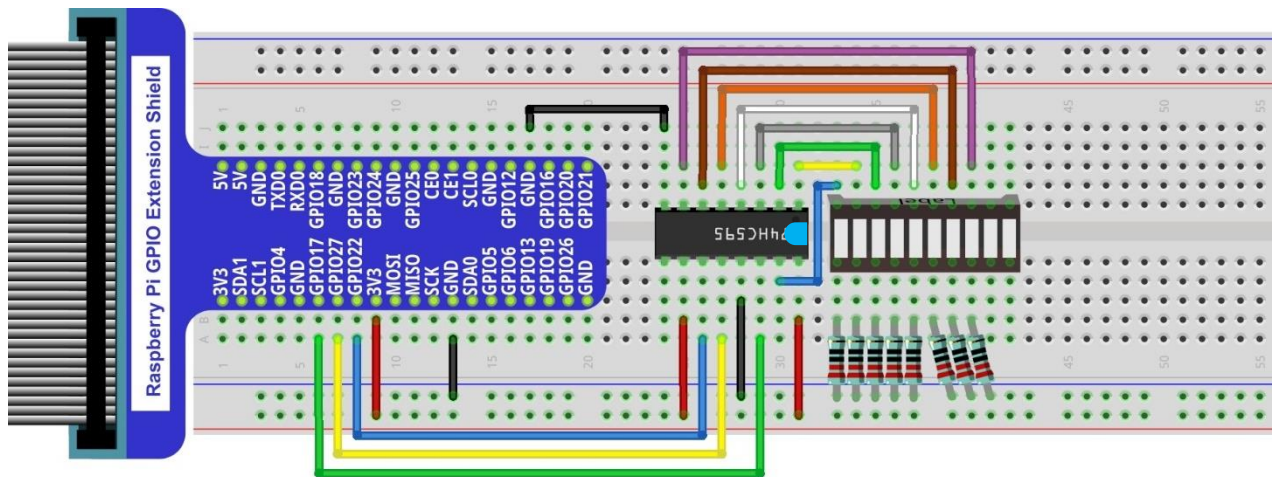
For more details, please refer to the datasheet on the 74HC595 chip.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: https://www.youtube.com/watch?v=e_SZUGKH2KU

Code

In this project we will make a flowing water light with a 74HC595 chip to learn about its functions.

Python Code 17.1.1 LightWater02

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 17.1.1_LightWater02 directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWater02/17.1.1_LightWater02
```

2. Use python command to execute Python code "LightWater02.py".

```
python LightWater02.py
```

After the program is executed, you will see that Bar Graph LED starts with the flowing water pattern flashing from left to right and then back from right to left.

The following is the program code:

```
1  from gpiozero import OutputDevice
2  import time
3  # Defines the data bit that is transmitted preferentially in the shiftOut function.
4  LSBFIRST = 1
5  MSBFIRST = 2
6  # define the pins for 74HC595
7  dataPin  = OutputDevice(17)      # DS Pin of 74HC595(Pin14)
8  latchPin = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
9  clockPin = OutputDevice(22)      # CH_CP Pin of 74HC595(Pin11)
10
11 # shiftOut function, use bit serial transmission.
12 def shiftOut(order, val):
13     for i in range(0, 8):
14         clockPin.off()
15         if(order == LSBFIRST):
16             dataPin.on() if (0x01 & (val >> i)) == 0x01 else dataPin.off()
17         elif(order == MSBFIRST):
18             dataPin.on() if (0x80 & (val << i)) == 0x80 else dataPin.off()
19         clockPin.on()
20
21 def loop():
22     while True:
23         x = 0x01
24         for i in range(0, 8):
25             latchPin.off() # Output low level to latchPin
26             shiftOut(LSBFIRST, x) # Send serial data to 74HC595
27             latchPin.on()    # Output high level to latchPin, and 74HC595 will update the data
28             to the parallel output port.
29             x <<= 1 # make the variable move one bit to left once, then the bright LED move one
30             step to the left once.
```

```

31         time.sleep(0.1)
32         x=0x80
33         for i in range(0,8):
34             latchPin.off()
35             shiftOut(LSBFIRST,x)
36             latchPin.on()
37             x>>=1
38             time.sleep(0.1)
39
40 def destroy():
41     dataPin.close()
42     latchPin.close()
43     clockPin.close()
44
45 if __name__ == '__main__': # Program entrance
46     print ('Program is starting..')
47     try:
48         loop()
49     except KeyboardInterrupt: # Press ctrl-c to end the program.
50         destroy()
51         print("Ending program")

```

Import the OutputDevice class that controls the 74HC595 chip from the gpiozero library.

```
from gpiozero import OutputDevice
```

Create the OutputDevice class for controlling the 74HC595 chip.

```

dataPin = OutputDevice(17)    # DS Pin of 74HC595 (Pin14)
latchPin = OutputDevice(27)   # ST_CP Pin of 74HC595 (Pin12)
clockPin = OutputDevice(22)   # CH_CP Pin of 74HC595 (Pin11)

```

In the code, we define a shiftOut() function, which is used to output values with bits in order, where the dPin for the data pin, cPin for the clock and order for the priority bit flag (high or low). This function conforms to the operational modes of the 74HC595. LSBFIRST and MSBFIRST are two different flow directions.

```

# shiftOut function, use bit serial transmission.
def shiftOut(order, val):
    for i in range(0,8):
        clockPin.off()
        if(order == LSBFIRST):
            dataPin.on() if (0x01&(val>>i)==0x01) else dataPin.off()
        elif(order == MSBFIRST):
            dataPin.on() if (0x80&(val<<i)==0x80) else dataPin.off()
        clockPin.on()

```

In the loop() function, we use two cycles to achieve the action goal. First, define a variable x=0x01, binary 00000001. When it is transferred to the output port of 74HC595, the low bit outputs high level, then an LED turns ON. Next, x is shifted one bit, when x is transferred to the output port of 74HC595 once again, the LED that turns ON will be shifted. Repeat the operation, over and over and the effect of a flowing water light will be visible. If the direction of the shift operation for x is different, the flowing direction is different.

```
def loop():
    while True:
        x=0x01
        for i in range(0,8):
            latchPin.off()# Output low level to latchPin
            shiftOut(LSBFIRST,x) # Send serial data to 74HC595
            latchPin.on() # Output high level to latchPin, and 74HC595 will update the data
to the parallel output port.
            x<<=1 # make the variable move one bit to left once, then the bright LED move one
step to the left once.
            time.sleep(0.1)
        x=0x80
        for i in range(0,8):
            latchPin.off()
            shiftOut(LSBFIRST,x)
            latchPin.on()
            x>>=1
            time.sleep(0.1)
```

For more information about the methods used by the OutputDevice class in the GPIO Zero library, please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#outputdevice

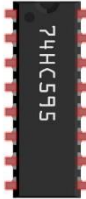
Chapter 18 74HC595 & 7-Segment Display

In this chapter, we will introduce the 7-Segment Display.

Project 18.1 7-Segment Display

We will use a 74HC595 IC Chip to control a 7-Segment Display and make it display sixteen decimal characters "0" to "F".

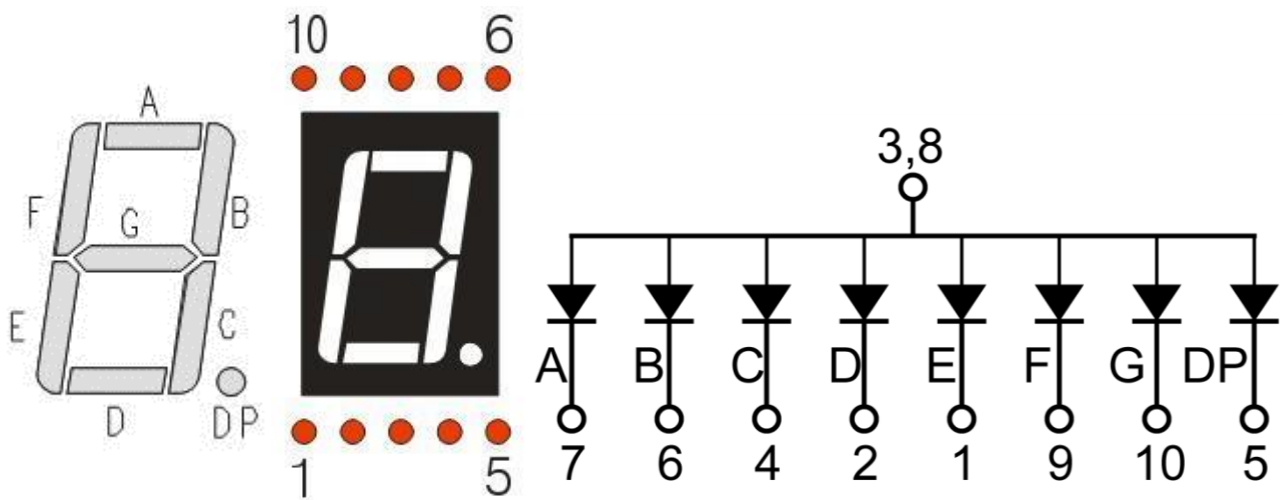
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1		Jumper Wire x18 
74HC595 x1 	7-Segment Display x1 	Resistor 220Ω x8 

Component knowledge

7-segment display

A 7-Segment Display is a digital electronic display device. There is a figure "8" and a decimal point represented, which consists of 8 LEDs. The LEDs have a Common Anode and individual Cathodes. Its internal structure and pin designation diagram is shown below:



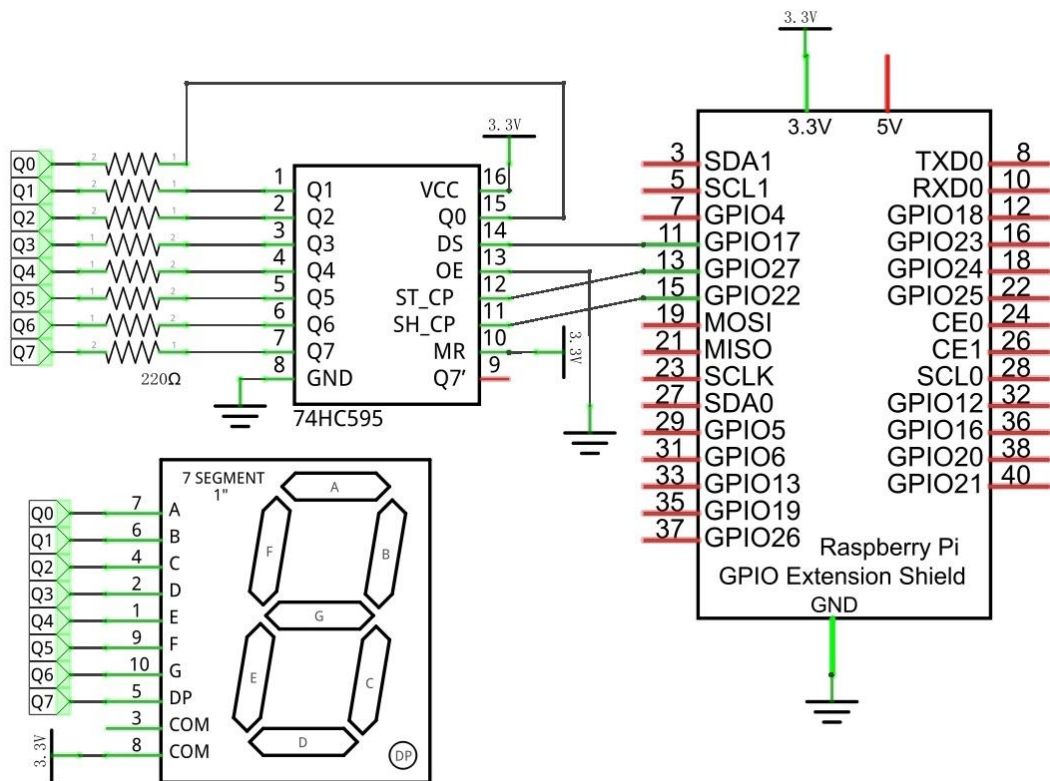
As we can see in the above circuit diagram, we can control the state of each LED separately. Also, by combining LEDs with different states of ON and OFF, we can display different characters (Numbers and Letters). For example, to display a "0": we need to turn ON LED segments A, B, C, D, E and F, and turn OFF LED segments G and DP.



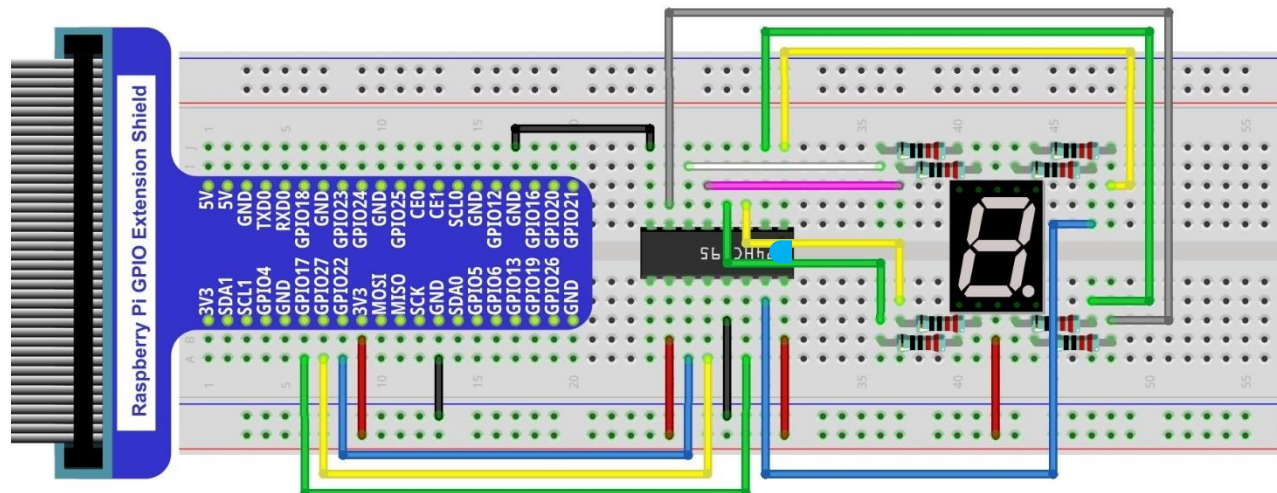
In this project, we will use a 7-Segment Display with a Common Anode. Therefore, when there is an input low level to an LED segment the LED will turn ON. Defining segment "A" as the lowest level and segment "DP" as the highest level, from high to low would look like this: "DP", "G", "F", "E", "D", "C", "B", "A". Character "0" corresponds to the code: 1100 0000b=0xc0.

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: <https://youtu.be/KSE0LdyuOFM>

Code

This code uses a 74HC595 IC Chip to control the 7-Segment Display. The use of the 74HC595 IC Chip is generally the same throughout this Tutorial. We need code to display the characters “0” to “F” one character at a time, and then output to display them with the 74HC595 IC Chip.

Python Code 18.1.1 SevenSegmentDisplay

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 18.1.1_SevenSegmentDisplay directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless/18.1.1_SevenSegmentDisplay
```

2. Use Python command to execute Python code “SevenSegmentDisplay.py”.

```
python SevenSegmentDisplay.py
```

After the program is executed, the 7-Segment Display starts to display the characters “0” to “F” in succession. The following is the program code:

```

1  from gpiozero import OutputDevice
2  import time
3
4  LSBFIRST = 1
5  MSBFIRST = 2
6  # define the pins for 74HC595
7  dataPin  = OutputDevice(17)      # DS Pin of 74HC595(Pin14)
8  latchPin = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
9  clockPin = OutputDevice(22)      # CH_CP Pin of 74HC595(Pin11)
10 # SevenSegmentDisplay display the character "0"- "F" successively
11 num = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
12
13 def shiftOut(order, val):
14     for i in range(0, 8):
15         clockPin.off()
16         if(order == LSBFIRST):
17             dataPin.on() if (0x01 & (val >> i)) == 0x01 else dataPin.off()
18         elif(order == MSBFIRST):
19             dataPin.on() if (0x80 & (val << i)) == 0x80 else dataPin.off()
20         clockPin.on()
21
22 def loop():
23     while True:
24         for i in range(0, len(num)):
25             latchPin.off()
26             shiftOut(MSBFIRST, num[i]) # Send serial data to 74HC595
27             latchPin.on()
28             time.sleep(0.5)
29         for i in range(0, len(num)):
```

```

30         latchPin.off()
31         shiftOut(MSBFIRST, num[i]&0x7f) # Use "&0x7f" to display the decimal point.
32         latchPin.on()
33         time.sleep(0.5)
34
35     def destroy():
36         dataPin.close()
37         latchPin.close()
38         clockPin.close()
39
40     if __name__ == '__main__': # Program entrance
41         print ('Program is starting...')
42         try:
43             loop()
44         except KeyboardInterrupt: # Press ctrl-c to end the program.
45             destroy()
46         print("Ending program")

```

First, we need to create encoding for characters "0" to "F" in the array.

```
num = [0xc0, 0xf9, 0xa4, 0xb0, 0x99, 0x92, 0x82, 0xf8, 0x80, 0x90, 0x88, 0x83, 0xc6, 0xa1, 0x86, 0x8e]
```

In the "for" loop of loop() function, use the 74HC595 IC Chip to output contents of array "num" successively. SevenSegmentDisplay can then correctly display the corresponding characters. Pay attention to this in regard to shiftOut function, the transmission bit, flag bit and highest bit will be transmitted preferentially.

```

for i in range(0, len(num)):
    latchPin.off()
    shiftOut(MSBFIRST, num[i]) #Output the figures and the highest level is transfered
preferentially.
    latchPin.on()
    time.sleep(0.5)

```

If you want to display the decimal point, make the highest bit of each array "0", which can be implemented easily by num[i]&0x7f.

```
shiftOut(MSBFIRST, num[i]&0x7f) # Use "&0x7f" to display the decimal point.
```

For more information about the methods used by the OutputDevice class in the GPIO Zero library, please refer to: https://gpiozero.readthedocs.io/en/stable/api_output.html#outputdevice

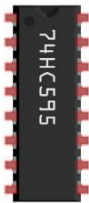
Chapter 19 74HC595 & LED Matrix

Thus far we have learned how to use the 74HC595 IC Chip to control the Bar Graph LED and the 7-Segment Display. We will now use 74HC595 IC Chips to control an LED Matrix.

Project 19.1 LED Matrix

In this project, we will use two 74HC595 IC chips to control a monochrome (one color) (8X8) LED Matrix to make it display both simple graphics and characters.

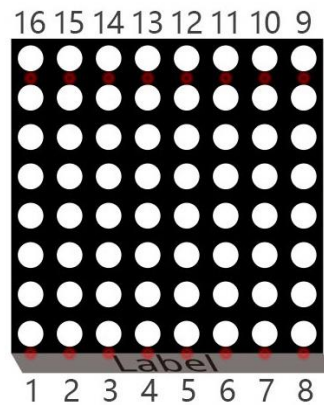
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1		Jumper x36 
74HC595 x2 	8X8 LEDMatrix x1 	Resistor 220Ω x8 

Component knowledge

LED matrix

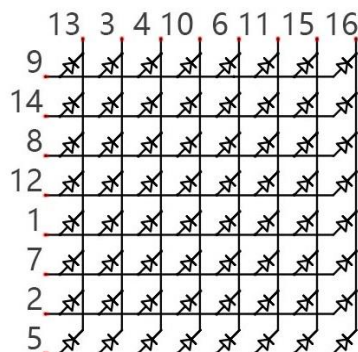
An LED Matrix is a rectangular display module that consists of a uniform grid of LEDs. The following is an 8X8 monochrome (one color) LED Matrix containing 64 LEDs (8 rows by 8 columns).



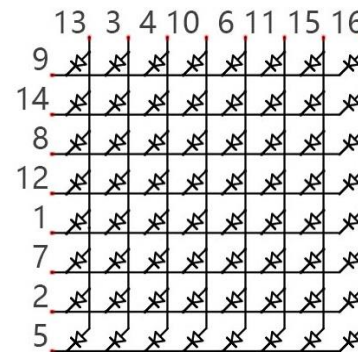
In order to facilitate the operation and reduce the number of ports required to drive this component, the Positive Poles of the LEDs in each row and Negative Poles of the LEDs in each column are respectively connected together inside the LED Matrix module, which is called a Common Anode. There is another arrangement type. Negative Poles of the LEDs in each row and the Positive Poles of the LEDs in each column are respectively connected together, which is called a Common Cathode.

The LED Matrix that we use in this project is a Common Anode LED Matrix.

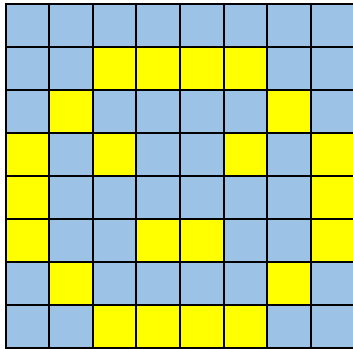
Connection mode of Common Anode



Connection mode of Common Cathode



Here is how a Common Anode LED Matrix works. First, choose 16 ports on RPI board to connect to the 16 ports of LED Matrix. Configure one port in columns for low level, which makes that column the selected port. Then configure the eight port in the row to display content in the selected column. Add a delay value and then select the next column that outputs the corresponding content. This kind of operation by column is called Scan. If you want to display the following image of a smiling face, you can display it in 8 columns, and each column is represented by one byte.



1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0
0	0	1	1	1	1	0	0
0	1	0	0	0	0	1	0
1	0	1	0	0	1	0	1
1	0	0	0	0	0	0	1
1	0	0	1	1	0	0	1
0	1	0	0	0	0	1	0
0	0	1	1	1	1	0	0

Column	Binary	Hexadecimal
1	0001 1100	0x1c
2	0010 0010	0x22
3	0101 0001	0x51
4	0100 0101	0x45
5	0100 0101	0x45
6	0101 0001	0x51
7	0010 0010	0x22
8	0001 1100	0x1c

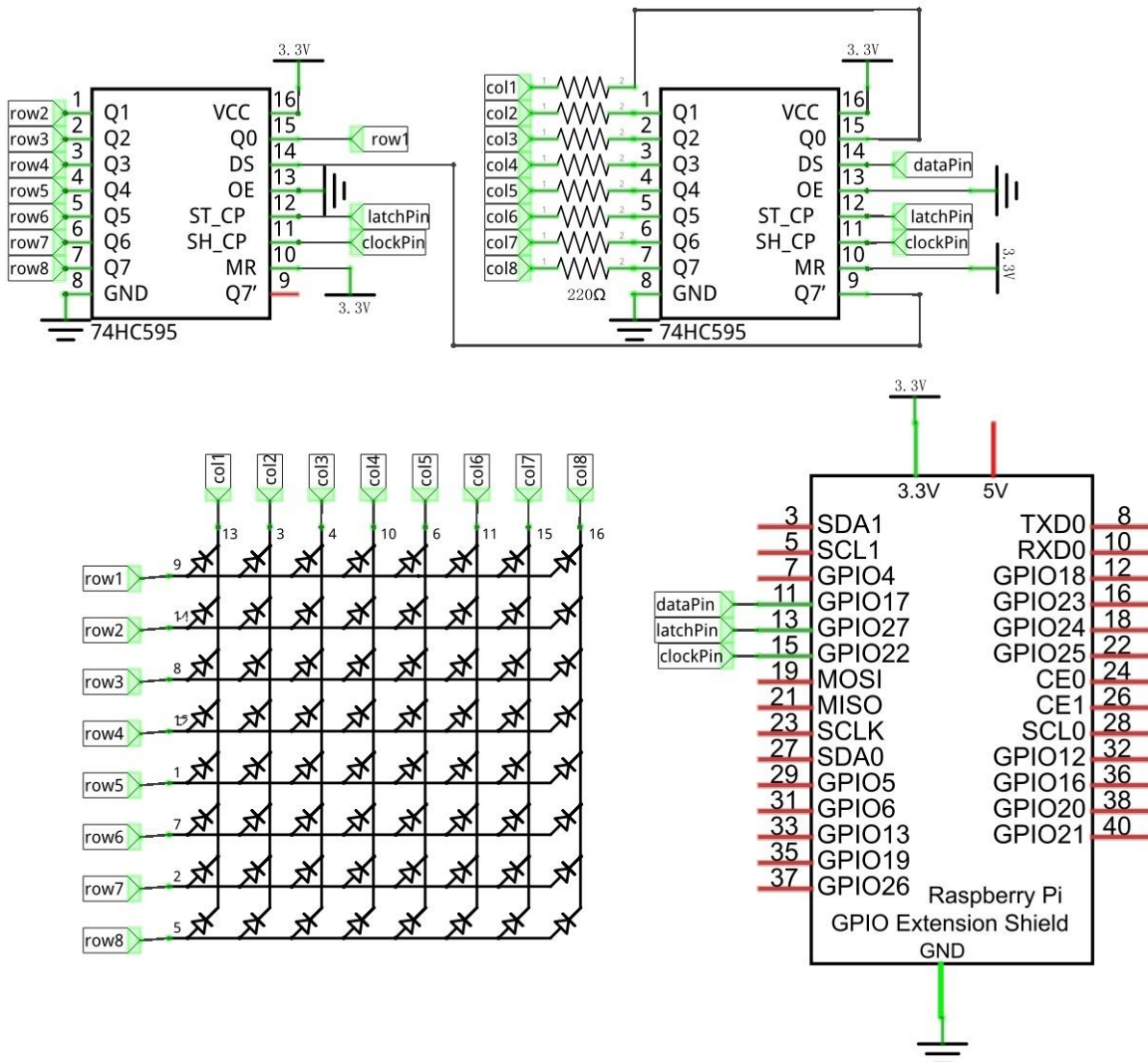
To begin, display the first column, then turn off the first column and display the second column. (and so on) turn off the seventh column and display the 8th column, and then start the process over from the first column again like the control of LED Bar Graph project. The whole process will be repeated rapidly in a loop. Due to the principle of optical afterglow effect and the vision persistence effect in human sight, we will see a picture of a smiling face directly rather than individual columns of LEDs turned ON one column at a time (although in fact this is the reality we cannot perceive).

Scanning rows is another option to display on an LED Matrix (dot matrix grid). Whether scanning by row or column, 16 GPIO is required. In order to save GPIO ports of control board, two 74HC595 IC Chips are used in the circuit. Every 74HC595 IC Chip has eight parallel output ports, so two of these have a combined total of 16 ports, which is just enough for our project. The control lines and data lines of the two 74HC595 IC Chips are not all connected to the RPi, but connect to the Q7 pin of first stage 74HC595 IC Chip and to the data pin of second IC Chip. The two 74HC595 IC Chips are connected in series, which is the same as using one "74HC595 IC Chip" with 16 parallel output ports.

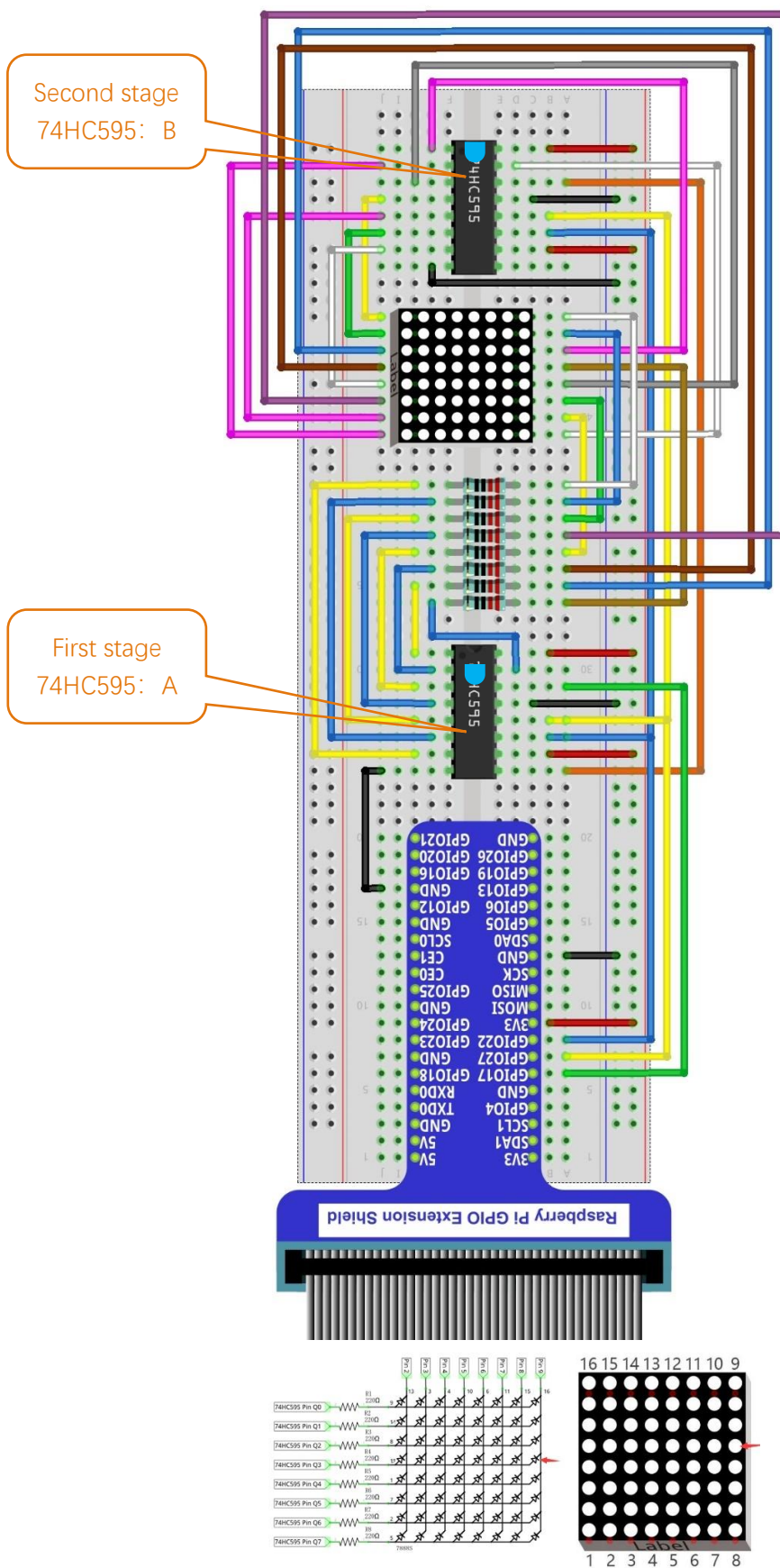
Circuit

In circuit of this project, the power pin of the 74HC595 IC Chip is connected to 3.3V. It can also be connected to 5V to make LED Matrix brighter.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: <https://youtu.be/KSE0LdyuOFM>

Code

Two 74HC595 IC Chips are used in this project, one for controlling the LED Matrix's columns and the other for controlling the rows. According to the circuit connection, row data should be sent first, then column data. The following code will make the LED Matrix display a smiling face, and then display characters "0 to F" scrolling in a loop on the LED Matrix.

Python Code 19.1.1 LEDMatrix

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 19.1.1_LEDMatrix directory of Python language.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless/19.1.1_LEDMatrix
```

2. Use Python command to execute Python code "LEDMatrix.py".

```
python LEDMatrix.py
```

After the program is executed, the LED Matrix display a smiling face, and then display characters "0 to F" scrolling in a loop on the LED Matrix.

The following is the program code:

```
1  from gpiozero import OutputDevice
2  import time
3
4  LSBFIRST = 1
5  MSBFIRST = 2
6  dataPin  = OutputDevice(17)      # DS Pin of 74HC595(Pin14)
7  latchPin = OutputDevice(27)      # ST_CP Pin of 74HC595(Pin12)
8  clockPin = OutputDevice(22)      # CH_CP Pin of 74HC595(Pin11)
9  pic = [0x1c, 0x22, 0x51, 0x45, 0x45, 0x51, 0x22, 0x1c] # data of smiling face
10 data = [      # data of "0-F"
11     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, # " "
12     0x00, 0x00, 0x3E, 0x41, 0x41, 0x3E, 0x00, 0x00, # "0"
13     0x00, 0x00, 0x21, 0x7F, 0x01, 0x00, 0x00, 0x00, # "1"
14     0x00, 0x00, 0x23, 0x45, 0x49, 0x31, 0x00, 0x00, # "2"
15     0x00, 0x00, 0x22, 0x49, 0x49, 0x36, 0x00, 0x00, # "3"
16     0x00, 0x00, 0x0E, 0x32, 0x7F, 0x02, 0x00, 0x00, # "4"
17     0x00, 0x00, 0x79, 0x49, 0x49, 0x46, 0x00, 0x00, # "5"
18     0x00, 0x00, 0x3E, 0x49, 0x49, 0x26, 0x00, 0x00, # "6"
19     0x00, 0x00, 0x60, 0x47, 0x48, 0x70, 0x00, 0x00, # "7"
20     0x00, 0x00, 0x36, 0x49, 0x49, 0x36, 0x00, 0x00, # "8"
21     0x00, 0x00, 0x32, 0x49, 0x49, 0x3E, 0x00, 0x00, # "9"
22     0x00, 0x00, 0x3F, 0x44, 0x44, 0x3F, 0x00, 0x00, # "A"
23     0x00, 0x00, 0x7F, 0x49, 0x49, 0x36, 0x00, 0x00, # "B"
24     0x00, 0x00, 0x3E, 0x41, 0x41, 0x22, 0x00, 0x00, # "C"
25     0x00, 0x00, 0x7F, 0x41, 0x41, 0x3E, 0x00, 0x00, # "D"
26     0x00, 0x00, 0x7F, 0x49, 0x49, 0x41, 0x00, 0x00, # "E"
27     0x00, 0x00, 0x7F, 0x48, 0x48, 0x40, 0x00, 0x00, # "F"
```

```

28         0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, # " "
29     ]
30
31     def shiftOut(order, val):
32         for i in range(0, 8):
33             clockPin.off()
34             if (order == LSBFIRST):
35                 dataPin.on() if (0x01 & (val >> i) == 0x01) else dataPin.off()
36             elif (order == MSBFIRST):
37                 dataPin.on() if (0x80 & (val << i) == 0x80) else dataPin.off()
38             clockPin.on()
39
40     def loop():
41         while True:
42             for j in range(0, 500): # Repeat enough times to display the smiling face a period of
time
43                 x = 0x80
44                 for i in range(0, 8):
45                     latchPin.off()
46                     shiftOut(MSBFIRST, pic[i]) #first shift data of line information to first stage
74HC959
47                     shiftOut(MSBFIRST, ~x) #then shift data of column information to second stage
74HC959
48                     latchPin.on() # Output data of two stage 74HC955 at the same time
time.sleep(0.001) # display the next column
49                     x >>= 1
50                 for k in range(0, len(data)-8): #len(data) total number of "0-F" columns
51                     for j in range(0, 20): # times of repeated displaying LEDMatrix in every frame, the
bigger the "j", the longer the display time.
52                         x = 0x80 # Set the column information to start from the first column
53                         for i in range(k, k+8):
54                             latchPin.off()
55                             shiftOut(MSBFIRST, data[i])
56                             shiftOut(MSBFIRST, ~x)
57                             latchPin.on()
58                             time.sleep(0.001)
59                             x >>= 1
60
61     def destroy():
62         dataPin.close()
63         latchPin.close()
64         clockPin.close()
65
66 if __name__ == '__main__': # Program entrance
67     print('Program is starting...')
68     try:

```

```

65     loop()
66     except KeyboardInterrupt:  # Press ctrl-c to end the program.
67         destroy()
68         print("Ending program")

```

The first “for” loop in the “while” loop is used to display a static smile. Displaying column information from left to right, one column at a time with a total of 8 columns. This repeats 500 times to ensure sufficient display time.

```

    for j in range(0,500): # Repeat enough times to display the smiling face a period of
time
        x=0x80
        for i in range(0,8):
            latchPin.off()
            shiftOut(MSBFIRST,pic[i]) #first shift data of line information to first stage
74HC959
            shiftOut(MSBFIRST,~x) #then shift data of column information to second stage
74HC959

            latchPin.on()# Output data of two stage 74HC595 at the same time
            time.sleep(0.001) # display the next column
        x>>=1

```

The second “for” loop is used to display scrolling characters “0 to F”, for a total of $18 \times 8 = 144$ columns. Displaying the 0-8 column, then the 1-9 column, then the 2-10 column..... and so on...138-144 column in consecutively to achieve the scrolling effect. The display of each frame is repeated a certain number of times and the more repetitions, the longer the single frame display will be and the slower the scrolling movement.

```

    for k in range(0,len(data)-8): #len(data) total number of "0-F" columns
        for j in range(0,20): # times of repeated displaying LEDMatrix in every frame, the
bigger the "j", the longer the display time.
            x=0x80          # Set the column information to start from the first column
            for i in range(k,k+8):
                latchPin.off()
                shiftOut(MSBFIRST,data[i])
                shiftOut(MSBFIRST,~x)
                latchPin.on()
                time.sleep(0.001)
            x>>=1

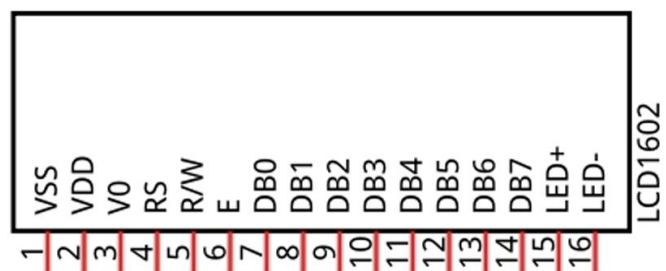
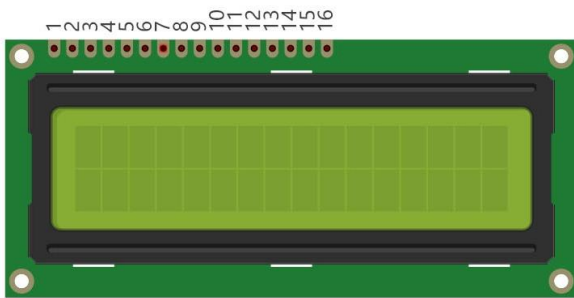
```

Chapter 20 LCD1602

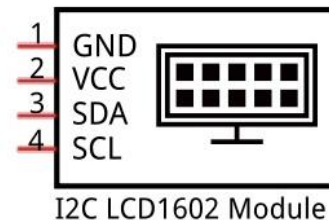
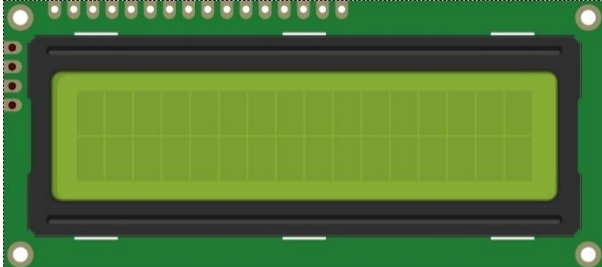
In this chapter, we will learn about the LCD1602 Display Screen,

Project 20.1 I2C LCD1602

There are LCD1602 display screen and the I2C LCD. We will introduce both of them in this chapter. But what we use in this project is an I2C LCD1602 display screen. The LCD1602 Display Screen can display 2 lines of characters in 16 columns. It is capable of displaying numbers, letters, symbols, ASCII code and so on. As shown below is a monochrome LCD1602 Display Screen along with its circuit pin diagram

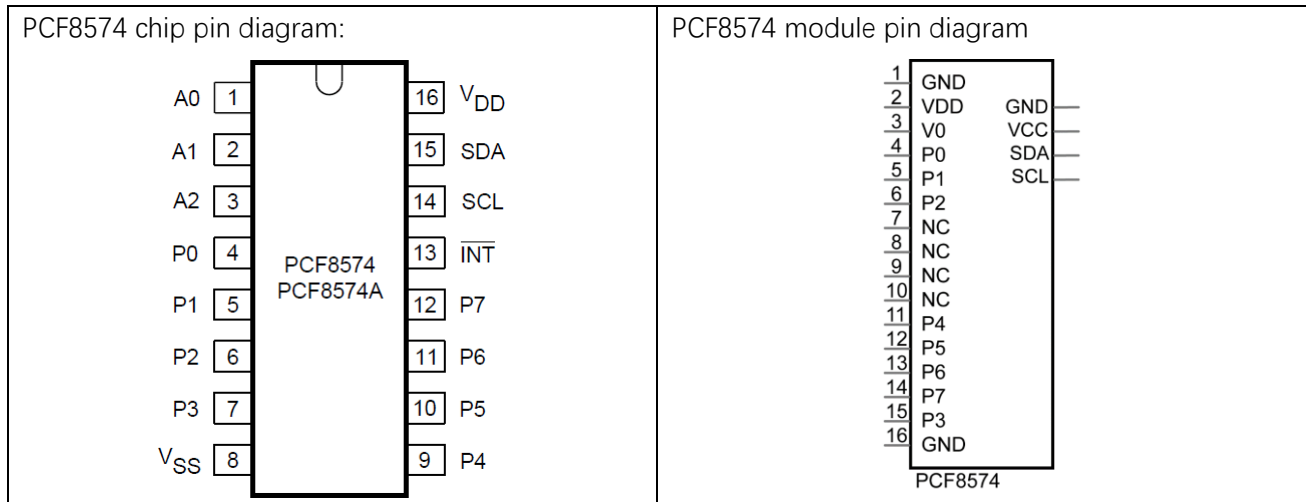


I2C LCD1602 Display Screen integrates a I2C interface, which connects the serial-input & parallel-output module to the LCD1602 Display Screen. This allows us to only use 4 lines to operate the LCD1602.

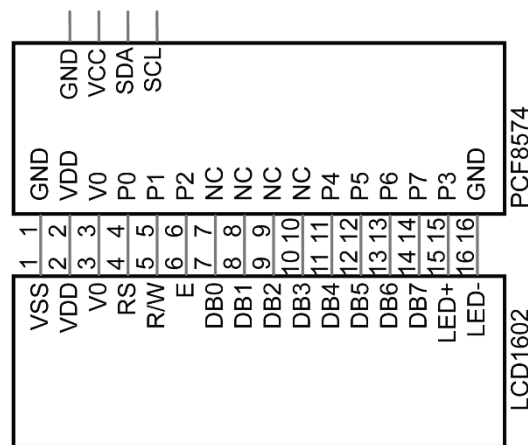


The serial-to-parallel IC chip used in this module is PCF8574T (PCF8574AT), and its default I2C address is 0x27(0x3F). You can also view the RPI bus on your I2C device address through command "i2cdetect -y 1" (refer to the "configuration I2C" section below).

Below is the PCF8574 chip pin diagram and its module pin diagram:



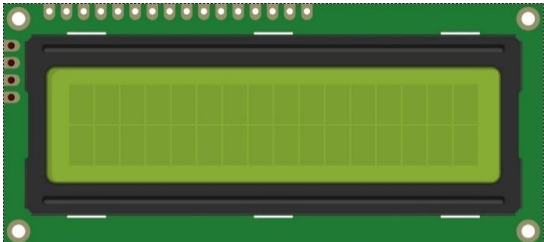
PCF8574 module pins and LCD1602 pins correspond to each other and connected to each other:



Because of this, as stated earlier, we only need 4 pins to control the 16 pins of the LCD1602 Display Screen through the I2C interface.

In this project, we will use the I2C LCD1602 to display some static characters and dynamic variables.

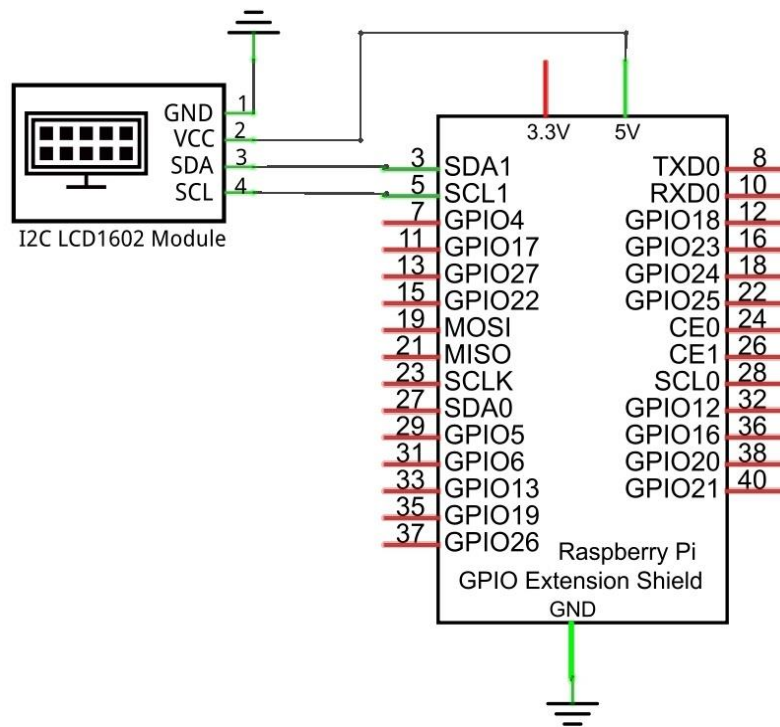
Component List

Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	Jumper Wire x4 
I2C LCD1602 Module x1 	

Circuit

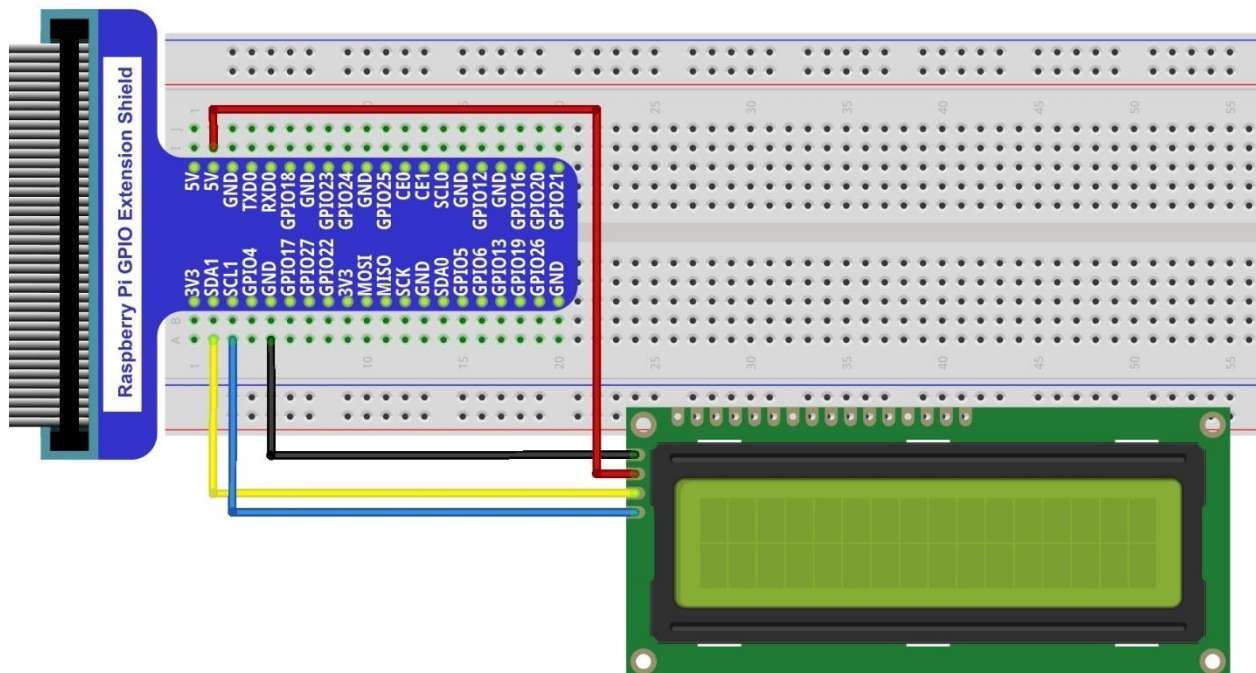
Note that the power supply for I2C LCD1602 in this circuit is 5V.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com

NOTE: It is necessary to configure 12C and install Smbus first (see [chapter 7](#) for details)



Video: <https://www.youtube.com/watch?v=L1bgX2f4Yio>

Code

This code will have your RPi's CPU temperature and System Time Displayed on the LCD1602.

Python Code 20.1.1 I2CLCD1602

If you did not [configure I2C and install Smbus](#), please refer to [Chapter 7](#). If you did, continue.

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 20.1.1_ I2CLCD1602 directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWZero_Code/20.1.1_I2CLCD1602
```

2. Use Python command to execute Python code "I2CLCD1602.py".

```
python I2CLCD1602.py
```

After the program is executed, the LCD1602 Screen will display your RPi's CPU Temperature and System Time. So far, at this writing, we have two types of LCD1602 on sale. One needs to adjust the backlight, and the other does not.

The LCD1602 that does not need to adjust the backlight is shown in the figure below.



The following is the program code:

```
1  import smbus
2  from time import sleep, strftime
3  from datetime import datetime
4  from LCD1602 import CharLCD1602
5
6  lcd1602 = CharLCD1602()
7  def get_cpu_temp():      # get CPU temperature from file
8      "/sys/class/thermal/thermal_zone0/temp"
9      tmp = open('/sys/class/thermal/thermal_zone0/temp')
10     cpu = tmp.read()
11     tmp.close()
12     return '{:.2f}'.format( float(cpu)/1000 ) + ' C '
13
14  def get_time_now():      # get system time
15     return datetime.now().strftime(' %H:%M:%S')
16
17  def loop():
18     lcd1602.init_lcd()
19     count = 0
```

```

20     while(True):
21         lcd1602.clear()
22         lcd1602.write(0, 0, 'CPU: ' + get_cpu_temp() )# display CPU temperature
23         lcd1602.write(0, 1, get_time_now() )    # display the time
24         sleep(1)
25     def destroy():
26         lcd1602.clear()
27     if __name__ == '__main__':
28         print ('Program is starting ... ')
29         try:
30             loop()
31         except KeyboardInterrupt:
32             destroy()

```

In a while loop, set the cursor position, and display the CPU temperature and time.

```

while(True):
    lcd1602.clear()
    lcd1602.write(0, 0, 'CPU: ' + get_cpu_temp() )# display CPU temperature
    lcd1602.write(0, 1, get_time_now() )    # display the time
    sleep(1)

```

CPU temperature is stored in file “/sys/class/thermal/thermal_zone0/temp”. Open the file and read content of the file, and then convert it to Celsius degrees and return. Subfunction used to get CPU temperature is shown below:

```

def get_cpu_temp():    # get CPU temperature and store it into file
    “/sys/class/thermal/thermal_zone0/temp”
    tmp = open('/sys/class/thermal/thermal_zone0/temp')
    cpu = tmp.read()
    tmp.close()
    return '{:.2f}'.format( float(cpu)/1000 ) + ' C'

```

Subfunction used to get time:

```

def get_time_now():    # get the time
    return datetime.now().strftime('%H:%M:%S')

```

Details about LCD1602.py:

Module LCD1602

This module provides the basic operation method of LCD1602, including class CharLCD1602.

Some member functions are described as follows:

def init_lcd(self,addr=None, bl=1) : LDC1602 initializes the setting. When the addr is None, the I2C address of the device will be automatically scanned. You can also specify the I2C address, bl=1 to enable the backlight setting.

def clear(self): clear the screen

def send_command(self,comm): set the cursor position

def i2c_scan(self): scan the device I2C address

def write(self,x, y, str): display contents

More information can be viewed through opening LCD1602.py.

Chapter 21 Ultrasonic Ranging

In this chapter, we learn a module which use ultrasonic to measure distance.

Project 21.1 Ultrasonic Ranging

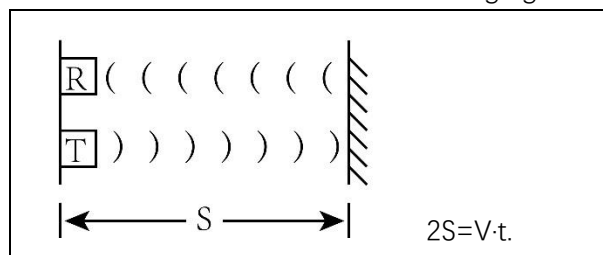
In this project, we use ultrasonic ranging module to measure distance, and print out the data in the terminal.

Component List

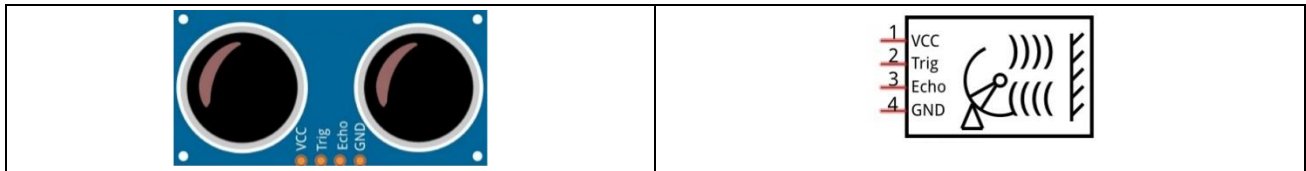
Raspberry Pi (with 40 GPIO) x1 GPIO Expansion Board & Ribbon Cable x1 Breadboard x1	Ultrasonic Module x1 
Jumper Wire x4 	Resistor 1kΩ x3 

Component Knowledge

The Ultrasonic Ranging Module uses the principle that ultrasonic waves will be reflected when they encounter any obstacles. This is possible by counting the time interval between when the ultrasonic wave is transmitted to when the ultrasonic wave reflects back after encountering an obstacle. Time interval counting will end after an ultrasonic wave is received, and the time difference (delta) is the total time of the ultrasonic wave's journey from being transmitted to being received. Because the speed of sound in air is a constant, and is about $v=340\text{m/s}$, we can calculate the distance between the Ultrasonic Ranging Module and the obstacle: $s=vt/2$.



The Ultrasonic Ranging Module integrates a both an ultrasonic transmitter and a receiver. The transmitter is used to convert electrical signals (electrical energy) into high frequency (beyond human hearing) sound waves (mechanical energy) and the function of the receiver is opposite of this. The picture and the diagram of the Ultrasonic Ranging Module are shown below:



Pin description:

VCC	power supply pin
Trig	trigger pin
Echo	Echo pin
GND	GND

Technical specs:

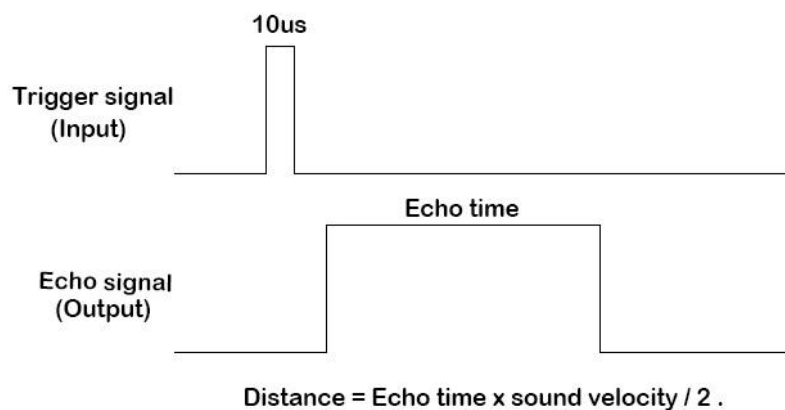
Working voltage: 5V

Working current: 12mA

Minimum measured distance: 2cm

Maximum measured distance: 200cm

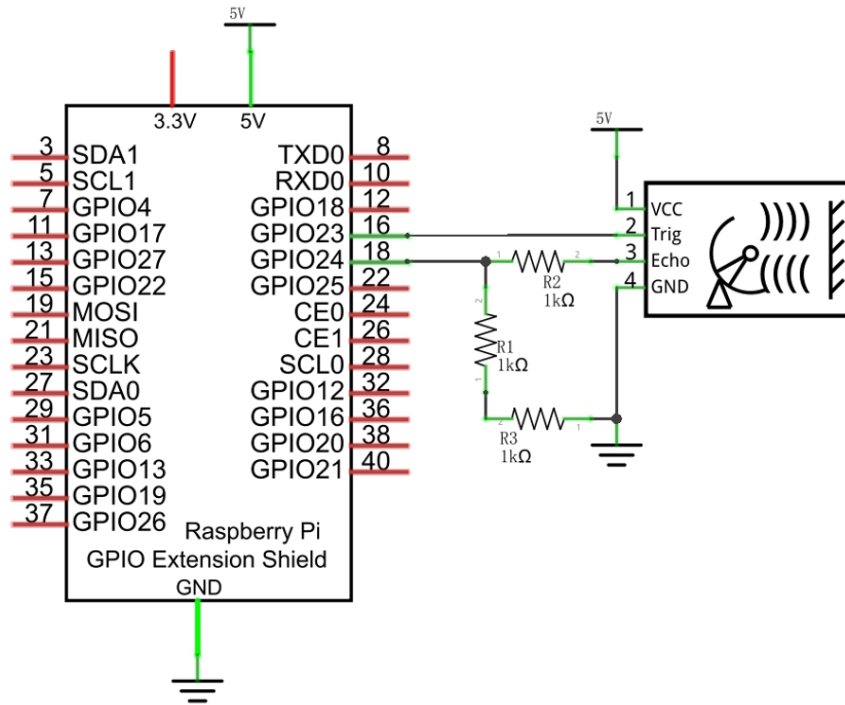
Instructions for Use: output a high-level pulse in Trig pin lasting for least 10uS, the module begins to transmit ultrasonic waves. At the same time, the Echo pin is pulled up. When the module receives the returned ultrasonic waves from encountering an obstacle, the Echo pin will be pulled down. The duration of high level in the Echo pin is the total time of the ultrasonic wave from transmitting to receiving, $s=vt/2$. This is done constantly.



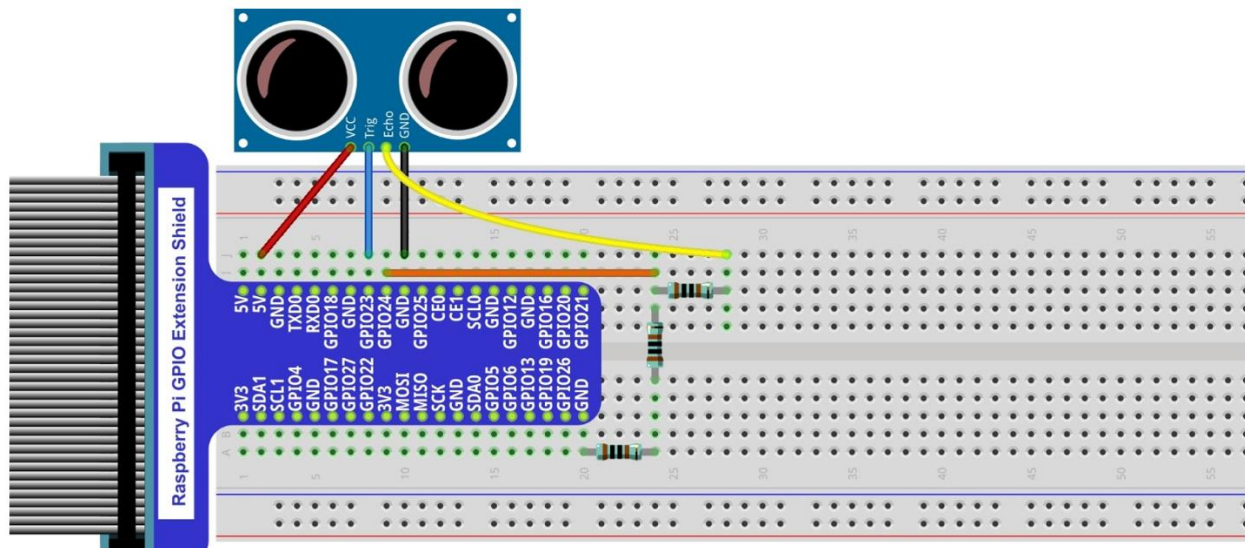
Circuit

Note that the voltage of ultrasonic module is 5V in this circuit.

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Video: <https://www.youtube.com/watch?v=6nffzIN-IUE>

Code

Python Code 21.1.1 UltrasonicRanging

First, observe the project result, and then learn about the code in detail.

If you have any concerns, please contact us via: support@freenove.com

1. Use cd command to enter 21.1.1_UltrasonicRanging directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOWireless/21.1.1_UltrasonicRanging
```

2. Use Python command to execute code "UltrasonicRanging.py".

```
python UltrasonicRanging.py
```

After the program is executed, aim the Ultrasonic Ranging Module's detectors ("eyes") perpendicular to the surface of an object (try using your hand). The distance between the ultrasonic module and the object will be displayed in the terminal. As is shown below:

```
The distance is : 198.75 cm
The distance is : 199.22 cm
The distance is : 198.42 cm
The distance is : 198.74 cm
The distance is : 198.37 cm
The distance is : 198.47 cm
The distance is : 198.41 cm
```

The following is the program code:

```
1  from gpiozero import DistanceSensor
2  from time import sleep
3
4  trigPin = 23
5  echoPin = 24
6  sensor = DistanceSensor(echo=echoPin, trigger=trigPin, max_distance=3)
7
8  def loop():
9      while True:
10         print('Distance: ', sensor.distance * 100, ' cm')
11         sleep(1)
12
13  if __name__ == '__main__':    # Program entrance
14     print ('Program is starting...')
15     try:
16         loop()
17     except KeyboardInterrupt:  # Press ctrl-c to end the program.
18         sensor.close()
19         print("Ending program")
```

First, define the pins and the maximum measurement distance.

```
trigPin = 23
echoPin = 24
```

```
sensor = DistanceSensor(echo=echoPin, trigger=trigPin, max_distance=3) # define the maximum
measured distance 300cm
```

Finally, in the while loop of main function, get the measurement distance and display it continually.

```
def loop():
    while True:
        print('Distance: ', sensor.distance * 100, ' cm')
        sleep(1)
```

For more information about the methods used by the DistanceSensor class in the GPIO Zero library, please refer to: https://gpiozero.readthedocs.io/en/stable/api_input.html#distancesensor-hc-sr04

In the above experiments, you can see that the measurement data is unstable.

Note: For improved accuracy, use the pigpio pin driver rather than the default RPi.GPIO driver (pigpio uses DMA sampling for much more precise edge timing). This is particularly relevant if you're using Pi 1 or Pi Zero.

You can refer to UltrasonicRanging2.py for detailed code.

1. Use cd command to enter 15.1.1_Sweep directory of Python code.

```
cd ~/Freenove_Kit/Code/Python_GPIOZero_Code/21.1.1_UltrasonicRanging
```

2. Use python command to execute code "Sweep.py".

```
python UltrasonicRanging2.py
```

After the program is executed, the distance between the ultrasonic module and the measured object will be displayed on the terminal. At this point, the data is more accurate and stable.

This code is based on pigpio library. In the latest Raspberry Pi OS, "pigpio" library has been installed. You only need to run the command to enable it.

```
sudo pigpiod
```

```
pi@raspberrypi:~ $ sudo pigpiod
pi@raspberrypi:~ $
```

If the "pigpio" library has not yet been installed, please follow the steps to install it.

Run the command to install "pigpio" library.

```
sudo apt-get update
```

```
sudo apt-get install pigpio python-pigpio python3-pigpio
```

The following is the program code:

```
1 import os
2 os.system("sudo pigpiod")
3 from gpiozero import DistanceSensor
4 from gpiozero.pins.pigpio import PiGPIOFactory
5 from time import sleep
6
7 trigPin = 23
8 echoPin = 24
9 my_factory = PiGPIOFactory()
```

```
10 sensor = DistanceSensor(echo=echoPin, trigger=trigPin ,max_distance=3,pin_factory=my_factory)
11 def loop():
12     while True:
13         print('Distance: ', sensor.distance * 100, ' cm')
14         sleep(1)
15
16 if __name__ == '__main__':    # Program entrance
17     print ('Program is starting...')
18     try:
19         loop()
20     except KeyboardInterrupt: # Press ctrl-c to end the program.
21         sensor.close()
22         os.system("sudo killall pigpiod")
23         print("Ending program")
```

See Changing the pin factory for further information:

https://gpiozero.readthedocs.io/en/stable/api_pins.html#changing-pin-factory

Chapter 22 Web IoT

In this chapter, we will learn how to use GPIO to control the RPi remotely via a network and how to build a WebIO service on the RPi.

This concept is known as “IoT” or Internet of Things. The development of IoT will greatly change our habits and make our lives more convenient and efficient

Project 22.1 Remote LED

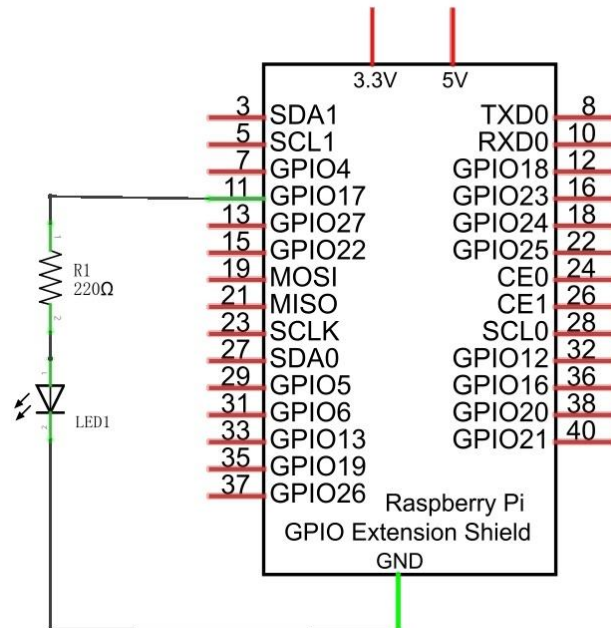
In this project, we need to build a WebIOPi service, and then use the RPi GPIO to control an LED through the web browser of phone or PC.

Component List

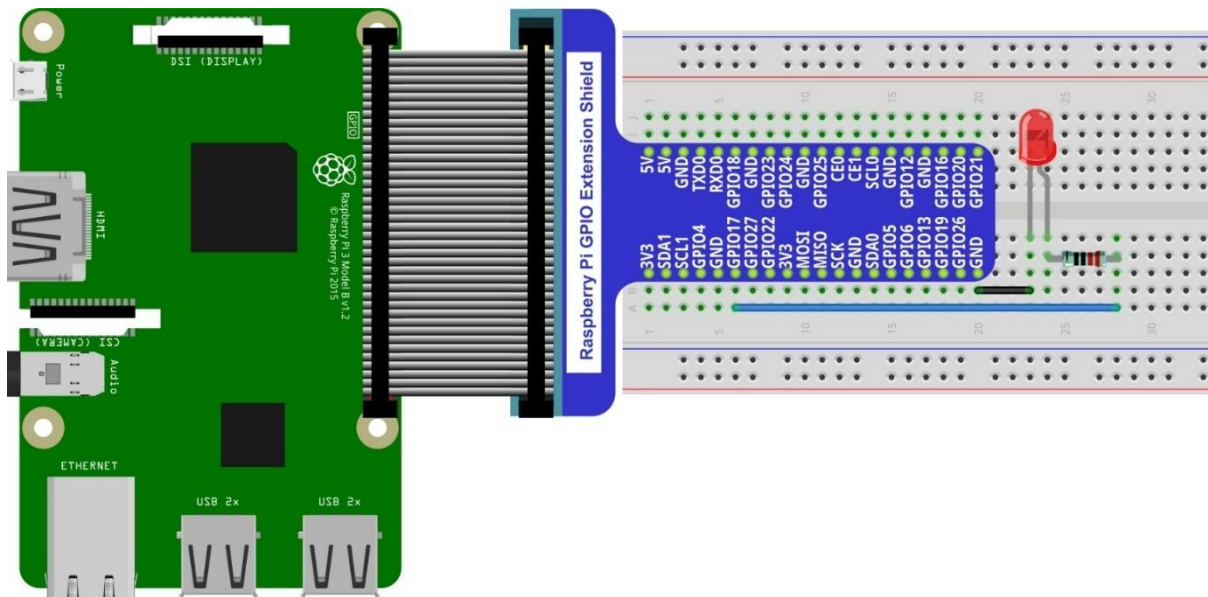
Raspberry Pi (with 40 GPIO) x1 GPIO Extension Board & Ribbon Cable x1 Breadboard x1	LED x1 	Resistor 220Ω x1 
Jumper M/M x2 		

Circuit

Schematic diagram



Hardware connection. If you need any support, please feel free to contact us via: support@freenove.com



Solution from E-Tinkers

Here is a solution from blog E-Tinkers, author Henry Cheung. For more details, please refer to link below:

<https://www.e-tinkers.com/2018/04/how-to-control-raspberry-pi-gpio-via-http-web-server/>

1, Make sure you have set python3 as default python. Then run following command in terminal to install http.server in your Raspberry Pi.

```
sudo apt-get install http.server
```

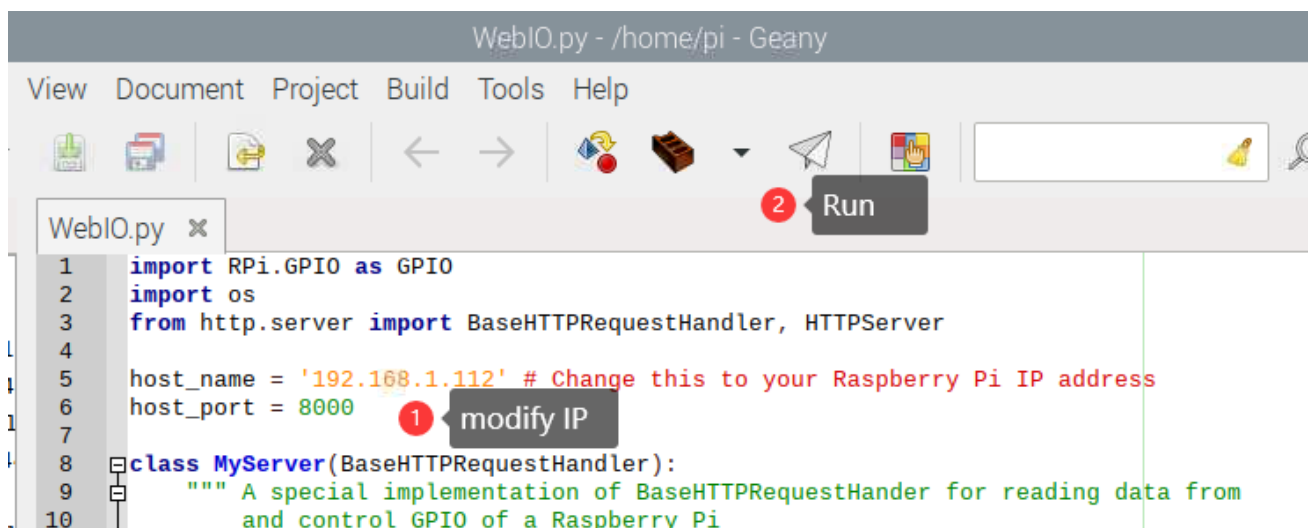
2, Open WebIO.py

```
cd ~/Freenove_Kit/Code/Python_GPIOWZero_Code/22.1.1_WebIO
geany WebIO.py
```

3, Change the host_name into your Raspberry Pi IP address.

```
host_name = '192.168.1.112' # Change this to your Raspberry Pi IP address
```

Then run the code WebIO.py



3, Visit <http://192.168.1.112:8000/> in web browser on computer under local area networks. Change IP to your Raspberry Pi IP address.



WebIOPi Service Framework

Note: If you have a Raspberry Pi 4B, you may have some trouble. The reason for changing the file in the configuration process is that the newer generation models of the RPi CPUs are different from the older ones and you may not be able to access the GPIO Header at the end of this tutorial. A solution to this is given in an online tutorial by from E-Tinkers blogger Henry Cheung. For more details, please refer to previous section.

The following is the key part of this chapter. The installation steps refer to WebIOPi official. And you also can directly refer to the official installation steps. The latest version (in 2016-6-27) of WebIOPi is 0.7.1. So, you may encounter some issues in using it. We will explain these issues and provide the solution in the following installation steps.

Here are the steps to build a WebIOPi:

Installation

1. Get the installation package. You can use the following command to obtain.

```
wget https://github.com/Freenove/WebIOPi/archive/master.zip -O WebIOPi.zip
```

2. Extract the package and generate a folder named "WebIOPi-master". Then enter the folder.

```
unzip WebIOPi.zip
cd WebIOPi-master/WebIOPi-0.7.1
```

3. Patch for Raspberry Pi B+, 2B, 3B, 3B+.

```
patch -p1 -i webiopi-pi2bplus.patch
```

4. Run setup.sh to start the installation, the process takes a while and you will need to be patient.

```
sudo ./setup.sh
```

5. If setup.sh does not have permission to execute, execute the following command

```
sudo sh ./setup.sh
```

Run

After the installation is completed, you can use the webiopi command to start running.

```
$ sudo webiopi [-h] [-c config] [-l log] [-s script] [-d] [port]
```

Options:

-h, --help		Display this help
-c, --config	file	Load config from file
-l, --log	file	Log to file
-s, --script	file	Load script from file
-d, --debug		Enable DEBUG

Arguments:

port	Port to bind the HTTP Server
------	------------------------------

Run webiopi with verbose output and the default config file:

```
sudo webiopi -d -c /etc/webiopi/config
```

The Port is 8000 in default. Now WebIOPi has been launched. Keep it running.

Access WebIOPi over local network

Under the same network, use a mobile phone or PC browser to open your RPi IP address, and add a port number like 8000. For example, my personal Raspberry Pi IP address is 192.168.1.109. Then, in the browser, I then should input: <http://192.168.1.109:8000/>

Default user is "webiopi" and password is "raspberrry".

Then, enter the main control interface:

WebIOPi Main Menu

GPIO Header

Control and Debug the Raspberry Pi GPIO with a display which looks like the physical header.

GPIO List

Control and Debug the Raspberry Pi GPIO ordered in a single column.

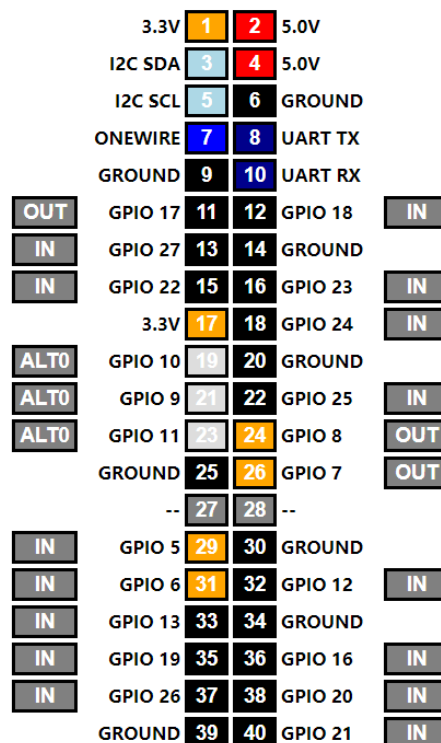
Serial Monitor

Use the browser to play with Serial interfaces configured in WebIOPi.

Devices Monitor

Control and Debug devices and circuits wired to your Pi and configured in WebIOPi.

Click on GPIO Header to enter the GPIO control interface.



Control methods:

- Click/Tap the OUT/IN button to change GPIO direction.
- Click/Tap pins to change the GPIO output state.

Completed

According to the circuit we build, set GPIO17 to OUT, then click Header11 to control the LED. You can end the webioPi in the terminal by "Ctrl+C".

What's Next?

THANK YOU for participating in this learning experience! If you have completed all of the projects successfully you can consider yourself a Raspberry Pi Master.

We have reached the end of this Tutorial. If you find errors, omissions or you have suggestions and/or questions about the Tutorial or component contents of this Kit, please feel free to contact us: support@freenove.com

We will make every effort to make changes and correct errors as soon as feasibly possible and publish a revised version.

If you are interesting in processing, you can study the Processing.pdf in the unzipped folder.

If you want to learn more about Arduino, Raspberry Pi, Smart Cars, Robotics and other interesting products in science and technology, please continue to visit our website. We will continue to launch fun, cost-effective, innovative and exciting products.

<http://www.freenove.com/>

Thank you again for choosing Freenove products.